Outline

Features Page Facelift Outline

New Page Role

Capability Control Center

Purpose

Turn the current Features page from a static registry into a live system capability board that shows:

- what exists
- what is actually implemented
- what is partial
- what is blocked
- what depends on what
- what needs work next

Nav Label

Capabilities

Page Title

Capability Control Center

Subtitle

Replace the current fixed registry subtitle with something dynamic, such as:

- “System capability readiness, dependency health, and implementation coverage”
- “Operational map of Axion subsystems and their build status”

Core Design Goal

The page should answer these questions immediately:

- What capabilities are real right now?
- Which are only planned or partially built?

- Which are blocked by missing dependencies, modules, or gates?
- What is risky to change?
- What should be worked on next?

Page Structure

1. Header Band

Left side

- Page title
- live subtitle
- last sync / last registry refresh timestamp

Right side

Replace the current two-stat block with a richer summary strip:

- Total Capabilities
- Implemented
- Partial
- Spec Only
- Blocked
- At Risk

---

2. Global Control Bar

A control row under the header.

Controls

- Search by name, ID, module, dependency
- Filter by status
- Filter by category
- Filter by readiness
- Filter by risk
- Filter by missing coverage
- Sort by priority / risk / dependency count / gate count / module count

Quick Toggles

- Show only blocked
- Show only incomplete
- Show only unverified
- Show only high dependency impact

---

3. Main View Tabs

Instead of only grouping by category, the page should support multiple views.

Tab A — Overview

High-level readiness board of all capabilities.

Tab B — Readiness

Shows real implementation state:

- Implemented
- Partial
- Stubbed
- Spec Only
- Blocked
- Deprecated

Tab C — Dependencies

Shows dependency chains and downstream impact.

Tab D — Gates

Shows verification and gate coverage.

Tab E — Modules

Shows linked source modules and missing implementation surfaces.

Tab F — Risk

Shows fragile or high-blast-radius capabilities.

---

4. Overview Tab Layout

Top Section

A set of visual summary cards:

- implementation coverage
- verification coverage
- dependency health
- module completeness
- orphaned capabilities
- blocked capabilities

Middle Section

A “Needs Attention” panel:

- blocked capabilities
- capabilities with zero modules
- capabilities with no gates
- capabilities marked active but missing evidence
- dependency conflicts

Bottom Section

A card grid or compact board of capabilities with richer status badges.

---

5. Capability Card Redesign

The current row cards should become more informative and more obviously operational.

Card Header

- capability ID
- title
- implementation state
- risk badge
- category badge

Card Body

- short operational description
- implementation progress bar
- modules linked
- dependencies count
- gates count
- last updated
- owner/domain if available

Card Footer

Quick actions:

- View details
- Open dependency map
- View modules
- View gate coverage
- Review gaps

---

6. New Status Model

The current `active / draft / error` model is too thin for this page purpose.

features

Replace with Two Layers

A. Registry State

- Registered
- Draft
- Deprecated
- Error

B. Implementation State

- Implemented
- Partial
- Stubbed
- Spec Only
- Blocked

- Unverified

This is the key fix. It separates “exists in registry” from “actually built.”

---

7. Readiness Scoring

Each capability should get a readiness score.

Readiness Inputs

- source modules present
- routes/endpoints present
- UI surface present
- gate coverage present
- dependency satisfaction
- test/proof coverage
- no blocking issues

Output

A score or grade:

- Ready
- Near Ready
- Partial
- Fragile
- Blocked

---

8. Dependency View

This should become one of the highest-value sections.

Show

- dependencies required by each capability
- reverse dependencies
- dependency depth
- blocked-by chains
- blast radius if changed

Visual Patterns

- dependency chips
- expandable graph panel
- “X capabilities depend on this” warning

---

9. Gates View

The current page only shows gate counts. It should show gate usefulness instead.

features

Show

- required gates
- present gates
- missing gates
- last run status
- pass/fail/unknown
- verification coverage level

Result

This makes the page connect directly to your test suite and verification system.

---

10. Modules View

The page already exposes `src_modules`, so the facelift should make that actually useful.

features

Show

- linked modules
- missing expected modules
- duplicate ownership
- stale/unlinked modules
- frontend/backend/runtime coverage by capability

Purpose

This turns the page into a bridge between architecture and actual code surfaces.

---

11. Risk Layer

Add a risk dimension to every capability.

Risk Signals

- many reverse dependencies
- missing gates
- zero linked modules
- active but unverified
- partial implementation in critical category
- dependency on unstable capability

Risk Labels

- Low
- Moderate
- High
- Critical

---

12. Empty / Warning States

The page should call out misleading situations clearly.

Examples

- “Marked active, but no linked modules found”
- “Capability registered without gate coverage”
- “Dependency missing from active path”
- “Spec exists, implementation surface not found”

This is important because it directly addresses why the current page feels untrustworthy.

---

13. Visual Direction

The facelift should feel more like a lab control surface than a documentation table.

Style Direction

- darker command-center tone
- stronger hierarchy
- compact but high-signal cards
- segmented status colors
- icon-led health indicators
- progress/readiness bars
- expandable details instead of clutter

Keep

- clean card structure
- category tagging
- strong contrast

Change

- less passive list styling
- less “registry spreadsheet”
- more operational dashboard feel

---

14. Information Architecture Shift

Current

Category list of records.

New

Capability governance surface with:

- summary
- filtering
- readiness
- dependency health
- verification health
- implementation coverage

That is the real facelift.

---

15. MVP Facelift Scope

If you want the first version without overbuilding, phase it like this:

Phase 1

- rename page to Capabilities
- new header stats
- search + filters
- new implementation status model
- redesigned cards
- readiness badge

Phase 2

- tabs: Overview / Dependencies / Gates / Modules
- blocked/incomplete panels
- missing coverage warnings

Phase 3

- dependency map
- readiness scoring
- risk layer
- action shortcuts into detail pages and test surfaces

---

16. Success Criteria

The facelift is successful when the page can answer, at a glance:

- what is actually built
- what is only on paper
- what is blocked
- what is risky
- what should be done next

That is the page's new job.

Capability Control Center

Capability Control Center — Build Spec

Capability Control Center — Build Spec

0. Objective

Replace the current **Feature Registry** page with a more truthful and more actionable control surface.

1. New Page Identity

Nav label

Capabilities

Route

Keep current route if needed for now:
`/features`

Later optional rename:
`/capabilities`

Page title

Capability Control Center

Page subtitle

Dynamic subtitle, not hardcoded.
Examples:

- “Capability readiness, dependency health, and implementation coverage”
- “Operational map of Axion subsystems and their current build state”

This replaces the current hardcoded subtitle `17 subsystem contracts powering the Axion pipeline`.

features

---

2. Purpose

The page must answer:

- what capabilities exist
- what is actually implemented
- what is partial or stubbed
- what is blocked
- what has missing gates
- what has missing module coverage
- what is risky to change
- what needs work next

This is a different job than the current grouped registry list.

features

---

3. Page Model

3.1 Current model

The current page uses:

- `feature_id`
- `title`
- `description`
- `category`
- `status`
- `dependencies`
- `src_modules`
- `gates`
  features

3.2 New presentation model

Keep the existing fields, but derive richer UI state from them.

Registry layer

What the registry says the capability is.

Fields:

- `feature_id`
- `title`
- `category`
- `status`
- `description`

Operational layer

What the system believes about implementation quality.

Derived fields:

- `implementation_state`
- `readiness_score`
- `risk_level`
- `dependency_health`
- `gate_health`
- `module_health`
- `attention_flags`

---

4. Status Model Rewrite

4.1 Problem

The current page uses only:

- active
- draft
- error
  features

That is too shallow. “active” is not the same as “implemented.”

4.2 New two-layer status system

A. Registry Status

Keep:

- active
- draft
- error
- deprecated

B. Implementation Status

Add derived status:

- implemented
- partial
- stubbed
- spec_only
- blocked
- unverified

The page should always show both, but visually emphasize implementation status.

---

5. Header Spec

5.1 Layout

Two-row header.

Row 1

- page title
- subtitle
- optional “last synced” timestamp

Row 2

summary stat strip

5.2 Summary cards

Replace current `Total` and `Active` only.

features

Required cards:

- Total Capabilities
- Implemented
- Partial
- Blocked
- Unverified
- High Risk

Optional:

- Missing Gates
- Missing Modules
- Orphaned Specs

---

6. Global Control Bar

6.1 Required controls

- search input
- category filter
- registry status filter
- implementation status filter
- risk filter
- sort dropdown

6.2 Quick toggles

- only blocked
- only incomplete
- only unverified
- only high impact
- only missing gates
- only missing modules

6.3 Sort options

- title
- feature id
- readiness descending

- risk descending
- dependency count descending
- gates count descending
- module count descending

---

7. Main Layout Structure

The current page is category groups stacked vertically.

features

Replace with tabbed views.

7.1 Tabs

- Overview
- Readiness
- Dependencies
- Gates
- Modules
- Risk

Default tab:
Overview

---

8. Overview Tab Spec

8.1 Purpose

This is the default operational dashboard.

8.2 Sections

A. Summary strip

Uses the header metrics.

B. Needs Attention panel

Show capabilities with:

- blocked implementation state
- zero `src_modules`
- zero `gates`
- active registry status but low readiness
- dependency conflicts
- error registry state

C. Capability board

Grid or list of redesigned capability cards.

D. Coverage callouts

Small highlight cards:

- X capabilities missing modules
- X capabilities missing gates
- X capabilities marked active but not implemented
- X capabilities with high dependency blast radius

---

9. Readiness Tab Spec

9.1 Purpose

Provide a truth-based implementation board.

9.2 Groupings

Group capabilities by implementation status:

- Implemented
- Partial
- Stubbed
- Spec Only
- Blocked
- Unverified

9.3 Card presentation

Each group contains cards with:

- title
- feature id
- readiness score
- status reasons
- missing pieces
- quick actions

---

10. Dependencies Tab Spec

10.1 Purpose

Make dependency relationships visible and actionable.

10.2 Required content

For each capability show:

- direct dependencies
- reverse dependencies
- dependency count
- blocked-by list
- impact count

10.3 States

Dependency health:

- healthy
- warning
- blocked
- unknown

10.4 UX pattern

Each card row should expand to reveal:

- depends on
- depended on by

- risk note
- impact summary

Example labels:

- “Blocked by 2 missing dependencies”
- “Used by 5 downstream capabilities”

---

11. Gates Tab Spec

11.1 Purpose

Turn `gates.length` into meaningful verification visibility. The current page only shows count.

features

11.2 Required content

For each capability:

- gate count
- required gates
- present gates
- missing gates
- verification state
- last evidence state if available later

11.3 Gate health states

- covered
- partial
- missing
- failed
- unknown

11.4 Callouts

Highlight:

- active capabilities with no gates

- critical capabilities with weak coverage
- capabilities blocked on verification

---

12. Modules Tab Spec

12.1 Purpose

Turn `src_modules` from a count into a real code-coverage signal.

features

12.2 Required content

For each capability:

- linked modules list
- module count
- expected implementation surfaces
- missing module warning
- frontend/backend/runtime tags if available later

12.3 Module health states

- linked
- partial
- missing
- orphaned
- stale

12.4 Important warning state

If capability is registry `active` and `src_modules.length === 0`, show:
“Marked active but has no linked source modules.”

---

13. Risk Tab Spec

13.1 Purpose

Surface fragile parts of the system.

13.2 Risk inputs

Derived from:

- reverse dependency count
- no modules
- no gates
- blocked dependencies
- error status
- active but low readiness

13.3 Risk levels

- low
- moderate
- high
- critical

13.4 Risk panel sections

- highest blast radius
- active but unsupported
- blocked critical path
- weak verification coverage

---

14. Capability Card Spec

14.1 Replace current card

The current card shows:

- icon from status
- feature id
- title
- status pill

- category pill
- modules / gates / deps counts
- description
- chevron navigation
  features

Keep the good parts, but restructure the card into a more operational layout.

14.2 New card anatomy

Top row

- feature id
- title
- implementation state badge
- risk badge

Meta row

- category badge
- registry status badge
- readiness score
- dependency health badge

Description row

- one-line or two-line summary

Metrics row

- modules
- gates
- dependencies
- blockers
- warnings

Footer row

actions:

- view detail
- view deps
- view gates
- view modules

---

15. Readiness Scoring Spec

15.1 Goal

Provide one condensed signal for how real a capability is.

15.2 Inputs

Initial version can use only current fields:

- registry status
- `src_modules.length`
- `gates.length`
- `dependencies.length`

Later version can add:

- route coverage
- API coverage
- proof/test evidence
- last validation result

15.3 Initial heuristic

Example simple scoring:

- base from registry status
    - modules present
    - gates present
    - blocked dependencies
    - error state
    - active with zero modules/gates

15.4 Output labels

- Ready
- Near Ready
- Partial
- Fragile

- Blocked

---

16. Derived Rule Pack

These are the first rules the page should compute.

Rule 01

If `status === "active"` and `src_modules.length === 0`
→ flag `active_without_modules`

Rule 02

If `status === "active"` and `gates.length === 0`
→ flag `active_without_gates`

Rule 03

If `status === "draft"` and `src_modules.length > 0`
→ flag `implementation_ahead_of_registry`

Rule 04

If `dependencies.length > 0` and one or more dependencies are unresolved
→ implementation state `blocked`

Rule 05

If `status === "error"`
→ risk `high`

Rule 06

If `src_modules.length > 0` and `gates.length > 0` but status is not active
→ attention flag `registry_state_review`

---

17. Empty / Warning State Spec

The new page must explain misleading or thin data.

Required warnings:

- “No capabilities found for current filter set.”
- “Capability is registered but has no linked implementation modules.”
- “Capability has implementation modules but no gate coverage.”
- “Capability is marked active but appears incomplete.”
- “Dependency references exist but downstream health is unknown.”

---

18. Visual Style Spec

18.1 Direction

This should feel like a control surface, not a document list.

18.2 Required visual traits

- strong header hierarchy
- denser signal per card
- visible status layering
- warning and blocker emphasis
- compact control bar
- card hover states that imply drill-down

18.3 Keep from current page

- dark-panel card language
- badge-based metadata
- category grouping logic as fallback
- clickable rows/cards

18.4 Replace from current page

- fixed subtitle
- only two summary metrics
- passive category-only grouping
- binary active/non-active icon logic

---

19. Data Contract Additions

You can start with derived UI state only, but the page will get much better if the API later includes:

- `implementation_state`
- `readiness_score`
- `risk_level`
- `missing_dependencies`
- `missing_modules`
- `missing_gates`
- `last_verified_at`
- `owner`
- `last_updated_at`

These do not have to block the facelift. Phase 1 can derive them client-side.

---

20. Interaction Rules

Click on card

Open detail page, same as now.

Click dependency metric

Open dependencies drawer or tab focus.

Click gate metric

Open gates drawer or tab focus.

Click module metric

Open modules drawer or tab focus.

Click warning badge

Filter page to similar problem cases.

---

21. MVP Build Scope

Phase 1

- rename page to Capability Control Center
- replace subtitle
- add full stat strip
- add search and filters
- redesign cards
- add implementation state
- add readiness and warning flags

Phase 2

- add tabs
- add Needs Attention panel
- add dependency and gate views
- add module health warnings

Phase 3

- add risk layer
- add deeper dependency impact view
- add richer API-backed readiness and verification data

---

22. Acceptance Criteria

The page is successful when it can immediately show:

- what is actually built
- what is partial
- what is only registered on paper
- what is blocked
- what is risky
- what deserves attention next

If the page still only answers “what records exist in `/api/features`,” then the facelift failed.

Capability Control Center — Page Section Skeleton

Capability Control Center — Page Section Skeleton

1. Page Shell

Top-level wrapper for the entire page.

Structure

- `PageShell`
  - `PageHeader`
  - `SummaryStatStrip`
  - `GlobalControlBar`
  - `PrimaryTabBar`
  - `ActiveTabPanel`

---

2. PageHeader

Replaces the current “Feature Registry” header and static subtitle.

features

Contents

- page title: `Capability Control Center`
- dynamic subtitle
- optional last synced / registry refresh timestamp
- optional right-side page actions later

Skeleton

- `PageHeader`
  - `TitleBlock`
    - `PageTitle`
    - `PageSubtitle`
    - `MetaLine`

    - `HeaderActions` optional

---

3. SummaryStatStrip

Replaces the current two-card `Total / Active` strip.

features

Contents

- Total Capabilities
- Implemented
- Partial
- Blocked
- Unverified
- High Risk

Skeleton

- `SummaryStatStrip`
    - `StatCard total`
    - `StatCard implemented`
    - `StatCard partial`
    - `StatCard blocked`
    - `StatCard unverified`
    - `StatCard highRisk`

---

4. GlobalControlBar

New top-level operator controls.

Contents

- search input
- category filter
- registry status filter
- implementation status filter
- risk filter

- sort dropdown
- quick toggles

Skeleton

- `GlobalControlBar`
  - `SearchInput`
  - `FilterGroup`
    - `CategoryFilter`
    - `RegistryStatusFilter`
    - `ImplementationStatusFilter`
    - `RiskFilter`
  - `SortControl`
  - `QuickToggleRow`
    - `BlockedOnlyToggle`
    - `IncompleteOnlyToggle`
    - `UnverifiedOnlyToggle`
    - `MissingGatesOnlyToggle`
    - `MissingModulesOnlyToggle`

---

5. PrimaryTabBar

Replaces category-only browsing as the main organizing pattern. The current page groups only by category order and then renders stacked lists.

features

Tabs

- Overview
- Readiness
- Dependencies
- Gates
- Modules
- Risk

Skeleton

- `PrimaryTabBar`
  - `Tab overview`

    - `Tab readiness`
    - `Tab dependencies`
    - `Tab gates`
    - `Tab modules`
    - `Tab risk`

TAB PANELS

6. OverviewTabPanel

Default landing view.

Purpose

Fast operational snapshot of the full capability registry.

Sections

- `AttentionPanel`
- `CoverageCalloutRow`
- `CapabilityBoard`

Skeleton

- `OverviewTabPanel`
    - `AttentionPanel`
        - `BlockedList`
        - `MissingModulesList`
        - `MissingGatesList`
        - `ActiveButWeakList`
    - `CoverageCalloutRow`
        - `CoverageCallout missingModules`
        - `CoverageCallout missingGates`
        - `CoverageCallout activeButIncomplete`
        - `CoverageCallout highBlastRadius`
    - `CapabilityBoard`
        - repeated `CapabilityCard`

---

7. ReadinessTabPanel

Truth-based implementation grouping.

Purpose

Show what is actually built vs only registered.

Sections

- `ReadinessSummaryRow`
- `ReadinessColumns`

Skeleton

- `ReadinessTabPanel`
  - `ReadinessSummaryRow`
  - `ReadinessColumns`
    - `ReadinessColumn implemented`
    - `ReadinessColumn partial`
    - `ReadinessColumn stubbed`
    - `ReadinessColumn specOnly`
    - `ReadinessColumn blocked`
    - `ReadinessColumn unverified`

Each column contains repeated `CapabilityCardCompact`.

---

8. DependenciesTabPanel

Dependency map and blast-radius view.

Purpose

Show blocked-by chains and downstream impact.

Sections

- `DependencyHealthSummary`

- `DependencyBoard`

Skeleton

- `DependenciesTabPanel`
  - `DependencyHealthSummary`
  - `DependencyBoard`
    - repeated `DependencyCard`

DependencyCard skeleton

- `DependencyCard`
  - `CardHeader`
  - `DependencyHealthBadge`
  - `DependsOnRow`
  - `DependedOnByRow`
  - `BlockerRow`
  - `ImpactRow`
  - `CardActions`

---

9. GatesTabPanel

Verification and gate coverage view.

Purpose

Make gate coverage readable instead of only showing a gate count like the current card does.

features

Sections

- `GateCoverageSummary`
- `GateWarningsPanel`
- `GateBoard`

Skeleton

- `GatesTabPanel`

    - `GateCoverageSummary`
    - `GateWarningsPanel`
        - `NoGatesWarningList`
        - `WeakCoverageWarningList`
        - `BlockedVerificationWarningList`
    - `GateBoard`
        - repeated `GateCoverageCard`

GateCoverageCard skeleton

- `GateCoverageCard`
    - `CardHeader`
    - `GateHealthBadge`
    - `RequiredGatesRow`
    - `PresentGatesRow`
    - `MissingGatesRow`
    - `VerificationStateRow`
    - `CardActions`

---

10. ModulesTabPanel

Implementation surface view.

Purpose

Turn `src_modules` into a real code linkage panel instead of just a count. The current card only prints the number of modules.

features

Sections

- `ModuleCoverageSummary`
- `ModuleWarningsPanel`
- `ModuleBoard`

Skeleton

- `ModulesTabPanel`

    - `ModuleCoverageSummary`
    - `ModuleWarningsPanel`
        - `NoModulesWarningList`
        - `OrphanedModuleList`
        - `PartialCoverageList`
    - `ModuleBoard`
        - repeated `ModuleCoverageCard`

ModuleCoverageCard skeleton

- `ModuleCoverageCard`
    - `CardHeader`
    - `ModuleHealthBadge`
    - `LinkedModulesRow`
    - `ExpectedSurfaceRow`
    - `CoverageTagsRow`
    - `WarningsRow`
    - `CardActions`

---

11. RiskTabPanel

Fragility and change-impact view.

Purpose

Surface what is dangerous to touch or falsely marked as healthy.

Sections

- `RiskSummaryRow`
- `HighRiskPanel`
- `RiskBoard`

Skeleton

- `RiskTabPanel`
    - `RiskSummaryRow`
    - `HighRiskPanel`
        - `CriticalRiskList`

        - `HighBlastRadiusList`
        - `ActiveButUnsupportedList`
        - `BlockedCriticalPathList`
    - `RiskBoard`
        - repeated `RiskCard`

SHARED COMPONENTS

12. CapabilityCard

Main reusable card for Overview and other panels.

The current page already uses a clickable card/button with:

- feature id
- title
- status badge
- category badge
- modules/gates/deps counts
- description
- chevron nav
  features

This should become the core shared component.

New skeleton

- `CapabilityCard`
    - `CardHeader`
        - `FeatureId`
        - `Title`
        - `ImplementationStateBadge`
        - `RiskBadge`
    - `CardMetaRow`
        - `CategoryBadge`
        - `RegistryStatusBadge`
        - `ReadinessScore`
        - `DependencyHealthBadge`

    - `CardDescription`
    - `CardMetricsRow`
        - `MetricChip modules`
        - `MetricChip gates`
        - `MetricChip dependencies`
        - `MetricChip blockers`
        - `MetricChip warnings`
    - `CardFooter`
        - `Action view`
        - `Action deps`
        - `Action gates`
        - `Action modules`

---

13. CapabilityCardCompact

Smaller board card for dense readiness columns.

Skeleton

- `CapabilityCardCompact`
    - `TitleRow`
    - `StateBadgeRow`
    - `MiniMetricsRow`
    - `WarningLine`

---

14. AttentionPanel

Top-priority issue zone.

Skeleton

- `AttentionPanel`
    - `PanelHeader`
    - `AttentionGrid`
        - `AttentionCard blocked`
        - `AttentionCard missingModules`

        - `AttentionCard missingGates`
        - `AttentionCard activeButIncomplete`

---

15. CoverageCalloutRow

High-signal small cards.

Skeleton

- `CoverageCalloutRow`
    - `CalloutCard missingModules`
    - `CalloutCard missingGates`
    - `CalloutCard activeButIncomplete`
    - `CalloutCard highImpact`

---

16. EmptyState / WarningState

The new page needs explicit honesty states because the old page can make registry entries look more complete than they are.

Skeleton

- `EmptyState`
    - icon
    - title
    - explanation
    - reset filters action
- `WarningBanner`
    - severity badge
    - warning text
    - optional action

Required warning banners

- active but no linked modules
- active but no gates
- registered but implementation unknown
- dependency health unresolved

LAYOUT ORDER

17. Final Render Order

This is the actual top-to-bottom page skeleton.

- `PageShell`
  - `PageHeader`
  - `SummaryStatStrip`
  - `GlobalControlBar`
  - `PrimaryTabBar`
  - `TabContentArea`
    - if `overview` → `OverviewTabPanel`
    - if `readiness` → `ReadinessTabPanel`
    - if `dependencies` → `DependenciesTabPanel`
    - if `gates` → `GatesTabPanel`
    - if `modules` → `ModulesTabPanel`
    - if `risk` → `RiskTabPanel`

MINIMUM FIRST BUILD

18. MVP Skeleton

If you want the first rewrite pass without building everything at once:

- `PageHeader`
- `SummaryStatStrip`
- `GlobalControlBar`
- `PrimaryTabBar`
- `OverviewTabPanel`
  - `AttentionPanel`
  - `CoverageCalloutRow`
  - `CapabilityBoard`

        - `CapabilityCard`

That is the cleanest first cut.

19. Direct mapping from current page

Current page pieces map like this:

- current header → `PageHeader`
- current total/active cards → `SummaryStatStrip`
- current category stacks → move into `OverviewTabPanel`
- current flat feature button → evolve into `CapabilityCard`
- current `/features/:feature_id` navigation → keep as primary drill-in path

Capability Control Center — Component Breakdown

Capability Control Center — Component Breakdown Spec

1. Top-Level Page Component

`CapabilitiesPage`

Role

Top-level orchestration component for the full page.

Responsibilities

- fetch capability data
- hold page-level UI state
- compute derived metrics
- apply search/filter/sort
- choose active tab
- pass data into section components

Inputs

None directly

Internal state

- `activeTab`
- `searchQuery`
- `categoryFilter`
- `registryStatusFilter`
- `implementationStatusFilter`
- `riskFilter`
- `sortMode`
- quick-toggle flags

Outputs

Renders full page shell

---

2. Data / View-Model Layer

`useCapabilitiesPageModel`

Role

Central hook that transforms raw registry records into UI-ready capability objects.

Responsibilities

- take `/api/features` response
- compute derived statuses
- compute stat strip values
- compute warnings
- compute readiness and risk
- produce filtered/sorted lists per tab

Input

Raw `Feature[]`

Output

`CapabilityPageModel`

Suggested shape

```
type CapabilityPageModel = {
 raw: Feature[];
 derived: CapabilityRecord[];
 stats: CapabilityStats;
 filtered: CapabilityRecord[];
 groupedByReadiness: Record<string, CapabilityRecord[]>;
 groupedByCategory: Record<string, CapabilityRecord[]>;
 warnings: CapabilityWarnings;
};
```

---

3. Core Type Layer

`Feature`

Keep current API shape:

- `feature_id`
- `title`
- `description`
- `category`
- `status`
- `dependencies`
- `src_modules`
- `gates`
  features

`CapabilityRecord`

Derived UI object.

Fields

- all original `Feature` fields
- `registry_status`
- `implementation_status`
- `readiness_score`
- `readiness_label`
- `risk_level`
- `dependency_health`
- `gate_health`
- `module_health`
- `attention_flags`
- `blocker_count`
- `warning_count`

`CapabilityStats`

Fields

- `total`
- `implemented`

- `partial`
- `blocked`
- `unverified`
- `highRisk`
- `missingModules`
- `missingGates`

LAYOUT COMPONENTS

4. `CapabilitiesPageHeader`

Role

Owns top identity band for the page.

Responsibilities

- render page title
- render dynamic subtitle
- render last sync text if available

Props

```
type CapabilitiesPageHeaderProps = {
 title: string;
 subtitle: string;
 lastSyncedLabel?: string;
};
```

---

5. `CapabilityStatStrip`

Role

Owns summary cards row.

Responsibilities

- render high-level stats
- allow optional click-to-filter later

Props

```
type CapabilityStatStripProps = {
 stats: CapabilityStats;
 onSelectStat?: (key: string) => void;
};
```

Children

- `CapabilityStatCard`

---

6. `CapabilityStatCard`

Role

Single summary card.

Responsibilities

- render value
- render label
- render optional delta/warning state

Props

```
type CapabilityStatCardProps = {
 label: string;
 value: number;
 tone?: "default" | "good" | "warn" | "bad";
 onClick?: () => void;
};
```

---

7. `CapabilityControlBar`

Role

Owns the search/filter/sort row.

Responsibilities

- search
- filter selection
- toggle selection
- sort selection

Props

```
type CapabilityControlBarProps = {
 searchQuery: string;
 filters: CapabilityFilterState;
 sortMode: CapabilitySortMode;
 onSearchChange: (value: string) => void;
 onFiltersChange: (next: CapabilityFilterState) => void;
 onSortChange: (mode: CapabilitySortMode) => void;
};
```

Children

- `CapabilitySearchInput`
- `CapabilityFilterGroup`
- `CapabilityQuickToggles`
- `CapabilitySortSelect`

---

8. `CapabilityTabs`

Role

Owns top-level tab navigation.

Responsibilities

- render tabs
- switch active panel

Props

```
type CapabilityTabsProps = {
 activeTab: CapabilityTabId;
 onTabChange: (tab: CapabilityTabId) => void;
};
```

TAB PANEL COMPONENTS

9. `OverviewPanel`

Role

Default dashboard tab.

Responsibilities

- render attention summary
- render coverage callouts
- render main capability board

Props

```
type OverviewPanelProps = {
 capabilities: CapabilityRecord[];
 warnings: CapabilityWarnings;
 onOpenCapability: (featureId: string) => void;
};
```

Children

- `AttentionPanel`
- `CoverageCalloutRow`
- `CapabilityBoard`

---

10. `ReadinessPanel`

Role

Shows implementation-state grouping.

Responsibilities

- group capabilities by implementation status

- render readiness lanes/columns

Props

```
type ReadinessPanelProps = {
 grouped: Record<ImplementationStatus, CapabilityRecord[]>;
 onOpenCapability: (featureId: string) => void;
};
```

Children

- `ReadinessColumn`
- `CapabilityCardCompact`

---

11. `DependenciesPanel`

Role

Shows dependency health and impact.

Responsibilities

- render dependency summary
- render dependency cards

Props

```
type DependenciesPanelProps = {
 capabilities: CapabilityRecord[];
 onOpenCapability: (featureId: string) => void;
};
```

Children

- `DependencyHealthSummary`
- `DependencyBoard`
- `DependencyCard`

---

12. `GatesPanel`

Role

Shows verification coverage.

Responsibilities

- summarize gate coverage
- list missing or weak coverage
- render gate cards

Props

```
type GatesPanelProps = {
 capabilities: CapabilityRecord[];
 onOpenCapability: (featureId: string) => void;
};
```

Children

- `GateCoverageSummary`
- `GateWarningsPanel`
- `GateCoverageCard`

---

13. `ModulesPanel`

Role

Shows implementation surface linkage.

Responsibilities

- summarize module linkage
- call out no-module / orphan / partial cases
- render module cards

Props

```
type ModulesPanelProps = {
 capabilities: CapabilityRecord[];
 onOpenCapability: (featureId: string) => void;
};
```

Children

- `ModuleCoverageSummary`
- `ModuleWarningsPanel`
- `ModuleCoverageCard`

---

14. `RiskPanel`

Role

Shows fragile and high-impact capabilities.

Responsibilities

- render risk summary
- render high-risk subsets
- render risk cards

Props

```
type RiskPanelProps = {
 capabilities: CapabilityRecord[];
 onOpenCapability: (featureId: string) => void;
};
```

Children

- `RiskSummaryRow`
- `HighRiskPanel`
- `RiskCard`

SHARED DISPLAY COMPONENTS

15. `CapabilityBoard`

Role

Reusable board/list wrapper for full cards.

Responsibilities

- layout capability cards
- render empty state if none

Props

```
type CapabilityBoardProps = {
 capabilities: CapabilityRecord[];
 onOpenCapability: (featureId: string) => void;
};
```

---

16. `CapabilityCard`

Role

Primary reusable capability card.

This replaces the current inline card/button that shows ID, title, status, category, counts, and description.

features

Responsibilities

- render core identity
- render layered status
- render metrics
- render warnings
- expose actions

Props

```
type CapabilityCardProps = {
 capability: CapabilityRecord;
 onOpen: (featureId: string) => void;
 onOpenDeps?: (featureId: string) => void;
 onOpenGates?: (featureId: string) => void;
 onOpenModules?: (featureId: string) => void;
};
```

Children

- `CapabilityCardHeader`
- `CapabilityCardMeta`
- `CapabilityCardMetrics`
- `CapabilityCardWarnings`
- `CapabilityCardActions`

---

17. `CapabilityCardCompact`

Role

Dense card for readiness columns.

Responsibilities

- show condensed status
- show minimal metrics
- show top warning

Props

```
type CapabilityCardCompactProps = {
 capability: CapabilityRecord;
 onOpen: (featureId: string) => void;
};
```

---

18. `CapabilityBadge`

Role

Reusable badge primitive.

Responsibilities

- render category
- render registry status
- render implementation status
- render risk

Props

```
type CapabilityBadgeProps = {
 label: string;
 kind: "category" | "registry" | "implementation" | "risk" | "health";
};
```

---

19. `MetricChip`

Role

Reusable compact metric token.

Responsibilities

- show count + label
- optionally be clickable

Props

```
type MetricChipProps = {
 label: string;
 value: number;
 tone?: "neutral" | "good" | "warn" | "bad";
 onClick?: () => void;
};
```

---

20. `WarningPill`

Role

Reusable warning indicator.

Responsibilities

- surface attention flags
- optionally filter on click

Props

```
type WarningPillProps = {
 label: string;
 severity: "info" | "warn" | "error";
```

```
 onClick?: () => void;
};
```

OVERVIEW-SPECIFIC COMPONENTS

21. `AttentionPanel`

Role

Top-level alert zone for the default tab.

Responsibilities

- show most important issues first

Props

```
type AttentionPanelProps = {
 warnings: CapabilityWarnings;
 onSelectSlice?: (slice: string) => void;
};
```

Children

- `AttentionCard blocked`
- `AttentionCard missingModules`
- `AttentionCard missingGates`
- `AttentionCard activeButIncomplete`

---

22. `CoverageCalloutRow`

Role

Quick high-signal status row.

Responsibilities

- show compact counts of important gaps

Props

```
type CoverageCalloutRowProps = {
 stats: CapabilityStats;
 onSelectCallout?: (kind: string) => void;
};
```

DEPENDENCY COMPONENTS

23. `DependencyCard`

Role

Dependency-focused capability card.

Responsibilities

- render direct deps
- render reverse deps
- render impact count
- render blocker note

Props

```
type DependencyCardProps = {
 capability: CapabilityRecord;
 onOpen: (featureId: string) => void;
};
```

---

24. `DependencyHealthSummary`

Role

Summary strip for dependency health.

Responsibilities

- blocked dependencies count
- healthy count
- unknown count

- highest blast radius count

Props

```
type DependencyHealthSummaryProps = {
 capabilities: CapabilityRecord[];
};
```

GATE COMPONENTS

25. `GateCoverageCard`

Role

Gate/verification-focused card.

Responsibilities

- show required vs present
- show gate health
- show missing coverage

Props

```
type GateCoverageCardProps = {
 capability: CapabilityRecord;
 onOpen: (featureId: string) => void;
};
```

---

26. `GateWarningsPanel`

Role

Surfaces weak verification coverage.

Responsibilities

- list no-gates cases
- list partial coverage cases
- list blocked verification cases

Props

```
type GateWarningsPanelProps = {
 capabilities: CapabilityRecord[];
 onSelectCapability: (featureId: string) => void;
};
```

MODULE COMPONENTS

27. `ModuleCoverageCard`

Role

Module linkage-focused card.

Responsibilities

- show linked modules
- show module health
- show implementation-surface warnings

Props

```
type ModuleCoverageCardProps = {
 capability: CapabilityRecord;
 onOpen: (featureId: string) => void;
};
```

---

28. `ModuleWarningsPanel`

Role

Surfaces missing-code-linkage cases.

Responsibilities

- no linked modules
- partial module linkage
- orphan-like cases

Props

```
type ModuleWarningsPanelProps = {
 capabilities: CapabilityRecord[];
 onSelectCapability: (featureId: string) => void;
};
```

RISK COMPONENTS

29. `RiskCard`

Role

Risk-focused card.

Responsibilities

- render risk level
- render risk reasons
- render blast-radius note
- render weak-support warnings

Props

```
type RiskCardProps = {
 capability: CapabilityRecord;
 onOpen: (featureId: string) => void;
};
```

---

30. `HighRiskPanel`

Role

Top summary block for dangerous capabilities.

Responsibilities

- critical risk slice
- active but unsupported
- blocked critical path

- high blast radius

Props

```
type HighRiskPanelProps = {
 capabilities: CapabilityRecord[];
 onSelectCapability: (featureId: string) => void;
};
```

UTILITY COMPONENTS

31. `CapabilitiesEmptyState`

Role

Shared empty state.

Responsibilities

- render when filters return no capabilities
- offer reset action

Props

```
type CapabilitiesEmptyStateProps = {
 title: string;
 message: string;
 onReset?: () => void;
};
```

---

32. `CapabilityWarningBanner`

Role

Shared honesty banner for misleading registry states.

Responsibilities

Show states like:

- marked active but no modules
- marked active but no gates
- implementation unknown
- unresolved dependency health

Props

```
type CapabilityWarningBannerProps = {
 label: string;
 severity: "info" | "warn" | "error";
};
```

HOOKS / HELPERS

33. `useCapabilityFilters`

Role

Own filter state and filtered outputs.

Responsibilities

- apply text search
- apply category/status/risk filters
- apply quick toggles

---

34. `useCapabilitySorting`

Role

Own sort logic.

Responsibilities

- sort by title
- sort by readiness
- sort by risk
- sort by dependency count

- sort by module count
- sort by gate count

---

35. `deriveImplementationStatus`

Role

Compute implementation truth state from raw feature metadata.

Inputs

`Feature`

Output

`ImplementationStatus`

---

36. `deriveReadinessScore`

Role

Compute readiness numeric and label.

Inputs

`Feature` or `CapabilityRecord`

Output

{ score: number; label: ReadinessLabel }

---

37. `deriveRiskLevel`

Role

Compute risk class.

Inputs

`CapabilityRecord`

Output

`RiskLevel`

---

38. `collectAttentionFlags`

Role

Create warning labels from rule pack.

Inputs

`Feature`

Output

`string[]`

BUILD ORDER

39. Recommended implementation order

Build in this order so the page becomes usable early:

1. `CapabilitiesPage`
2. `useCapabilitiesPageModel`
3. `CapabilitiesPageHeader`
4. `CapabilityStatStrip`
5. `CapabilityControlBar`
6. `CapabilityTabs`
7. `CapabilityCard`
8. `CapabilityBoard`

9. `OverviewPanel`
10. `CapabilitiesEmptyState`

Then add:

11. `ReadinessPanel`
12. `DependenciesPanel`
13. `GatesPanel`
14. `ModulesPanel`
15. `RiskPanel`

---

40. Direct mapping from current page

Current inline logic should be extracted like this:

- current `useQuery("/api/features")` → `CapabilitiesPage` / `useCapabilitiesPageModel`
- current `activeCount` → `CapabilityStatStrip`
- current `groupByCategory()` → fallback helper inside model layer
- current inline button card → `CapabilityCard`
- current category sections → temporary `OverviewPanel` grouping until new tabs fully land

Capability Control Center — Data Schema Pack

Capability Control Center — Data Schema Pack

1. Base API Record

This is the current raw record shape coming from `/api/features`.

features

```
export interface Feature {
 feature_id: string;
 title: string;
 description: string;
 category: string;
 status: string;
 dependencies: string[];
 src_modules: string[];
 gates: string[];
}
```

This stays as the raw input contract.

---

2. Locked Enum Set

2.1 Category

Start with current known categories from the page and allow extension. The current page explicitly recognizes `infrastructure`, `core-logic`, `interface`, and `security`.

```
export type CapabilityCategory =
 | "infrastructure"
 | "core-logic"
 | "interface"
 | "security"
 | "other";
```

2.2 Registry Status

The current page only colors `active`, `draft`, and `error`. Add `deprecated` now so the new page has a stable archival state.

features

```
export type RegistryStatus =
 | "active"
 | "draft"
 | "error"
 | "deprecated";
```

2.3 Implementation Status

This is the new truth layer.

```
export type ImplementationStatus =
 | "implemented"
 | "partial"
 | "stubbed"
 | "spec_only"
 | "blocked"
 | "unverified";
```

2.4 Readiness Label

```
export type ReadinessLabel =
 | "ready"
 | "near_ready"
 | "partial"
 | "fragile"
 | "blocked";
```

2.5 Risk Level

```
export type RiskLevel =
 | "low"
 | "moderate"
 | "high"
```

```
 | "critical";
```

2.6 Health States

```
export type DependencyHealth =
 | "healthy"
 | "warning"
 | "blocked"
 | "unknown";

export type GateHealth =
 | "covered"
 | "partial"
 | "missing"
 | "failed"
 | "unknown";

export type ModuleHealth =
 | "linked"
 | "partial"
 | "missing"
 | "orphaned"
 | "stale"
 | "unknown";
```

2.7 Tab IDs

```
export type CapabilityTabId =
 | "overview"
 | "readiness"
 | "dependencies"
 | "gates"
 | "modules"
 | "risk";
```

2.8 Sort Modes

```
export type CapabilitySortMode =
 | "title_asc"
 | "feature_id_asc"
 | "readiness_desc"
```

```
 | "risk_desc"
 | "dependency_count_desc"
 | "gate_count_desc"
 | "module_count_desc";
```

---

3. Derived UI Record

This is the main working record the page should use after transforming raw `Feature`.

```
export interface CapabilityRecord {
 feature_id: string;
 title: string;
 description: string;
 category: CapabilityCategory;
 registry_status: RegistryStatus;
 dependencies: string[];
 src_modules: string[];
 gates: string[];

 implementation_status: ImplementationStatus;
 readiness_score: number;
 readiness_label: ReadinessLabel;
 risk_level: RiskLevel;

 dependency_health: DependencyHealth;
 gate_health: GateHealth;
 module_health: ModuleHealth;

 blocker_count: number;
 warning_count: number;

 attention_flags: AttentionFlag[];
 warnings: CapabilityWarningCode[];

 reverse_dependency_count: number;
 unresolved_dependencies: string[];
 missing_gates: string[];
 missing_modules: string[];

 last_verified_at?: string | null;
 last_updated_at?: string | null;
 owner?: string | null;
}
```

---

4. Attention Flag Set

This gives the page a stable warning vocabulary.

```
export type AttentionFlag =
 | "active_without_modules"
 | "active_without_gates"
 | "active_but_low_readiness"
 | "implementation_ahead_of_registry"
 | "registry_state_review"
 | "unresolved_dependencies"
 | "missing_verification"
 | "missing_implementation_surface"
 | "high_blast_radius"
 | "error_state";
```

---

5. Warning Code Set

This is the display-safe warning layer.

```
export type CapabilityWarningCode =
 | "no_modules"
 | "no_gates"
 | "dependency_unknown"
 | "dependency_blocked"
 | "implementation_unknown"
 | "marked_active_but_incomplete"
 | "orphan_registry_entry"
 | "weak_verification"
 | "high_risk_dependency_chain";
```

---

6. Stats Contract

The current page only computes `features.length` and `activeCount`. The facelift needs a fuller stat contract.

features

```
export interface CapabilityStats {
```

```
 total: number;
 implemented: number;
 partial: number;
 blocked: number;
 unverified: number;
 highRisk: number;

 active: number;
 draft: number;
 error: number;
 deprecated: number;

 missingModules: number;
 missingGates: number;
 unresolvedDependencies: number;
 activeButIncomplete: number;
}
```

---

7. Filter State Contract

```
export interface CapabilityFilterState {
 category: CapabilityCategory | "all";
 registryStatus: RegistryStatus | "all";
 implementationStatus: ImplementationStatus | "all";
 risk: RiskLevel | "all";

 blockedOnly: boolean;
 incompleteOnly: boolean;
 unverifiedOnly: boolean;
 missingGatesOnly: boolean;
 missingModulesOnly: boolean;
 highImpactOnly: boolean;
}
```

---

8. Page Model Contract

This is the output of `useCapabilitiesPageModel`.

```
export interface CapabilityPageModel {
 raw: Feature[];
 derived: CapabilityRecord[];
 filtered: CapabilityRecord[];
```

```
 stats: CapabilityStats;

 groupedByCategory: Record<string, CapabilityRecord[]>;
 groupedByReadiness: Record<ImplementationStatus, CapabilityRecord[]>;
 groupedByRisk: Record<RiskLevel, CapabilityRecord[]>;

 warnings: CapabilityWarnings;
}
```

---

9. Warning Bucket Contract

```
export interface CapabilityWarnings {
 blocked: CapabilityRecord[];
 missingModules: CapabilityRecord[];
 missingGates: CapabilityRecord[];
 activeButIncomplete: CapabilityRecord[];
 highRisk: CapabilityRecord[];
 unresolvedDependencies: CapabilityRecord[];
}
```

---

10. Dependency Contracts

The current page only shows dependency count. The facelift needs structure for dependency health.

features

```
export interface CapabilityDependencyState {
 feature_id: string;
 depends_on: string[];
 depended_on_by: string[];
 unresolved_dependencies: string[];
 dependency_health: DependencyHealth;
 reverse_dependency_count: number;
}
```

---

11. Gate Coverage Contracts

The current page only shows gate count.

features

```
export interface CapabilityGateState {
 feature_id: string;
 gates_present: string[];
 gates_required: string[];
 missing_gates: string[];
 gate_health: GateHealth;
 verification_state: "covered" | "partial" | "missing" | "unknown";
 last_verified_at?: string | null;
}
```

For phase 1, `gates_required` can equal `gates_present` until a richer backend contract exists.

---

12. Module Coverage Contracts

The current page only shows `src_modules.length`.

features

```
export interface CapabilityModuleState {
 feature_id: string;
 linked_modules: string[];
 expected_modules: string[];
 missing_modules: string[];
 module_health: ModuleHealth;
}
```

For phase 1, `expected_modules` can be empty and `missing_modules` can be derived conservatively.

---

13. Display Token Contracts

These help keep the UI consistent.

```
export interface BadgeToken {
 label: string;
 tone: "neutral" | "good" | "warn" | "bad" | "info";
}
```

```
export interface MetricToken {
 label: string;
 value: number;
 tone: "neutral" | "good" | "warn" | "bad";
}

export interface WarningToken {
 code: CapabilityWarningCode;
 label: string;
 severity: "info" | "warn" | "error";
}
```

---

14. Rule Pack Inputs

The current page already uses raw counts and `status === "active"` to present health. That is exactly what we need to replace with explicit rules.

```
export interface CapabilityRuleContext {
 feature: Feature;
 reverseDependencyCount: number;
 unresolvedDependencies: string[];
}
```

---

15. Rule Pack Outputs

```
export interface CapabilityRuleResult {
 implementation_status: ImplementationStatus;
 readiness_score: number;
 readiness_label: ReadinessLabel;
 risk_level: RiskLevel;
 dependency_health: DependencyHealth;
 gate_health: GateHealth;
 module_health: ModuleHealth;
 attention_flags: AttentionFlag[];
 warnings: CapabilityWarningCode[];
}
```

---

16. Locked Derivation Rules

These are the minimum rule contracts for phase 1.

Rule 01

If `registry_status === "active"` and `src_modules.length === 0`
→ add `active_without_modules`
→ add `no_modules`
→ module health = `missing`

Rule 02

If `registry_status === "active"` and `gates.length === 0`
→ add `active_without_gates`
→ add `no_gates`
→ gate health = `missing`

Rule 03

If `registry_status === "draft"` and `src_modules.length > 0`
→ add `implementation_ahead_of_registry`

Rule 04

If unresolved dependencies exist
→ implementation status = `blocked`
→ dependency health = `blocked`
→ add `dependency_blocked`

Rule 05

If `registry_status === "error"`
→ risk level at least `high`
→ add `error_state`

Rule 06

If modules and gates are both present, and no unresolved dependencies exist
→ implementation status cannot be `spec_only`

Rule 07

If modules are absent and gates are absent
→ implementation status defaults to `spec_only`

Rule 08

If modules are present but gates absent
→ implementation status defaults to `partial` or `unverified`

---

17. Initial Readiness Scoring Contract

Keep this simple and deterministic for the first pass.

```
export interface ReadinessScoreResult {
 score: number; // 0..100
 label: ReadinessLabel;
}
```

Suggested phase 1 heuristic

- start at 0
- +30 if registry status is active
- +25 if modules exist
- +20 if gates exist
- +15 if no unresolved dependencies
- +10 if not error
- clamp 0..100

Label mapping

- 85–100 → `ready`
- 65–84 → `near_ready`
- 40–64 → `partial`
- 15–39 → `fragile`
- 0–14 → `blocked`

---

18. Lookup Maps

The current page already relies on display maps for category and status colors. Keep that pattern, but formalize it.

features

```
export const CAPABILITY_CATEGORY_LABELS: Record<CapabilityCategory, string> = {
 infrastructure: "Infrastructure",
 "core-logic": "Core Logic",
 interface: "Interface",
 security: "Security",
 other: "Other",
};

export const REGISTRY_STATUS_LABELS: Record<RegistryStatus, string> = {
 active: "Active",
 draft: "Draft",
 error: "Error",
 deprecated: "Deprecated",
};

export const IMPLEMENTATION_STATUS_LABELS: Record<ImplementationStatus, string> = {
 implemented: "Implemented",
 partial: "Partial",
 stubbed: "Stubbed",
 spec_only: "Spec Only",
 blocked: "Blocked",
 unverified: "Unverified",
};
```

---

19. Future API Extension Contract

These are not required for phase 1, but this is the correct long-term contract for the backend to expose.

```
export interface CapabilityApiRecord extends Feature {
 implementation_status?: ImplementationStatus;
 readiness_score?: number;
 risk_level?: RiskLevel;
 unresolved_dependencies?: string[];
 missing_gates?: string[];
 missing_modules?: string[];
```

```
 reverse_dependency_count?: number;
 last_verified_at?: string | null;
 last_updated_at?: string | null;
 owner?: string | null;
}
```

That keeps phase 1 client-derived, but leaves room for server-backed truth later.

---

20. Locked File Roles

Here's how I'd split this into code files:

src/types/capabilities.ts
src/lib/capabilities/deriveImplementationStatus.ts
src/lib/capabilities/deriveReadinessScore.ts
src/lib/capabilities/deriveRiskLevel.ts
src/lib/capabilities/collectAttentionFlags.ts
src/lib/capabilities/buildCapabilityPageModel.ts

---

21. Minimal Type Pack

If you want the shortest possible first implementation pack, lock these first:

- `Feature`
- `CapabilityRecord`
- `CapabilityStats`
- `CapabilityFilterState`
- `ImplementationStatus`
- `RiskLevel`
- `CapabilityWarnings`

That is enough to build the new page shell and Overview tab.

---

22. Straight read on why this matters

Right now the page can only say "this registry record is active" because that is all the contract supports, and the UI turns that into a green healthy-looking signal.

This schema pack fixes that by separating:

- registry truth
- implementation truth
- risk truth
- readiness truth

That separation is the foundation of the whole facelift.

Capability Control Center — Hook Spec

Capability Control Center — Hook Spec

1. Hook Layer Goal

Create one clean transformation path from raw `/api/features` records into a stable page model that every tab can trust.

That means the hook layer must own:

- fetch state
- filter state
- sort state
- tab state
- derived capability truth
- summary stats
- grouped outputs
- warning buckets

The UI should render from hook output, not recalculate logic inside panels.

---

2. Hook Stack Overview

Primary hooks

- `useCapabilitiesPage`
- `useCapabilitiesPageModel`
- `useCapabilityFilters`
- `useCapabilitySorting`
- `useCapabilityTabs`

Optional split hooks later

- `useCapabilityWarnings`
- `useCapabilityStats`
- `useCapabilityGrouping`

For phase 1, keep the logic centralized so it does not fragment too early.

---

3. `useCapabilitiesPage`

Role

Top-level orchestration hook for the page.

Responsibilities

- fetch raw records from `/api/features`
- own UI state
- call model hook
- expose page-ready data and actions

Input

None

Output

```
type UseCapabilitiesPageResult = {
 isLoading: boolean;
 error: unknown;
 activeTab: CapabilityTabId;
 setActiveTab: (tab: CapabilityTabId) => void;

 searchQuery: string;
 setSearchQuery: (value: string) => void;

 filters: CapabilityFilterState;
 setFilters: (next: CapabilityFilterState) => void;
 patchFilters: (patch: Partial<CapabilityFilterState>) => void;
 resetFilters: () => void;

 sortMode: CapabilitySortMode;
 setSortMode: (mode: CapabilitySortMode) => void;

 model: CapabilityPageModel;
};
```

Internal flow

1. call `useQuery<Feature[]>({ queryKey: ["/api/features"] ... })`
2. own page-local state
3. pass raw records + state into `useCapabilitiesPageModel`
4. return unified result

---

4. `useCapabilitiesPageModel`

Role

Central derivation hook for the whole page.

Responsibilities

- normalize raw `Feature[]`
- derive `CapabilityRecord[]`
- compute stats
- apply filters
- apply sorting
- compute grouped views
- compute warning buckets

Input

```
type UseCapabilitiesPageModelArgs = {
 features: Feature[];
 searchQuery: string;
 filters: CapabilityFilterState;
 sortMode: CapabilitySortMode;
};
```

Output

```
type UseCapabilitiesPageModelResult = CapabilityPageModel;
```

Internal pipeline

This hook should run in this exact order:

1. normalize raw records
2. build reverse dependency map
3. derive capability records
4. compute stats from derived records
5. filter derived records
6. sort filtered records
7. group sorted records for tabs
8. build warning buckets

That order matters because filtering/sorting should happen on derived truth, not raw registry fields.

---

5. `useCapabilityTabs`

Role

Own active tab state.

Responsibilities

- store current tab
- expose tab setter
- optionally sync to URL later

Input

Optional initial tab

Output

```
type UseCapabilityTabsResult = {
 activeTab: CapabilityTabId;
 setActiveTab: (tab: CapabilityTabId) => void;
};
```

Default

`overview`

---

6. `useCapabilityFilters`

Role

Own search + filter state and filtering logic.

Responsibilities

- hold all filter values
- reset filters
- patch partial filter changes
- apply text and flag filters to `CapabilityRecord[]`

State shape

Use the schema already locked:

```
type CapabilityFilterState = {
 category: CapabilityCategory | "all";
 registryStatus: RegistryStatus | "all";
 implementationStatus: ImplementationStatus | "all";
 risk: RiskLevel | "all";

 blockedOnly: boolean;
 incompleteOnly: boolean;
 unverifiedOnly: boolean;
 missingGatesOnly: boolean;
 missingModulesOnly: boolean;
 highImpactOnly: boolean;
};
```

Hook output

```
type UseCapabilityFiltersResult = {
 searchQuery: string;
 setSearchQuery: (value: string) => void;
 filters: CapabilityFilterState;
```

```
 setFilters: (next: CapabilityFilterState) => void;
 patchFilters: (patch: Partial<CapabilityFilterState>) => void;
 resetFilters: () => void;
 applyFilters: (records: CapabilityRecord[]) => CapabilityRecord[];
};
```

---

7. `useCapabilitySorting`

Role

Own sorting state and sorted outputs.

Responsibilities

- hold current sort mode
- sort derived records consistently

Input

Sorted records should always be filtered records, not raw records.

Output

```
type UseCapabilitySortingResult = {
 sortMode: CapabilitySortMode;
 setSortMode: (mode: CapabilitySortMode) => void;
 applySort: (records: CapabilityRecord[]) => CapabilityRecord[];
};
```

Important rule

Sorting must be stable.
If two records tie on the active sort mode, fall back to:

1. `readiness_score desc`
2. `title asc`
3. `feature_id asc`

That prevents cards from jumping around unnecessarily.

---

8. `useCapabilityWarnings`

Role

Build page-level warning buckets.

Responsibilities

- slice derived records into high-signal groups
- provide `AttentionPanel` and warning panels with direct inputs

Input

`CapabilityRecord[]`

Output

type UseCapabilityWarningsResult = CapabilityWarnings;

Buckets

- blocked
- missingModules
- missingGates
- activeButIncomplete
- highRisk
- unresolvedDependencies

---

9. `useCapabilityStats`

Role

Compute the stat strip from derived records.

Responsibilities

- produce all summary counts from `CapabilityRecord[]`

Input

`CapabilityRecord[]`

Output

type UseCapabilityStatsResult = CapabilityStats;

Important rule

Stats should be based on **all derived records**, not the filtered set, unless you intentionally build a “filtered results stats” mode later.

That keeps the header stable while the board changes.

---

10. `useCapabilityGrouping`

Role

Build grouped outputs for each tab.

Responsibilities

- group by category
- group by readiness
- group by risk

Input

Filtered and sorted `CapabilityRecord[]`

Output

type UseCapabilityGroupingResult = {

```
 groupedByCategory: Record<string, CapabilityRecord[]>;
 groupedByReadiness: Record<ImplementationStatus, CapabilityRecord[]>;
 groupedByRisk: Record<RiskLevel, CapabilityRecord[]>;
};
```

DERIVATION HELPERS

11. `deriveCapabilityRecord`

Role

Convert one raw `Feature` into one derived `CapabilityRecord`.

Input

```
type DeriveCapabilityRecordArgs = {
 feature: Feature;
 reverseDependencyCount: number;
 unresolvedDependencies: string[];
};
```

Output

```
type DeriveCapabilityRecordResult = CapabilityRecord;
```

Steps

1. normalize category
2. normalize registry status
3. compute dependency health
4. compute gate health
5. compute module health
6. compute implementation status
7. compute readiness score + label
8. compute risk level
9. collect attention flags
10. collect warnings
11. compute blocker count
12. compute warning count

This helper is the core truth engine for the page.

---

12. `deriveImplementationStatus`

Role

Determine actual build state from registry metadata.

Input

```
type DeriveImplementationStatusArgs = {
 registryStatus: RegistryStatus;
 srcModulesCount: number;
 gatesCount: number;
 unresolvedDependencyCount: number;
};
```

Output

`ImplementationStatus`

Suggested rules

- unresolved dependencies > 0 → `blocked`
- modules = 0 and gates = 0 → `spec_only`
- modules > 0 and gates = 0 and registry active → `unverified`
- modules > 0 and gates = 0 and registry draft → `partial`
- modules > 0 and gates > 0 and registry active → `implemented`
- modules > 0 and gates > 0 and registry not active → `partial`
- fallback → `stubbed`

This replaces the current oversimplified “active means healthy” presentation.

features

---

13. `deriveDependencyHealth`

Role

Compute dependency state.

Input

```
type DeriveDependencyHealthArgs = {
 dependencies: string[];
 unresolvedDependencies: string[];
};
```

Output

`DependencyHealth`

Rules

- unresolved dependencies > 0 → `blocked`
- dependencies.length === 0 → `healthy`
- dependencies.length > 0 and unresolved = 0 → `healthy`
- uncertain resolution path later → `unknown`

---

14. `deriveGateHealth`

Role

Compute verification coverage state from gate data.

Input

```
type DeriveGateHealthArgs = {
 gates: string[];
 registryStatus: RegistryStatus;
};
```

Output

`GateHealth`

Rules

- gates.length === 0 and registry active → `missing`
- gates.length === 0 and registry draft → `unknown`
- gates.length > 0 → `covered`

Phase 1 is intentionally simple because the current page only exposes gate presence, not pass/fail evidence.

features

---

15. `deriveModuleHealth`

Role

Compute implementation linkage state.

Input

```
type DeriveModuleHealthArgs = {
 srcModules: string[];
 registryStatus: RegistryStatus;
};
```

Output

`ModuleHealth`

Rules

- srcModules.length === 0 and registry active → `missing`
- srcModules.length === 0 and registry draft → `unknown`
- srcModules.length > 0 → `linked`

Again, keep phase 1 conservative until expected-module contracts exist.

---

16. `deriveReadinessScore`

Role

Compute readiness number and label.

Input

```
type DeriveReadinessScoreArgs = {
 registryStatus: RegistryStatus;
 srcModulesCount: number;
 gatesCount: number;
 unresolvedDependencyCount: number;
};
```

Output

```
type DeriveReadinessScoreResult = {
 score: number;
 label: ReadinessLabel;
};
```

Phase 1 heuristic

- start at 0
- +30 if registry active
- +25 if modules > 0
- +20 if gates > 0
- +15 if unresolved dependencies = 0
- +10 if registry status is not error
- clamp 0..100

Mapping

- 85–100 → `ready`
- 65–84 → `near_ready`
- 40–64 → `partial`
- 15–39 → `fragile`
- 0–14 → `blocked`

---

17. `deriveRiskLevel`

Role

Compute fragility and blast-radius signal.

Input

```
type DeriveRiskLevelArgs = {
 registryStatus: RegistryStatus;
 reverseDependencyCount: number;
 unresolvedDependencyCount: number;
 srcModulesCount: number;
 gatesCount: number;
 readinessScore: number;
};
```

Output

`RiskLevel`

Suggested rules

- registry error → at least `high`
- reverseDependencyCount >= 5 → increase one band
- active with zero modules or zero gates → at least `high`
- unresolved dependencies > 0 → at least `high`
- readinessScore < 40 → at least `moderate`
- default → `low`

---

18. `collectAttentionFlags`

Role

Build warning/attention codes from the locked rule pack.

Input

```
type CollectAttentionFlagsArgs = {
 registryStatus: RegistryStatus;
 implementationStatus: ImplementationStatus;
 srcModulesCount: number;
 gatesCount: number;
 unresolvedDependencyCount: number;
 reverseDependencyCount: number;
 readinessScore: number;
};
```

Output

```
type CollectAttentionFlagsResult = {
 attentionFlags: AttentionFlag[];
 warnings: CapabilityWarningCode[];
};
```

Minimum rules

- active + no modules → `active_without_modules`
- active + no gates → `active_without_gates`
- active + readiness < 65 → `active_but_low_readiness`
- draft + modules > 0 → `implementation_ahead_of_registry`
- unresolved deps > 0 → `unresolved_dependencies`
- reverse deps >= threshold → `high_blast_radius`
- registry error → `error_state`

PAGE-MODEL PIPELINE

19. Exact `useCapabilitiesPageModel` Pipeline

Step 1 — normalize raw features

Map raw records into safe defaults:

- missing category → `other`

- missing arrays → `[ ]`
- missing description → empty string

Step 2 — build feature ID set

Create a `Set<string>` of all feature IDs.

Step 3 — build reverse dependency map

For each feature:

- inspect `dependencies`
- increment reverse counts on the referenced feature IDs

Step 4 — resolve unresolved dependencies

For each feature:

- compare `dependencies` against the feature ID set
- any missing feature ID becomes unresolved

Step 5 — derive capability records

Call `deriveCapabilityRecord` for each feature.

Step 6 — compute stats

Build `CapabilityStats` from all derived records.

Step 7 — apply filters

Search + filter on derived records.

Step 8 — apply sort

Sort filtered records.

Step 9 — group outputs

Build:

- `groupedByCategory`
- `groupedByReadiness`
- `groupedByRisk`

Step 10 — build warnings

Slice derived records into warning buckets.

Step 11 — return page model

Return final `CapabilityPageModel`

---

20. Memoization Rules

Use `useMemo` around:

- normalized features
- reverse dependency map
- derived records
- stats
- filtered records
- sorted records
- grouped outputs
- warning buckets

This page will otherwise recalculate a lot as search/filter state changes.

Important dependency rule

Do not memoize at panel level first.
Memoize at the model layer first.

---

21. Search Rules

Search should match against:

- `feature_id`
- `title`
- `description`
- `category`
- `src_modules`
- `dependencies`
- `gates`

That makes the page feel like a real control surface instead of title-only search.

Matching mode

Phase 1:

- case-insensitive substring match

---

22. Filter Rules

Filters should apply in this order:

1. category
2. registry status
3. implementation status
4. risk
5. quick toggles
6. search query

This order is mostly for readability and debugging.

Quick-toggle rules

- `blockedOnly` → `implementation_status === "blocked"`
- `incompleteOnly` → implementation in `partial | stubbed | spec_only | blocked | unverified`
- `unverifiedOnly` → `implementation_status === "unverified"`
- `missingGatesOnly` → `gate_health === "missing"`
- `missingModulesOnly` → `module_health === "missing"`
- `highImpactOnly` → `reverse_dependency_count >= threshold`

---

23. Sort Rules

Sort implementations

- `title_asc`
- `feature_id_asc`
- `readiness_desc`

- `risk_desc`
- `dependency_count_desc`
- `gate_count_desc`
- `module_count_desc`

Stable tie-breaker

Always fall back to:

1. `title asc`
2. `feature_id asc`

---

24. Query Hook Contract

The current page fetches with React Query directly in the component. Keep that foundation, but move it into the hook-owned page flow.

features

Query spec

```
useQuery<Feature[]>({
 queryKey: ["/api/features"],
 queryFn: () => apiRequest("/api/features"),
});
```

Returned defaults

The hook should always normalize to:

- `features = []`
- `isLoading`
- `error`

The UI should never have to defend against `undefined` feature arrays.

---

25. Hook File Layout

Recommended files:

src/hooks/capabilities/useCapabilitiesPage.ts
src/hooks/capabilities/useCapabilitiesPageModel.ts
src/hooks/capabilities/useCapabilityFilters.ts
src/hooks/capabilities/useCapabilitySorting.ts
src/hooks/capabilities/useCapabilityTabs.ts

Derivation helpers stay in:

src/lib/capabilities/deriveCapabilityRecord.ts
src/lib/capabilities/deriveImplementationStatus.ts
src/lib/capabilities/deriveDependencyHealth.ts
src/lib/capabilities/deriveGateHealth.ts
src/lib/capabilities/deriveModuleHealth.ts
src/lib/capabilities/deriveReadinessScore.ts
src/lib/capabilities/deriveRiskLevel.ts
src/lib/capabilities/collectAttentionFlags.ts

---

26. Minimum Phase 1 Hook Build

If you want the shortest usable implementation path, build only these first:

- `useCapabilitiesPage`
- `useCapabilitiesPageModel`
- `deriveCapabilityRecord`
- `deriveImplementationStatus`
- `deriveReadinessScore`
- `collectAttentionFlags`

That is enough to support:

- header
- stat strip
- filters
- Overview tab
- capability cards
- honesty warnings

---

27. What this fixes

Right now the page logic is trapped inside one component and the only real health computation is `status === "active"` for the header count and the active icon.

This hook layer fixes that by moving the source of truth to:

- derived capability state
- not raw registry status alone

That is the structural change that makes the facelift real.

Capability Control Center — TSX Build Order

Capability Control Center — TSX Build Order

Phase 1 — Replace the shell first

Build the new page frame before deeper tab logic.

1. `CapabilitiesPage.tsx`

This becomes the new page container replacing the current `FeaturesPage`.

Responsibilities

- call `useCapabilitiesPage`
- render top-level sections in order
- route tab content
- pass open handlers into cards/panels

First render shape

```
<PageShell>
 <CapabilitiesPageHeader />
 <CapabilityStatStrip />
 <CapabilityControlBar />
 <CapabilityTabs />
 <OverviewPanel />
</PageShell>
```

Why first

Because the current page is still monolithic. You need the new shell before any subcomponents have a place to live.

features

Phase 2 — Header and stat strip

Replace the misleading registry framing immediately.

2. `CapabilitiesPageHeader.tsx`

Replace:

- `Feature Registry`
- fixed subtitle text

Current page hardcodes both.

Output

- title: `Capability Control Center`
- dynamic subtitle
- optional last sync line

---

3. `CapabilityStatStrip.tsx`

Replace the current two-card strip:

- Total
- Active

New cards

- Total
- Implemented
- Partial
- Blocked
- Unverified
- High Risk

Child

- `CapabilityStatCard.tsx`

Phase 3 — Control layer

Add real control over the page before rebuilding the board.

4. `CapabilityControlBar.tsx`

New section under the stat strip.

Contents

- search
- category filter
- registry status filter
- implementation status filter
- risk filter
- sort select
- quick toggles

Children

- `CapabilitySearchInput.tsx`
- `CapabilityFilterGroup.tsx`
- `CapabilityQuickToggles.tsx`
- `CapabilitySortSelect.tsx`

---

5. `CapabilityTabs.tsx`

The current page only groups by category and stacks sections vertically.

Replace that with:

- Overview
- Readiness
- Dependencies
- Gates
- Modules
- Risk

For the first pass, only `Overview` has to work.

Phase 4 — Main reusable card system

Build the shared card before building panel variations.

6. `CapabilityCard.tsx`

This is the core replacement for the current inline feature button card. The current card already has the right raw ingredients, just not the right truth model.

features

Internal structure

```
<article>
 <CapabilityCardHeader />
 <CapabilityCardMeta />
 <CapabilityCardDescription />
 <CapabilityCardMetrics />
 <CapabilityCardWarnings />
 <CapabilityCardActions />
</article>
```

Child components

- `CapabilityCardHeader.tsx`
- `CapabilityCardMeta.tsx`
- `CapabilityCardMetrics.tsx`
- `CapabilityCardWarnings.tsx`
- `CapabilityCardActions.tsx`

---

7. Shared primitives

Build these right after the card.

`CapabilityBadge.tsx`

For:

- category
- registry status
- implementation status
- risk

`MetricChip.tsx`

For:

- modules
- gates

- deps
- blockers
- warnings

`WarningPill.tsx`

For:

- active without modules
- active without gates
- unresolved deps
- high blast radius

Phase 5 — Overview tab

Make the default tab actually useful before adding the rest.

8. `OverviewPanel.tsx`

This is the first real content panel.

Structure

```
<OverviewPanel>
 <AttentionPanel />
 <CoverageCalloutRow />
 <CapabilityBoard />
</OverviewPanel>
```

---

9. `AttentionPanel.tsx`

Top issue zone.

Initial slices

- blocked
- missing modules
- missing gates
- active but incomplete

---

10. `CoverageCalloutRow.tsx`

Quick summary cards for:

- missing modules
- missing gates
- active but incomplete
- high impact

---

11. `CapabilityBoard.tsx`

Reusable board container that renders repeated `CapabilityCard`.

First layout

Use a simple vertical stack or responsive grid.
No need to overbuild this first pass.

---

12. `CapabilitiesEmptyState.tsx`

You need this early because once filters exist, empty results become normal.

Phase 6 — Hook wiring into the shell

Once the shell and overview components exist, wire them to the real model.

13. Connect `useCapabilitiesPage`

`CapabilitiesPage.tsx` should now consume:

- loading
- error
- activeTab
- search/filter/sort state
- page model

Render rules

- loading → spinner state
- error → error panel
- success → full shell

This replaces the current inline `isLoading ? spinner : grouped category list` logic.

Phase 7 — Readiness view

After Overview works, add the first alternate truth-based view.

14. `ReadinessPanel.tsx`

Group capabilities by:

- implemented
- partial
- stubbed
- spec_only
- blocked
- unverified

Children

- `ReadinessColumn.tsx`
- `CapabilityCardCompact.tsx`

This is the first tab that directly fixes the “features page feels fake” problem.

Phase 8 — Dependency view

Now add structural insight.

15. `DependenciesPanel.tsx`

Children

- `DependencyHealthSummary.tsx`
- `DependencyBoard.tsx`
- `DependencyCard.tsx`

Purpose

Show:

- depends on
- depended on by
- unresolved deps
- impact count

Phase 9 — Gates view

Then add verification visibility.

16. `GatesPanel.tsx`

Children

- `GateCoverageSummary.tsx`
- `GateWarningsPanel.tsx`
- `GateCoverageCard.tsx`

Purpose

Upgrade the current “X gates” count into real coverage insight. The current page only prints gate count when present.

features

Phase 10 — Modules view

Then turn module count into implementation coverage.

17. `ModulesPanel.tsx`

Children

- `ModuleCoverageSummary.tsx`
- `ModuleWarningsPanel.tsx`

- `ModuleCoverageCard.tsx`

Purpose

Upgrade the current “X modules” count into module health. Right now it only shows the number.

features

Phase 11 — Risk view

Then surface fragility.

18. `RiskPanel.tsx`

Children

- `RiskSummaryRow.tsx`
- `HighRiskPanel.tsx`
- `RiskCard.tsx`

Purpose

Show:

- highest blast radius
- active but unsupported
- blocked critical path
- weak verification coverage

FILE BUILD ORDER

19. Actual recommended file creation order

If you want the smoothest implementation sequence, build files in this exact order:

1. pages/CapabilitiesPage.tsx
2. components/capabilities/CapabilitiesPageHeader.tsx

3. components/capabilities/CapabilityStatStrip.tsx
4. components/capabilities/CapabilityStatCard.tsx
5. components/capabilities/CapabilityControlBar.tsx
6. components/capabilities/CapabilityTabs.tsx
7. components/capabilities/CapabilityBadge.tsx
8. components/capabilities/MetricChip.tsx
9. components/capabilities/WarningPill.tsx
10. components/capabilities/CapabilityCard.tsx
11. components/capabilities/CapabilityBoard.tsx
12. components/capabilities/AttentionPanel.tsx
13. components/capabilities/CoverageCalloutRow.tsx
14. components/capabilities/CapabilitiesEmptyState.tsx
15. components/capabilities/panels/OverviewPanel.tsx
16. hooks/capabilities/useCapabilitiesPage.ts
17. hooks/capabilities/useCapabilitiesPageModel.ts
18. panels/ReadinessPanel.tsx
19. panels/DependenciesPanel.tsx
20. panels/GatesPanel.tsx
21. panels/ModulesPanel.tsx
22. panels/RiskPanel.tsx

FIRST USABLE CHECKPOINT

20. Stop after this milestone first

The first fully useful checkpoint is:

- `CapabilitiesPage`
- `CapabilitiesPageHeader`
- `CapabilityStatStrip`
- `CapabilityControlBar`
- `CapabilityTabs`
- `OverviewPanel`
- `CapabilityCard`
- hook wiring

At that point, the page already stops being a passive grouped registry and becomes a real capability board.

WHAT TO REUSE FROM THE CURRENT PAGE

21. Keep these from the existing implementation

You do not need to throw everything away.

Reuse:

- the `/api/features` query source
  features
- click-through navigation to `/features/:feature_id`
  features
- category color concept
  features
- status color concept
  features
- card-based dark-panel styling language

Replace:

- hardcoded header/subtitle
- Total/Active-only stat logic
- category-only page organization
- active/non-active icon as the main truth signal

Capability Control Center — Implementation

Capability Control Center — Implementation Sequence Pack

Pass 1 — Lock the page identity

Goal

Stop the page from presenting itself as a simple registry before deeper logic changes land.

Change

Replace:

- page title
- subtitle
- stat strip labels

The current page hardcodes **“Feature Registry”**, a fixed subtitle, and only shows **Total** and **Active**.

Build

- `CapabilitiesPageHeader`
- `CapabilityStatStrip`
- `CapabilityStatCard`

Minimum done state

The page header now reads:

- `Capability Control Center`
- dynamic subtitle
- stat strip placeholders for future metrics

Do not do yet

- no tabs
- no filters
- no card redesign yet

Why this pass first

It immediately fixes the page's framing problem without risking a large behavioral rewrite.

Pass 2 — Extract the page shell

Goal

Break the page out of the monolith.

Change

Move the current all-in-one render into a top-level shell component.

Build

- `CapabilitiesPage.tsx`
- `PageShell` structure
- loading state wrapper
- error state wrapper

Minimum done state

The page renders as:

- header
- stat strip
- control bar placeholder
- tabs placeholder
- content region

Reuse

Keep:

- the existing `/api/features` query source
- current route behavior
- current detail-page navigation path pattern

Why this pass here

Once the shell exists, every later change becomes additive instead of destructive.

Pass 3 — Move logic into the hook layer

Goal

Stop the UI from deriving truth inline.

Change

Take the current inline logic:

- fetch
- `activeCount`
- `groupByCategory`
- `categoryOrder`
  and move it behind hooks/model helpers.

Build

- `useCapabilitiesPage`
- `useCapabilitiesPageModel`
- `deriveCapabilityRecord`
- `deriveImplementationStatus`
- `deriveReadinessScore`
- `collectAttentionFlags`

Minimum done state

The page component only consumes:

- `model`
- `filters`
- `sortMode`
- `activeTab`
- state setters

Success check

No panel or card should calculate implementation truth on its own.

Pass 4 — Replace the current flat card

Goal

Upgrade the main unit of the page.

Change

The current item is a large clickable button showing:

- active/non-active icon
- feature ID
- title
- status badge
- category badge
- modules/gates/deps counts
- description
  features

Replace that with a layered capability card.

Build

- `CapabilityCard`
- `CapabilityBadge`
- `MetricChip`
- `WarningPill`

Minimum done state

Each card now shows:

- feature identity
- registry status
- implementation status
- readiness
- risk
- warnings
- metrics
- actions

Keep

- click-through to `/features/:feature_id`
  features
- dark card language
- badge-based metadata

Why this pass here

This is the point where the page starts feeling like a control surface instead of a registry list.

Pass 5 — Ship the new Overview only

Goal

Get one fully useful tab live before expanding the page.

Change

Replace category-only page browsing with a real default overview.

The current page only organizes capabilities by category stacks.

Build

- `OverviewPanel`
- `AttentionPanel`
- `CoverageCalloutRow`
- `CapabilityBoard`
- `CapabilitiesEmptyState`

Minimum done state

The page can now answer:

- what is blocked
- what is missing modules
- what is missing gates
- what is active but weak
- what needs attention first

Important constraint

Do not build the other tabs yet.
Get Overview right first.

Pass 6 — Add search, filters, and sorting

Goal

Turn the page into an operator surface.

Build

- `CapabilityControlBar`
- `CapabilitySearchInput`
- `CapabilityFilterGroup`
- `CapabilityQuickToggles`
- `CapabilitySortSelect`

Minimum done state

You can:

- search by ID/title/module/dependency/gate
- filter by category
- filter by registry status
- filter by implementation status
- filter by risk
- sort by readiness/risk/module count/gate count/dependency count

Why this pass after Overview

You want the board and derived model to exist first so filters operate on the right truth layer.

Pass 7 — Add tab navigation

Goal

Prepare the page for multiple operational views.

Build

- `CapabilityTabs`

Tabs

- Overview
- Readiness
- Dependencies
- Gates
- Modules
- Risk

Minimum done state

Only `Overview` has to be functional at first.
The other tabs can be disabled or marked as upcoming during the transition.

Why not earlier

Tabs without useful content create dead UI.

Pass 8 — Build the Readiness tab

Goal

Directly solve the “these features aren’t really implemented” problem.

Build

- `ReadinessPanel`
- `ReadinessColumn`
- `CapabilityCardCompact`

Minimum done state

Capabilities are grouped by:

- implemented
- partial
- stubbed

- spec only
- blocked
- unverified

Success check

This tab should instantly reveal the difference between:

- registered capabilities
- implemented capabilities

That is the biggest honesty fix in the whole page.

Pass 9 — Build the Dependencies tab

Goal

Expose dependency chains and change impact.

Build

- `DependenciesPanel`
- `DependencyHealthSummary`
- `DependencyBoard`
- `DependencyCard`

Minimum done state

Each capability can show:

- what it depends on
- what depends on it
- unresolved dependencies
- downstream impact count

Success check

You should be able to tell what is safe to change and what causes blast radius.

Pass 10 — Build the Gates tab

Goal

Turn gate presence into real verification visibility.

The current page only shows a gate count when it exists.

features

Build

- `GatesPanel`
- `GateCoverageSummary`
- `GateWarningsPanel`
- `GateCoverageCard`

Minimum done state

The page can show:

- no gates
- weak coverage
- blocked verification
- gate health state

Success check

This tab should make “X gates” mean something operational.

Pass 11 — Build the Modules tab

Goal

Turn module count into code-surface coverage.

The current page only shows `src_modules.length`.

features

Build

- `ModulesPanel`
- `ModuleCoverageSummary`
- `ModuleWarningsPanel`
- `ModuleCoverageCard`

Minimum done state

The page can show:

- linked modules
- missing modules
- partial implementation linkage
- active without modules

Success check

You should be able to tell whether a capability has a real implementation footprint.

Pass 12 — Build the Risk tab

Goal

Surface fragile and dangerous capabilities.

Build

- `RiskPanel`
- `RiskSummaryRow`
- `HighRiskPanel`
- `RiskCard`

Minimum done state

The page can show:

- high blast radius
- blocked critical path
- active but unsupported
- low-readiness high-risk capabilities

Success check

This tab should answer: what should not be touched casually?

Pass 13 — Tighten derived rules

Goal

Refine the truth model after the UI structure is in place.

Work

Tune:

- implementation status rules
- readiness scoring
- risk escalation
- attention flag thresholds

Minimum done state

False positives and false confidence are reduced.

Important rule

Do this after the UI exists, not before.
You need to see the model rendered before tuning it.

Pass 14 — Add API-backed enrichment later

Goal

Move from client-derived heuristics toward server-backed truth.

Future additions

- `implementation_status`
- `readiness_score`
- `risk_level`
- `missing_gates`
- `missing_modules`

- `unresolved_dependencies`
- `last_verified_at`
- `last_updated_at`
- `owner`

Minimum done state

Client no longer has to infer everything from:

- `status`
- `src_modules`
- `gates`
- `dependencies` alone

Completion gates by pass

Pass 1 complete when

The page no longer presents itself as plain “Feature Registry.”

Pass 2 complete when

The page shell is separated from the old monolith.

Pass 3 complete when

The model layer owns derivation logic.

Pass 4 complete when

The old flat registry row is gone.

Pass 5 complete when

Overview is useful on its own.

Pass 6 complete when

The page can actually be operated, not just read.

Pass 7 complete when

Tab architecture is in place.

Pass 8 complete when

Implemented vs planned is visible at a glance.

Pass 9–12 complete when

Dependencies, gates, modules, and risk all have dedicated views.

Pass 13 complete when

The truth model feels believable.

Pass 14 complete when

The backend starts providing richer capability truth directly.

Strongest stopping points

Stop Point A

After Pass 5

At that point you already have:

- new identity
- new shell
- hook model
- new card
- real overview

That is the first version worth shipping.

Stop Point B

After Pass 8

At that point you have:

- Overview
- filters
- tabs
- readiness truth

That is the first version that truly fixes the page’s credibility problem.

Stop Point C

After Pass 12

That is the full control-center version

Recommended real build cadence

Sprint 1

Pass 1–5

Sprint 2

Pass 6–8

Sprint 3

Pass 9–12

Sprint 4

Pass 13–14

That pacing keeps the rewrite controlled and testable.

First-Pass File Pack

First-Pass File Pack

Scope covered

This pack covers:

- Pass 1 — page identity
- Pass 2 — page shell
- Pass 3 — hook/model extraction
- Pass 4 — main card replacement
- Pass 5 — Overview tab only

It does **not** include:

- Readiness tab
- Dependencies tab
- Gates tab
- Modules tab
- Risk tab

---

1. Files to create

Page

App/src/pages/capabilities.tsx

Capability page components

App/src/components/capabilities/CapabilitiesPageHeader.tsx
App/src/components/capabilities/CapabilityStatStrip.tsx
App/src/components/capabilities/CapabilityStatCard.tsx
App/src/components/capabilities/CapabilityControlBar.tsx
App/src/components/capabilities/CapabilityTabs.tsx
App/src/components/capabilities/CapabilityBoard.tsx
App/src/components/capabilities/CapabilityCard.tsx
App/src/components/capabilities/CapabilityBadge.tsx
App/src/components/capabilities/MetricChip.tsx

App/src/components/capabilities/WarningPill.tsx
App/src/components/capabilities/AttentionPanel.tsx
App/src/components/capabilities/CoverageCalloutRow.tsx
App/src/components/capabilities/CapabilitiesEmptyState.tsx

Panels

App/src/components/capabilities/panels/OverviewPanel.tsx

Hooks

App/src/hooks/capabilities/useCapabilitiesPage.ts
App/src/hooks/capabilities/useCapabilitiesPageModel.ts

Lib / derivation helpers

App/src/lib/capabilities/deriveCapabilityRecord.ts
App/src/lib/capabilities/deriveImplementationStatus.ts
App/src/lib/capabilities/deriveReadinessScore.ts
App/src/lib/capabilities/collectAttentionFlags.ts
App/src/lib/capabilities/buildCapabilityPageModel.ts

Types

App/src/types/capabilities.ts

---

2. Files to replace or repoint

Replace current page file role

The current `features.tsx` is still the live page and hardcodes:

- `Feature Registry`
- fixed subtitle
- Total / Active stats

- category-only rendering

You have two clean options:

Option A — safest

Keep:

App/src/pages/features.tsx

but convert it into a thin alias that renders the new page container.

Example role:

export { default } from "./capabilities";

Option B — direct replacement

Keep the same file path:

App/src/pages/features.tsx

and rewrite it in place using the new page shell.

For the first pass, I recommend **Option A** because it gives you a cleaner separation.

---

3. Exact responsibility for each file

`App/src/pages/capabilities.tsx`

Top-level page container.

Owns

- `useCapabilitiesPage`
- loading/error guards
- page shell render
- active tab switch

- detail navigation handoff

Renders

- `CapabilitiesPageHeader`
- `CapabilityStatStrip`
- `CapabilityControlBar`
- `CapabilityTabs`
- `OverviewPanel`

`App/src/components/capabilities/CapabilitiesPageHeader.tsx`

Replaces the current hardcoded heading/subtitle.

Owns

- page title
- subtitle
- optional meta line

`App/src/components/capabilities/CapabilityStatStrip.tsx`

Replaces the current `Total` and `Active` stat pair.

Owns

- stat row layout

Uses

- `CapabilityStatCard`

App/src/components/capabilities/CapabilityStatCard.tsx

Single stat tile primitive.

Used for

- Total
- Implemented
- Partial
- Blocked
- Unverified
- High Risk

App/src/components/capabilities/CapabilityControlBar.tsx

New operator control strip.

Owns

- search input
- filters
- sort
- quick toggles

For first pass, this can render simple placeholders for tabs you have not built yet, but the search/filter model should already work.

App/src/components/capabilities/CapabilityTabs.tsx

Top-level tabs.

First-pass rule

Tabs may show:

- Overview = enabled
- all others = disabled / muted / coming soon

That keeps the architecture in place without fake empty views.

`App/src/components/capabilities/CapabilityBoard.tsx`

Board/list wrapper for capability cards.

Owns

- grid or vertical stack layout
- empty-state fallback

Uses

- `CapabilityCard`
- `CapabilitiesEmptyState`

`App/src/components/capabilities/CapabilityCard.tsx`

Core replacement for the current flat registry button. The existing card shows ID, title, registry status, category, counts, and description; this file upgrades that into a layered capability card.

Owns

- identity
- badges
- metrics
- warnings
- actions

Uses

- `CapabilityBadge`
- `MetricChip`
- `WarningPill`

`App/src/components/capabilities/CapabilityBadge.tsx`

Shared badge primitive.

Used for

- category
- registry status
- implementation status
- risk

`App/src/components/capabilities/MetricChip.tsx`

Shared compact metric display.

Used for

- modules
- gates
- deps
- blockers
- warnings

`App/src/components/capabilities/WarningPill.tsx`

Shared warning token.

Used for

- active without modules
- active without gates
- unresolved dependencies
- active but low readiness

App/src/components/capabilities/AttentionPanel.tsx

Top issue zone in Overview.

Owns first-pass buckets

- blocked
- missing modules
- missing gates
- active but incomplete

App/src/components/capabilities/CoverageCalloutRow.tsx

Compact gap summary row.

Owns first-pass callouts

- missing modules
- missing gates
- active but incomplete
- high impact

App/src/components/capabilities/CapabilitiesEmptyState.tsx

Fallback for filtered empty results.

`App/src/components/capabilities/panels/OverviewPanel.tsx`

First real working panel.

Owns

- `AttentionPanel`
- `CoverageCalloutRow`
- `CapabilityBoard`

This is the only fully functional tab in the first pass.

`App/src/hooks/capabilities/useCapabilitiesPage.ts`

Top-level page orchestration hook.

Owns

- raw query state
- tab state
- search/filter/sort state
- model hook call

`App/src/hooks/capabilities/useCapabilitiesPageModel.ts`

Central truth/model hook.

Owns

- feature normalization
- derived capability records
- stats
- filtering
- sorting
- warning buckets

This is the main separation point from the current inline page logic. Right now the current file calculates `activeCount`, groups by category, and renders everything directly in one component.

App/src/lib/capabilities/deriveCapabilityRecord.ts

Build one `CapabilityRecord` from one raw `Feature`.

App/src/lib/capabilities/deriveImplementationStatus.ts

Separates registry state from implementation truth.

This is the key fix for the current problem where `active` can visually read as “implemented.”

App/src/lib/capabilities/deriveReadinessScore.ts

Computes readiness score + label.

App/src/lib/capabilities/collectAttentionFlags.ts

Builds warning flags from derived rules.

`App/src/lib/capabilities/buildCapabilityPageModel.ts`

Pure model builder used by `useCapabilitiesPageModel`.

Owns

- normalization
- reverse dependency map
- unresolved dependency checks
- derived record creation
- stat aggregation
- warning bucket creation

`App/src/types/capabilities.ts`

Central type pack.

Must contain

- `Feature`
- `CapabilityRecord`
- `CapabilityStats`
- `CapabilityFilterState`
- `CapabilityTabId`
- `RegistryStatus`
- `ImplementationStatus`
- `RiskLevel`
- `AttentionFlag`
- `CapabilityWarnings`

---

4. First-pass directory shape

```
App/src/
  pages/
    capabilities.tsx
    features.tsx                // thin alias or rewritten entry

  components/
    capabilities/
      CapabilitiesPageHeader.tsx
      CapabilityStatStrip.tsx
      CapabilityStatCard.tsx
      CapabilityControlBar.tsx
      CapabilityTabs.tsx
      CapabilityBoard.tsx
      CapabilityCard.tsx
      CapabilityBadge.tsx
      MetricChip.tsx
      WarningPill.tsx
      AttentionPanel.tsx
      CoverageCalloutRow.tsx
      CapabilitiesEmptyState.tsx
      panels/
        OverviewPanel.tsx

  hooks/
    capabilities/
      useCapabilitiesPage.ts
      useCapabilitiesPageModel.ts

  lib/
    capabilities/
      buildCapabilityPageModel.ts
      deriveCapabilityRecord.ts
      deriveImplementationStatus.ts
      deriveReadinessScore.ts
      collectAttentionFlags.ts

  types/
    capabilities.ts
```

---

5. First-pass “done” criteria by file set

Identity layer done when

These files exist and are wired:

- `CapabilitiesPageHeader.tsx`
- `CapabilityStatStrip.tsx`
- `CapabilityStatCard.tsx`

And the page no longer presents as “Feature Registry” with a fixed “17 subsystem contracts” subtitle.

Shell layer done when

These files exist and are wired:

- `capabilities.tsx`
- `features.tsx` alias or rewrite
- `CapabilityControlBar.tsx`
- `CapabilityTabs.tsx`

Truth/model layer done when

These files exist and are wired:

- `useCapabilitiesPage.ts`
- `useCapabilitiesPageModel.ts`
- `buildCapabilityPageModel.ts`
- derivation helpers
- `types/capabilities.ts`

Card layer done when

These files exist and are wired:

- `CapabilityCard.tsx`
- `CapabilityBadge.tsx`
- `MetricChip.tsx`

- `WarningPill.tsx`

And the old flat inline card renderer is gone. The current card is still a single button with status icon, badges, counts, and description inline.

Overview layer done when

These files exist and are wired:

- `OverviewPanel.tsx`
- `AttentionPanel.tsx`
- `CoverageCalloutRow.tsx`
- `CapabilityBoard.tsx`
- `CapabilitiesEmptyState.tsx`

And Overview can already show:

- blocked
- missing modules
- missing gates
- active but incomplete

---

6. Minimum first coding order

Build these in this exact order:

1. App/src/types/capabilities.ts
2. App/src/lib/capabilities/deriveImplementationStatus.ts
3. App/src/lib/capabilities/deriveReadinessScore.ts
4. App/src/lib/capabilities/collectAttentionFlags.ts
5. App/src/lib/capabilities/deriveCapabilityRecord.ts
6. App/src/lib/capabilities/buildCapabilityPageModel.ts
7. App/src/hooks/capabilities/useCapabilitiesPageModel.ts
8. App/src/hooks/capabilities/useCapabilitiesPage.ts
9. App/src/components/capabilities/CapabilitiesPageHeader.tsx
10. App/src/components/capabilities/CapabilityStatCard.tsx
11. App/src/components/capabilities/CapabilityStatStrip.tsx
12. App/src/components/capabilities/CapabilityBadge.tsx
13. App/src/components/capabilities/MetricChip.tsx
14. App/src/components/capabilities/WarningPill.tsx

15. App/src/components/capabilities/CapabilityCard.tsx
16. App/src/components/capabilities/CapabilityBoard.tsx
17. App/src/components/capabilities/AttentionPanel.tsx
18. App/src/components/capabilities/CoverageCalloutRow.tsx
19. App/src/components/capabilities/CapabilitiesEmptyState.tsx
20. App/src/components/capabilities/CapabilityControlBar.tsx
21. App/src/components/capabilities/CapabilityTabs.tsx
22. App/src/components/capabilities/panels/OverviewPanel.tsx
23. App/src/pages/capabilities.tsx
24. App/src/pages/features.tsx

---

7. Strong recommendation

For this first pass, do **not** try to build secondary tabs yet.

The best first shippable slice is:

- new identity
- new shell
- derived truth model
- new card system
- Overview only

That is enough to stop the page from pretending registry entries are fully real features.

Capability Control Center — Starter Prop Pack

Capability Control Center — Starter Prop Pack

1. Core shared types

Put these in:

App/src/types/capabilities.ts

```
export type CapabilityCategory =
 | "infrastructure"
 | "core-logic"
 | "interface"
 | "security"
 | "other";

export type RegistryStatus =
 | "active"
 | "draft"
 | "error"
 | "deprecated";

export type ImplementationStatus =
 | "implemented"
 | "partial"
 | "stubbed"
 | "spec_only"
 | "blocked"
 | "unverified";

export type ReadinessLabel =
 | "ready"
 | "near_ready"
 | "partial"
 | "fragile"
 | "blocked";

export type RiskLevel =
 | "low"
 | "moderate"
 | "high"
```

```
 | "critical";

export type DependencyHealth =
 | "healthy"
 | "warning"
 | "blocked"
 | "unknown";

export type GateHealth =
 | "covered"
 | "partial"
 | "missing"
 | "failed"
 | "unknown";

export type ModuleHealth =
 | "linked"
 | "partial"
 | "missing"
 | "orphaned"
 | "stale"
 | "unknown";

export type CapabilityTabId =
 | "overview"
 | "readiness"
 | "dependencies"
 | "gates"
 | "modules"
 | "risk";

export type CapabilitySortMode =
 | "title_asc"
 | "feature_id_asc"
 | "readiness_desc"
 | "risk_desc"
 | "dependency_count_desc"
 | "gate_count_desc"
 | "module_count_desc";

export type AttentionFlag =
 | "active_without_modules"
 | "active_without_gates"
 | "active_but_low_readiness"
```

```
 | "implementation_ahead_of_registry"
 | "registry_state_review"
 | "unresolved_dependencies"
 | "missing_verification"
 | "missing_implementation_surface"
 | "high_blast_radius"
 | "error_state";

export type CapabilityWarningCode =
 | "no_modules"
 | "no_gates"
 | "dependency_unknown"
 | "dependency_blocked"
 | "implementation_unknown"
 | "marked_active_but_incomplete"
 | "orphan_registry_entry"
 | "weak_verification"
 | "high_risk_dependency_chain";

export interface Feature {
 feature_id: string;
 title: string;
 description: string;
 category: string;
 status: string;
 dependencies: string[];
 src_modules: string[];
 gates: string[];
}

export interface CapabilityRecord {
 feature_id: string;
 title: string;
 description: string;
 category: CapabilityCategory;
 registry_status: RegistryStatus;
 dependencies: string[];
 src_modules: string[];
 gates: string[];

 implementation_status: ImplementationStatus;
 readiness_score: number;
 readiness_label: ReadinessLabel;
 risk_level: RiskLevel;
```

```
 dependency_health: DependencyHealth;
 gate_health: GateHealth;
 module_health: ModuleHealth;

 blocker_count: number;
 warning_count: number;

 attention_flags: AttentionFlag[];
 warnings: CapabilityWarningCode[];

 reverse_dependency_count: number;
 unresolved_dependencies: string[];
 missing_gates: string[];
 missing_modules: string[];
}

export interface CapabilityStats {
 total: number;
 implemented: number;
 partial: number;
 blocked: number;
 unverified: number;
 highRisk: number;

 active: number;
 draft: number;
 error: number;
 deprecated: number;

 missingModules: number;
 missingGates: number;
 unresolvedDependencies: number;
 activeButIncomplete: number;
}

export interface CapabilityFilterState {
 category: CapabilityCategory | "all";
 registryStatus: RegistryStatus | "all";
 implementationStatus: ImplementationStatus | "all";
 risk: RiskLevel | "all";

 blockedOnly: boolean;
 incompleteOnly: boolean;
```

```
 unverifiedOnly: boolean;
 missingGatesOnly: boolean;
 missingModulesOnly: boolean;
 highImpactOnly: boolean;
}

export interface CapabilityWarnings {
 blocked: CapabilityRecord[];
 missingModules: CapabilityRecord[];
 missingGates: CapabilityRecord[];
 activeButIncomplete: CapabilityRecord[];
 highRisk: CapabilityRecord[];
 unresolvedDependencies: CapabilityRecord[];
}

export interface CapabilityPageModel {
 raw: Feature[];
 derived: CapabilityRecord[];
 filtered: CapabilityRecord[];
 stats: CapabilityStats;
 warnings: CapabilityWarnings;
}
```

---

2. Page prop contracts

CapabilitiesPageHeader

```
export interface CapabilitiesPageHeaderProps {
 title: string;
 subtitle: string;
 lastSyncedLabel?: string;
}
```

Notes

Keep this simple. First pass does not need actions in the header.

CapabilityStatCard

```
export interface CapabilityStatCardProps {
```

```
 label: string;
 value: number;
 tone?: "default" | "good" | "warn" | "bad";
 onClick?: () => void;
}
```

`CapabilityStatStrip`

```
import type { CapabilityStats } from "@/types/capabilities";

export interface CapabilityStatStripProps {
 stats: CapabilityStats;
 onSelectStat?: (key: keyof CapabilityStats) => void;
}
```

Notes

Even if you do not wire `onSelectStat` yet, keep it. It gives you click-to-filter later without reworking the prop shape.

---

3. Control layer props

`CapabilityControlBar`

```
import type {
 CapabilityFilterState,
 CapabilitySortMode,
} from "@/types/capabilities";

export interface CapabilityControlBarProps {
 searchQuery: string;
 filters: CapabilityFilterState;
 sortMode: CapabilitySortMode;
 onSearchChange: (value: string) => void;
 onFiltersChange: (next: CapabilityFilterState) => void;
 onPatchFilters: (patch: Partial<CapabilityFilterState>) => void;
 onSortChange: (mode: CapabilitySortMode) => void;
 onResetFilters: () => void;
}
```

Notes

Use both `onFiltersChange` and `onPatchFilters`.

- full replace is useful for selects
- patch is useful for toggles

`CapabilityTabs`

```
import type { CapabilityTabId } from "@/types/capabilities";

export interface CapabilityTabsProps {
 activeTab: CapabilityTabId;
 onTabChange: (tab: CapabilityTabId) => void;
 disabledTabs?: CapabilityTabId[];
}
```

First-pass recommendation

Use:

```
disabledTabs = ["readiness", "dependencies", "gates", "modules", "risk"]
```

---

4. Board and card props

`CapabilityBoard`

```
import type { CapabilityRecord } from "@/types/capabilities";

export interface CapabilityBoardProps {
 capabilities: CapabilityRecord[];
 onOpenCapability: (featureId: string) => void;
}
```

`CapabilityCard`

```
import type { CapabilityRecord } from "@/types/capabilities";
```

```
export interface CapabilityCardProps {
 capability: CapabilityRecord;
 onOpen: (featureId: string) => void;
 onOpenDeps?: (featureId: string) => void;
 onOpenGates?: (featureId: string) => void;
 onOpenModules?: (featureId: string) => void;
}
```

Notes

Only `onOpen` must be wired in first pass.
The others can stay optional and unused until later tabs exist.

CapabilityBadge

```
export interface CapabilityBadgeProps {
 label: string;
 kind: "category" | "registry" | "implementation" | "risk" | "health";
}
```

MetricChip

```
export interface MetricChipProps {
 label: string;
 value: number;
 tone?: "neutral" | "good" | "warn" | "bad";
 onClick?: () => void;
}
```

WarningPill

```
export interface WarningPillProps {
 label: string;
 severity: "info" | "warn" | "error";
 onClick?: () => void;
}
```

CapabilitiesEmptyState

```
export interface CapabilitiesEmptyStateProps {
 title: string;
 message: string;
 onReset?: () => void;
}
```

---

5. Overview section props

AttentionPanel

```
import type { CapabilityWarnings } from "@/types/capabilities";

export interface AttentionPanelProps {
 warnings: CapabilityWarnings;
 onSelectSlice?: (slice: keyof CapabilityWarnings) => void;
}
```

Notes

This lets you click a warning bucket later and auto-filter the board.

CoverageCalloutRow

```
import type { CapabilityStats } from "@/types/capabilities";

export interface CoverageCalloutRowProps {
 stats: CapabilityStats;
 onSelectCallout?: (
   kind:
     | "missingModules"
     | "missingGates"
     | "activeButIncomplete"
     | "highRisk"
 ) => void;
}
```

OverviewPanel

```
import type {
 CapabilityRecord,
 CapabilityStats,
 CapabilityWarnings,
} from "@/types/capabilities";

export interface OverviewPanelProps {
 capabilities: CapabilityRecord[];
 stats: CapabilityStats;
 warnings: CapabilityWarnings;
 onOpenCapability: (featureId: string) => void;
 onResetFilters?: () => void;
}
```

Notes

This panel should not compute anything.
 It only renders the already-derived page model.

---

6. Hook result contracts

useCapabilitiesPage

```
import type {
 CapabilityFilterState,
 CapabilityPageModel,
 CapabilitySortMode,
 CapabilityTabId,
} from "@/types/capabilities";

export interface UseCapabilitiesPageResult {
 isLoading: boolean;
 error: unknown;

 activeTab: CapabilityTabId;
 setActiveTab: (tab: CapabilityTabId) => void;

 searchQuery: string;
 setSearchQuery: (value: string) => void;
```

```
 filters: CapabilityFilterState;
 setFilters: (next: CapabilityFilterState) => void;
 patchFilters: (patch: Partial<CapabilityFilterState>) => void;
 resetFilters: () => void;

 sortMode: CapabilitySortMode;
 setSortMode: (mode: CapabilitySortMode) => void;

 model: CapabilityPageModel;
}
```

---

useCapabilitiesPageModel

```
import type {
 CapabilityFilterState,
 CapabilityPageModel,
 CapabilitySortMode,
 Feature,
} from "@/types/capabilities";

export interface UseCapabilitiesPageModelArgs {
 features: Feature[];
 searchQuery: string;
 filters: CapabilityFilterState;
 sortMode: CapabilitySortMode;
}

export type UseCapabilitiesPageModelResult = CapabilityPageModel;
```

---

7. Lib helper contracts

deriveImplementationStatus

```
import type {
 ImplementationStatus,
 RegistryStatus,
} from "@/types/capabilities";

export interface DeriveImplementationStatusArgs {
 registryStatus: RegistryStatus;
```

```
 srcModulesCount: number;
 gatesCount: number;
 unresolvedDependencyCount: number;
}

export type DeriveImplementationStatus = (
 args: DeriveImplementationStatusArgs
) => ImplementationStatus;
```

---

deriveReadinessScore

```
import type {
 ReadinessLabel,
 RegistryStatus,
} from "@/types/capabilities";

export interface DeriveReadinessScoreArgs {
 registryStatus: RegistryStatus;
 srcModulesCount: number;
 gatesCount: number;
 unresolvedDependencyCount: number;
}

export interface DeriveReadinessScoreResult {
 score: number;
 label: ReadinessLabel;
}
```

---

collectAttentionFlags

```
import type {
 AttentionFlag,
 CapabilityWarningCode,
 ImplementationStatus,
 RegistryStatus,
} from "@/types/capabilities";

export interface CollectAttentionFlagsArgs {
 registryStatus: RegistryStatus;
 implementationStatus: ImplementationStatus;
 srcModulesCount: number;
 gatesCount: number;
```

```
 unresolvedDependencyCount: number;
 reverseDependencyCount: number;
 readinessScore: number;
}

export interface CollectAttentionFlagsResult {
 attentionFlags: AttentionFlag[];
 warnings: CapabilityWarningCode[];
}
```

---

deriveCapabilityRecord

```
import type { CapabilityRecord, Feature } from "@/types/capabilities";

export interface DeriveCapabilityRecordArgs {
 feature: Feature;
 reverseDependencyCount: number;
 unresolvedDependencies: string[];
}

export type DeriveCapabilityRecord = (
 args: DeriveCapabilityRecordArgs
) => CapabilityRecord;
```

---

buildCapabilityPageModel

```
import type {
 CapabilityFilterState,
 CapabilityPageModel,
 CapabilitySortMode,
 Feature,
} from "@/types/capabilities";

export interface BuildCapabilityPageModelArgs {
 features: Feature[];
 searchQuery: string;
 filters: CapabilityFilterState;
 sortMode: CapabilitySortMode;
}

export type BuildCapabilityPageModel = (
 args: BuildCapabilityPageModelArgs
```

```
) => CapabilityPageModel;
```

---

8. Default state contracts

These should also live in `types/capabilities.ts` or near the hook layer.

```
export const DEFAULT_CAPABILITY_FILTERS: CapabilityFilterState = {
 category: "all",
 registryStatus: "all",
 implementationStatus: "all",
 risk: "all",
 blockedOnly: false,
 incompleteOnly: false,
 unverifiedOnly: false,
 missingGatesOnly: false,
 missingModulesOnly: false,
 highImpactOnly: false,
};

export const DEFAULT_CAPABILITY_TAB: CapabilityTabId = "overview";

export const DEFAULT_CAPABILITY_SORT: CapabilitySortMode = "readiness_desc";
```

---

9. Practical page wiring contract

Your first-pass page should be able to render with this shape:

```
const {
 isLoading,
 error,
 activeTab,
 setActiveTab,
 searchQuery,
 setSearchQuery,
 filters,
 setFilters,
 patchFilters,
 resetFilters,
 sortMode,
 setSortMode,
 model,
```

```
} = useCapabilitiesPage();
```

Then:

```
<CapabilitiesPageHeader
 title="Capability Control Center"
 subtitle="Capability readiness, dependency health, and implementation coverage"
/>

<CapabilityStatStrip stats={model.stats} />

<CapabilityControlBar
 searchQuery={searchQuery}
 filters={filters}
 sortMode={sortMode}
 onSearchChange={setSearchQuery}
 onFiltersChange={setFilters}
 onPatchFilters={patchFilters}
 onSortChange={setSortMode}
 onResetFilters={resetFilters}
/>

<CapabilityTabs
 activeTab={activeTab}
 onTabChange={setActiveTab}
 disabledTabs={["readiness", "dependencies", "gates", "modules", "risk"]}
/>

<OverviewPanel
 capabilities={model.filtered}
 stats={model.stats}
 warnings={model.warnings}
 onOpenCapability={handleOpenCapability}
 onResetFilters={resetFilters}
/>
```

---

10. Locking rule

For first pass, do not let any component invent missing data on its own.

Every component should receive:

- already-derived records
- already-aggregated stats
- already-grouped warning buckets

That means:

- `CapabilityCard` does not compute readiness
- `OverviewPanel` does not compute warnings
- `StatStrip` does not compute counts

All of that comes from the model layer.

That separation is what keeps this rewrite clean.

starter code skeleton pack

Good. Here's the **starter code skeleton pack** for the **first-pass slice only**.

This pack is designed to replace the current single-file `FeaturesPage`, which today fetches `/api/features`, computes `activeCount`, groups by category, renders the hardcoded "Feature Registry" header, and outputs clickable cards with status/category/modules/gates/deps metadata.

1. `App/src/types/capabilities.ts`

```
export type CapabilityCategory =
  | "infrastructure"
  | "core-logic"
  | "interface"
  | "security"
  | "other";

export type RegistryStatus =
  | "active"
  | "draft"
  | "error"
  | "deprecated";

export type ImplementationStatus =
  | "implemented"
  | "partial"
  | "stubbed"
  | "spec_only"
  | "blocked"
  | "unverified";

export type ReadinessLabel =
  | "ready"
  | "near_ready"
  | "partial"
  | "fragile"
  | "blocked";

export type RiskLevel =
  | "low"
  | "moderate"
  | "high"
  | "critical";
```

```
export type DependencyHealth =
  | "healthy"
  | "warning"
  | "blocked"
  | "unknown";

export type GateHealth =
  | "covered"
  | "partial"
  | "missing"
  | "failed"
  | "unknown";

export type ModuleHealth =
  | "linked"
  | "partial"
  | "missing"
  | "orphaned"
  | "stale"
  | "unknown";

export type CapabilityTabId =
  | "overview"
  | "readiness"
  | "dependencies"
  | "gates"
  | "modules"
  | "risk";

export type CapabilitySortMode =
  | "title_asc"
  | "feature_id_asc"
  | "readiness_desc"
  | "risk_desc"
  | "dependency_count_desc"
  | "gate_count_desc"
  | "module_count_desc";

export type AttentionFlag =
  | "active_without_modules"
  | "active_without_gates"
  | "active_but_low_readiness"
  | "implementation_ahead_of_registry"
```

```
  | "registry_state_review"
  | "unresolved_dependencies"
  | "missing_verification"
  | "missing_implementation_surface"
  | "high_blast_radius"
  | "error_state";

export type CapabilityWarningCode =
  | "no_modules"
  | "no_gates"
  | "dependency_unknown"
  | "dependency_blocked"
  | "implementation_unknown"
  | "marked_active_but_incomplete"
  | "orphan_registry_entry"
  | "weak_verification"
  | "high_risk_dependency_chain";

export interface Feature {
  feature_id: string;
  title: string;
  description: string;
  category: string;
  status: string;
  dependencies: string[];
  src_modules: string[];
  gates: string[];
}

export interface CapabilityRecord {
  feature_id: string;
  title: string;
  description: string;
  category: CapabilityCategory;
  registry_status: RegistryStatus;
  dependencies: string[];
  src_modules: string[];
  gates: string[];

  implementation_status: ImplementationStatus;
  readiness_score: number;
  readiness_label: ReadinessLabel;
  risk_level: RiskLevel;
```

```
  dependency_health: DependencyHealth;
  gate_health: GateHealth;
  module_health: ModuleHealth;

  blocker_count: number;
  warning_count: number;

  attention_flags: AttentionFlag[];
  warnings: CapabilityWarningCode[];

  reverse_dependency_count: number;
  unresolved_dependencies: string[];
  missing_gates: string[];
  missing_modules: string[];
}

export interface CapabilityStats {
  total: number;
  implemented: number;
  partial: number;
  blocked: number;
  unverified: number;
  highRisk: number;

  active: number;
  draft: number;
  error: number;
  deprecated: number;

  missingModules: number;
  missingGates: number;
  unresolvedDependencies: number;
  activeButIncomplete: number;
}

export interface CapabilityFilterState {
  category: CapabilityCategory | "all";
  registryStatus: RegistryStatus | "all";
  implementationStatus: ImplementationStatus | "all";
  risk: RiskLevel | "all";

  blockedOnly: boolean;
  incompleteOnly: boolean;
  unverifiedOnly: boolean;
```

```
  missingGatesOnly: boolean;
  missingModulesOnly: boolean;
  highImpactOnly: boolean;
}

export interface CapabilityWarnings {
  blocked: CapabilityRecord[];
  missingModules: CapabilityRecord[];
  missingGates: CapabilityRecord[];
  activeButIncomplete: CapabilityRecord[];
  highRisk: CapabilityRecord[];
  unresolvedDependencies: CapabilityRecord[];
}

export interface CapabilityPageModel {
  raw: Feature[];
  derived: CapabilityRecord[];
  filtered: CapabilityRecord[];
  stats: CapabilityStats;
  warnings: CapabilityWarnings;
}

export const DEFAULT_CAPABILITY_FILTERS: CapabilityFilterState = {
  category: "all",
  registryStatus: "all",
  implementationStatus: "all",
  risk: "all",
  blockedOnly: false,
  incompleteOnly: false,
  unverifiedOnly: false,
  missingGatesOnly: false,
  missingModulesOnly: false,
  highImpactOnly: false,
};

export const DEFAULT_CAPABILITY_TAB: CapabilityTabId = "overview";
export const DEFAULT_CAPABILITY_SORT: CapabilitySortMode = "readiness_desc";
```

2. App/src/lib/capabilities/deriveImplementationStatus.ts

```ts
import type {
  DeriveImplementationStatusArgs,
  ImplementationStatus,
} from "@/types/capabilities";

export interface DeriveImplementationStatusArgs {
  registryStatus: "active" | "draft" | "error" | "deprecated";
  srcModulesCount: number;
  gatesCount: number;
  unresolvedDependencyCount: number;
}

export function deriveImplementationStatus({
  registryStatus,
  srcModulesCount,
  gatesCount,
  unresolvedDependencyCount,
}: DeriveImplementationStatusArgs): ImplementationStatus {
  if (unresolvedDependencyCount > 0) return "blocked";

  if (srcModulesCount === 0 && gatesCount === 0) return "spec_only";

  if (srcModulesCount > 0 && gatesCount === 0) {
    return registryStatus === "active" ? "unverified" : "partial";
  }

  if (srcModulesCount > 0 && gatesCount > 0) {
    return registryStatus === "active" ? "implemented" : "partial";
  }

  return "stubbed";
}
```

3. App/src/lib/capabilities/deriveReadinessScore.ts

```
import type {
  ReadinessLabel,
  RegistryStatus,
} from "@/types/capabilities";

export interface DeriveReadinessScoreArgs {
  registryStatus: RegistryStatus;
  srcModulesCount: number;
  gatesCount: number;
  unresolvedDependencyCount: number;
}

export interface DeriveReadinessScoreResult {
  score: number;
  label: ReadinessLabel;
}

export function deriveReadinessScore({
  registryStatus,
  srcModulesCount,
  gatesCount,
  unresolvedDependencyCount,
}: DeriveReadinessScoreArgs): DeriveReadinessScoreResult {
  let score = 0;

  if (registryStatus === "active") score += 30;
  if (srcModulesCount > 0) score += 25;
  if (gatesCount > 0) score += 20;
  if (unresolvedDependencyCount === 0) score += 15;
  if (registryStatus !== "error") score += 10;

  score = Math.max(0, Math.min(100, score));

  let label: ReadinessLabel = "blocked";
  if (score >= 85) label = "ready";
  else if (score >= 65) label = "near_ready";
  else if (score >= 40) label = "partial";
```

```
  else if (score >= 15) label = "fragile";

  return { score, label };
}
```

4. App/src/lib/capabilities/collectAttentionFlags.ts

```
import type {
  AttentionFlag,
  CapabilityWarningCode,
  ImplementationStatus,
  RegistryStatus,
} from "@/types/capabilities";

export interface CollectAttentionFlagsArgs {
  registryStatus: RegistryStatus;
  implementationStatus: ImplementationStatus;
  srcModulesCount: number;
  gatesCount: number;
  unresolvedDependencyCount: number;
  reverseDependencyCount: number;
  readinessScore: number;
}

export interface CollectAttentionFlagsResult {
  attentionFlags: AttentionFlag[];
  warnings: CapabilityWarningCode[];
}

export function collectAttentionFlags({
  registryStatus,
  implementationStatus,
  srcModulesCount,
  gatesCount,
  unresolvedDependencyCount,
  reverseDependencyCount,
  readinessScore,
```

```
}: CollectAttentionFlagsArgs): CollectAttentionFlagsResult {
  const attentionFlags: AttentionFlag[] = [];
  const warnings: CapabilityWarningCode[] = [];

  if (registryStatus === "active" && srcModulesCount === 0) {
    attentionFlags.push("active_without_modules");
    warnings.push("no_modules");
  }

  if (registryStatus === "active" && gatesCount === 0) {
    attentionFlags.push("active_without_gates");
    warnings.push("no_gates");
  }

  if (registryStatus === "active" && readinessScore < 65) {
    attentionFlags.push("active_but_low_readiness");
    warnings.push("marked_active_but_incomplete");
  }

  if (registryStatus === "draft" && srcModulesCount > 0) {
    attentionFlags.push("implementation_ahead_of_registry");
  }

  if (unresolvedDependencyCount > 0) {
    attentionFlags.push("unresolved_dependencies");
    warnings.push("dependency_blocked");
  }

  if (reverseDependencyCount >= 5) {
    attentionFlags.push("high_blast_radius");
    warnings.push("high_risk_dependency_chain");
  }

  if (registryStatus === "error") {
    attentionFlags.push("error_state");
  }

  if (implementationStatus === "unverified") {
    attentionFlags.push("missing_verification");
    warnings.push("weak_verification");
  }

  if (srcModulesCount === 0) {
    attentionFlags.push("missing_implementation_surface");
```

```
  }

  return { attentionFlags, warnings };
}
```

5. App/src/lib/capabilities/deriveCapabilityRecord.ts

```
import type {
  CapabilityCategory,
  CapabilityRecord,
  DependencyHealth,
  Feature,
  GateHealth,
  ModuleHealth,
  RegistryStatus,
  RiskLevel,
} from "@/types/capabilities";
import { collectAttentionFlags } from "./collectAttentionFlags";
import { deriveImplementationStatus } from "./deriveImplementationStatus";
import { deriveReadinessScore } from "./deriveReadinessScore";

export interface DeriveCapabilityRecordArgs {
  feature: Feature;
  reverseDependencyCount: number;
  unresolvedDependencies: string[];
}

function normalizeCategory(value: string): CapabilityCategory {
  if (
    value === "infrastructure" ||
    value === "core-logic" ||
    value === "interface" ||
    value === "security"
  ) {
    return value;
  }
  return "other";
```

```
}

function normalizeRegistryStatus(value: string): RegistryStatus {
  if (
    value === "active" ||
    value === "draft" ||
    value === "error" ||
    value === "deprecated"
  ) {
    return value;
  }
  return "draft";
}

function deriveDependencyHealth(
  dependencies: string[],
  unresolvedDependencies: string[]
): DependencyHealth {
  if (unresolvedDependencies.length > 0) return "blocked";
  if (dependencies.length === 0) return "healthy";
  return "healthy";
}

function deriveGateHealth(gates: string[], registryStatus: RegistryStatus): GateHealth {
  if (gates.length === 0 && registryStatus === "active") return "missing";
  if (gates.length === 0) return "unknown";
  return "covered";
}

function deriveModuleHealth(
  srcModules: string[],
  registryStatus: RegistryStatus
): ModuleHealth {
  if (srcModules.length === 0 && registryStatus === "active") return "missing";
  if (srcModules.length === 0) return "unknown";
  return "linked";
}

function deriveRiskLevel(args: {
  registryStatus: RegistryStatus;
  reverseDependencyCount: number;
  unresolvedDependencyCount: number;
  srcModulesCount: number;
  gatesCount: number;
```

```
  readinessScore: number;
}): RiskLevel {
  let level: RiskLevel = "low";

  if (args.readinessScore < 40) level = "moderate";
  if (args.unresolvedDependencyCount > 0) level = "high";
  if (
    args.registryStatus === "active" &&
    (args.srcModulesCount === 0 || args.gatesCount === 0)
  ) {
    level = "high";
  }
  if (args.registryStatus === "error") level = "high";
  if (args.reverseDependencyCount >= 5) {
    level = level === "low" ? "moderate" : "critical";
  }

  return level;
}

export function deriveCapabilityRecord({
  feature,
  reverseDependencyCount,
  unresolvedDependencies,
}: DeriveCapabilityRecordArgs): CapabilityRecord {
  const category = normalizeCategory(feature.category);
  const registry_status = normalizeRegistryStatus(feature.status);
  const dependencies = feature.dependencies ?? [];
  const src_modules = feature.src_modules ?? [];
  const gates = feature.gates ?? [];

  const implementation_status = deriveImplementationStatus({
    registryStatus: registry_status,
    srcModulesCount: src_modules.length,
    gatesCount: gates.length,
    unresolvedDependencyCount: unresolvedDependencies.length,
  });

  const { score: readiness_score, label: readiness_label } = deriveReadinessScore({
    registryStatus: registry_status,
    srcModulesCount: src_modules.length,
    gatesCount: gates.length,
    unresolvedDependencyCount: unresolvedDependencies.length,
  });
```

```
  const dependency_health = deriveDependencyHealth(dependencies,
unresolvedDependencies);
  const gate_health = deriveGateHealth(gates, registry_status);
  const module_health = deriveModuleHealth(src_modules, registry_status);

  const risk_level = deriveRiskLevel({
    registryStatus: registry_status,
    reverseDependencyCount,
    unresolvedDependencyCount: unresolvedDependencies.length,
    srcModulesCount: src_modules.length,
    gatesCount: gates.length,
    readinessScore: readiness_score,
  });

  const { attentionFlags, warnings } = collectAttentionFlags({
    registryStatus: registry_status,
    implementationStatus: implementation_status,
    srcModulesCount: src_modules.length,
    gatesCount: gates.length,
    unresolvedDependencyCount: unresolvedDependencies.length,
    reverseDependencyCount,
    readinessScore: readiness_score,
  });

  return {
    feature_id: feature.feature_id,
    title: feature.title,
    description: feature.description ?? "",
    category,
    registry_status,
    dependencies,
    src_modules,
    gates,

    implementation_status,
    readiness_score,
    readiness_label,
    risk_level,

    dependency_health,
    gate_health,
    module_health,
```

```
    blocker_count: unresolvedDependencies.length,
    warning_count: warnings.length,

    attention_flags: attentionFlags,
    warnings,

    reverse_dependency_count: reverseDependencyCount,
    unresolved_dependencies: unresolvedDependencies,
    missing_gates: gate_health === "missing" ? ["coverage_missing"] : [],
    missing_modules: module_health === "missing" ? ["implementation_missing"] : [],
  };
}
```

6. App/src/lib/capabilities/buildCapabilityPageModel.ts

```
import type {
  CapabilityFilterState,
  CapabilityPageModel,
  CapabilityRecord,
  CapabilitySortMode,
  CapabilityStats,
  CapabilityWarnings,
  Feature,
} from "@/types/capabilities";
import { deriveCapabilityRecord } from "./deriveCapabilityRecord";

export interface BuildCapabilityPageModelArgs {
  features: Feature[];
  searchQuery: string;
  filters: CapabilityFilterState;
  sortMode: CapabilitySortMode;
}

function normalizeFeature(feature: Feature): Feature {
  return {
    feature_id: feature.feature_id,
    title: feature.title ?? "",
```

```
    description: feature.description ?? "",
    category: feature.category ?? "other",
    status: feature.status ?? "draft",
    dependencies: feature.dependencies ?? [],
    src_modules: feature.src_modules ?? [],
    gates: feature.gates ?? [],
  };
}

function buildReverseDependencyMap(features: Feature[]): Record<string, number> {
  const map: Record<string, number> = {};
  for (const feature of features) {
    map[feature.feature_id] = 0;
  }
  for (const feature of features) {
    for (const dep of feature.dependencies ?? []) {
      if (dep in map) map[dep] += 1;
    }
  }
  return map;
}

function buildDerivedRecords(features: Feature[]): CapabilityRecord[] {
  const idSet = new Set(features.map((f) => f.feature_id));
  const reverseMap = buildReverseDependencyMap(features);

  return features.map((feature) => {
    const unresolvedDependencies = (feature.dependencies ?? []).filter(
      (dep) => !idSet.has(dep)
    );

    return deriveCapabilityRecord({
      feature,
      reverseDependencyCount: reverseMap[feature.feature_id] ?? 0,
      unresolvedDependencies,
    });
  });
}

function buildStats(records: CapabilityRecord[]): CapabilityStats {
  return {
    total: records.length,
    implemented: records.filter((r) => r.implementation_status === "implemented").length,
    partial: records.filter((r) => r.implementation_status === "partial").length,
```

```
    blocked: records.filter((r) => r.implementation_status === "blocked").length,
    unverified: records.filter((r) => r.implementation_status === "unverified").length,
    highRisk: records.filter((r) => r.risk_level === "high" || r.risk_level === "critical").length,

    active: records.filter((r) => r.registry_status === "active").length,
    draft: records.filter((r) => r.registry_status === "draft").length,
    error: records.filter((r) => r.registry_status === "error").length,
    deprecated: records.filter((r) => r.registry_status === "deprecated").length,

    missingModules: records.filter((r) => r.module_health === "missing").length,
    missingGates: records.filter((r) => r.gate_health === "missing").length,
    unresolvedDependencies: records.filter((r) => r.unresolved_dependencies.length > 0).length,
    activeButIncomplete: records.filter(
      (r) => r.registry_status === "active" && r.readiness_score < 65
    ).length,
  };
}

function matchesSearch(record: CapabilityRecord, searchQuery: string): boolean {
  if (!searchQuery.trim()) return true;
  const q = searchQuery.toLowerCase();

  return [
    record.feature_id,
    record.title,
    record.description,
    record.category,
    ...record.src_modules,
    ...record.dependencies,
    ...record.gates,
  ]
    .join(" ")
    .toLowerCase()
    .includes(q);
}

function applyFilters(
  records: CapabilityRecord[],
  filters: CapabilityFilterState,
  searchQuery: string
): CapabilityRecord[] {
  return records.filter((record) => {
    if (filters.category !== "all" && record.category !== filters.category) return false;
    if (
```

```
      filters.registryStatus !== "all" &&
      record.registry_status !== filters.registryStatus
    ) {
      return false;
    }
    if (
      filters.implementationStatus !== "all" &&
      record.implementation_status !== filters.implementationStatus
    ) {
      return false;
    }
    if (filters.risk !== "all" && record.risk_level !== filters.risk) return false;
    if (filters.blockedOnly && record.implementation_status !== "blocked") return false;
    if (
      filters.incompleteOnly &&
      ["partial", "stubbed", "spec_only", "blocked", "unverified"].includes(
        record.implementation_status
      )
    ) {
      return false;
    }
    if (filters.unverifiedOnly && record.implementation_status !== "unverified") {
      return false;
    }
    if (filters.missingGatesOnly && record.gate_health !== "missing") return false;
    if (filters.missingModulesOnly && record.module_health !== "missing") return false;
    if (filters.highImpactOnly && record.reverse_dependency_count < 5) return false;

    return matchesSearch(record, searchQuery);
  });
}

function sortRecords(
  records: CapabilityRecord[],
  sortMode: CapabilitySortMode
): CapabilityRecord[] {
  const sorted = [...records];

  const riskRank: Record<string, number> = {
    low: 0,
    moderate: 1,
    high: 2,
    critical: 3,
  };
```

```
  sorted.sort((a, b) => {
    switch (sortMode) {
      case "title_asc":
        return a.title.localeCompare(b.title);
      case "feature_id_asc":
        return a.feature_id.localeCompare(b.feature_id);
      case "readiness_desc":
        return b.readiness_score - a.readiness_score;
      case "risk_desc":
        return riskRank[b.risk_level] - riskRank[a.risk_level];
      case "dependency_count_desc":
        return b.dependencies.length - a.dependencies.length;
      case "gate_count_desc":
        return b.gates.length - a.gates.length;
      case "module_count_desc":
        return b.src_modules.length - a.src_modules.length;
      default:
        return b.readiness_score - a.readiness_score;
    }
  });

  return sorted;
}

function buildWarnings(records: CapabilityRecord[]): CapabilityWarnings {
  return {
    blocked: records.filter((r) => r.implementation_status === "blocked"),
    missingModules: records.filter((r) => r.module_health === "missing"),
    missingGates: records.filter((r) => r.gate_health === "missing"),
    activeButIncomplete: records.filter(
      (r) => r.registry_status === "active" && r.readiness_score < 65
    ),
    highRisk: records.filter(
      (r) => r.risk_level === "high" || r.risk_level === "critical"
    ),
    unresolvedDependencies: records.filter((r) => r.unresolved_dependencies.length > 0),
  };
}

export function buildCapabilityPageModel({
  features,
  searchQuery,
  filters,
```

```
  sortMode,
}: BuildCapabilityPageModelArgs): CapabilityPageModel {
  const raw = features.map(normalizeFeature);
  const derived = buildDerivedRecords(raw);
  const stats = buildStats(derived);
  const filtered = sortRecords(applyFilters(derived, filters, searchQuery), sortMode);
  const warnings = buildWarnings(derived);

  return {
    raw,
    derived,
    filtered,
    stats,
    warnings,
  };
}
```

7. App/src/hooks/capabilities/useCapabilitiesPageModel.ts

```
import { useMemo } from "react";
import type {
  CapabilityFilterState,
  CapabilityPageModel,
  CapabilitySortMode,
  Feature,
} from "@/types/capabilities";
import { buildCapabilityPageModel } from "@/lib/capabilities/buildCapabilityPageModel";

export interface UseCapabilitiesPageModelArgs {
  features: Feature[];
  searchQuery: string;
  filters: CapabilityFilterState;
  sortMode: CapabilitySortMode;
}

export function useCapabilitiesPageModel({
  features,
```

```
  searchQuery,
  filters,
  sortMode,
}: UseCapabilitiesPageModelArgs): CapabilityPageModel {
  return useMemo(() => {
    return buildCapabilityPageModel({
      features,
      searchQuery,
      filters,
      sortMode,
    });
  }, [features, searchQuery, filters, sortMode]);
}
```

8. App/src/hooks/capabilities/useCapabilitiesPage.ts

```
import { useState } from "react";
import { useQuery } from "@tanstack/react-query";
import { apiRequest } from "@/lib/queryClient";
import {
  DEFAULT_CAPABILITY_FILTERS,
  DEFAULT_CAPABILITY_SORT,
  DEFAULT_CAPABILITY_TAB,
  type CapabilityFilterState,
  type CapabilityPageModel,
  type CapabilitySortMode,
  type CapabilityTabId,
  type Feature,
} from "@/types/capabilities";
import { useCapabilitiesPageModel } from "./useCapabilitiesPageModel";

export interface UseCapabilitiesPageResult {
  isLoading: boolean;
  error: unknown;

  activeTab: CapabilityTabId;
  setActiveTab: (tab: CapabilityTabId) => void;
```

```
  searchQuery: string;
  setSearchQuery: (value: string) => void;

  filters: CapabilityFilterState;
  setFilters: (next: CapabilityFilterState) => void;
  patchFilters: (patch: Partial<CapabilityFilterState>) => void;
  resetFilters: () => void;

  sortMode: CapabilitySortMode;
  setSortMode: (mode: CapabilitySortMode) => void;

  model: CapabilityPageModel;
}

export function useCapabilitiesPage(): UseCapabilitiesPageResult {
  const [activeTab, setActiveTab] = useState<CapabilityTabId>(DEFAULT_CAPABILITY_TAB);
  const [searchQuery, setSearchQuery] = useState("");
  const [filters, setFilters] = useState<CapabilityFilterState>(DEFAULT_CAPABILITY_FILTERS);
  const [sortMode, setSortMode] =
    useState<CapabilitySortMode>(DEFAULT_CAPABILITY_SORT);

  const { data: features = [], isLoading, error } = useQuery<Feature[]>({
    queryKey: ["/api/features"],
    queryFn: () => apiRequest("/api/features"),
  });

  const model = useCapabilitiesPageModel({
    features,
    searchQuery,
    filters,
    sortMode,
  });

  function patchFilters(patch: Partial<CapabilityFilterState>) {
    setFilters((current) => ({ ...current, ...patch }));
  }

  function resetFilters() {
    setFilters(DEFAULT_CAPABILITY_FILTERS);
    setSearchQuery("");
    setSortMode(DEFAULT_CAPABILITY_SORT);
  }
```

```
  return {
    isLoading,
    error,

    activeTab,
    setActiveTab,

    searchQuery,
    setSearchQuery,

    filters,
    setFilters,
    patchFilters,
    resetFilters,

    sortMode,
    setSortMode,

    model,
  };
}
```

9. App/src/components/capabilities/CapabilitiesPageHeader.tsx

```
import type { ReactNode } from "react";

export interface CapabilitiesPageHeaderProps {
  title: string;
  subtitle: string;
  lastSyncedLabel?: string;
}

export default function CapabilitiesPageHeader({
  title,
  subtitle,
  lastSyncedLabel,
}: CapabilitiesPageHeaderProps) {
```

```
  return (
    <div className="flex items-start justify-between gap-4">
      <div>
        <h1 className="text-2xl font-bold text-[hsl(var(--foreground))]">{title}</h1>
        <p className="mt-1 text-sm text-[hsl(var(--muted-foreground))]">{subtitle}</p>
        {lastSyncedLabel ? (
          <p className="mt-2 text-xs text-[hsl(var(--muted-foreground))]">
            {lastSyncedLabel}
          </p>
        ) : null}
      </div>
    </div>
  );
}
```

10. App/src/components/capabilities/CapabilityStatCard.tsx

```
export interface CapabilityStatCardProps {
  label: string;
  value: number;
  tone?: "default" | "good" | "warn" | "bad";
  onClick?: () => void;
}

function toneClassName(tone: CapabilityStatCardProps["tone"]) {
  switch (tone) {
    case "good":
      return "border-green-500/30 bg-green-500/10";
    case "warn":
      return "border-yellow-500/30 bg-yellow-500/10";
    case "bad":
      return "border-red-500/30 bg-red-500/10";
    default:
      return "border-[hsl(var(--border))] bg-[hsl(var(--card))]";
  }
}
```

```
export default function CapabilityStatCard({
  label,
  value,
  tone = "default",
  onClick,
}: CapabilityStatCardProps) {
  return (
    <button
      type="button"
      onClick={onClick}
      className={`rounded-lg border px-4 py-3 text-left transition-colors ${toneClassName(
        tone
      )} ${onClick ? "hover:border-[hsl(var(--primary))]" : "cursor-default"}`}
    >
      <div className="text-xl font-bold text-[hsl(var(--foreground))]">{value}</div>
      <div className="text-xs text-[hsl(var(--muted-foreground))]">{label}</div>
    </button>
  );
}
```

11. App/src/components/capabilities/CapabilityStatStrip.tsx

```
import type { CapabilityStats } from "@/types/capabilities";
import CapabilityStatCard from "./CapabilityStatCard";

export interface CapabilityStatStripProps {
  stats: CapabilityStats;
  onSelectStat?: (key: keyof CapabilityStats) => void;
}

export default function CapabilityStatStrip({
  stats,
  onSelectStat,
}: CapabilityStatStripProps) {
  return (
    <div className="grid grid-cols-2 gap-3 md:grid-cols-3 xl:grid-cols-6">
      <CapabilityStatCard
```

```
      label="Total"
      value={stats.total}
      onClick={onSelectStat ? () => onSelectStat("total") : undefined}
    />
    <CapabilityStatCard
      label="Implemented"
      value={stats.implemented}
      tone="good"
    />
    <CapabilityStatCard
      label="Partial"
      value={stats.partial}
      tone="warn"
    />
    <CapabilityStatCard
      label="Blocked"
      value={stats.blocked}
      tone="bad"
    />
    <CapabilityStatCard
      label="Unverified"
      value={stats.unverified}
      tone="warn"
    />
    <CapabilityStatCard
      label="High Risk"
      value={stats.highRisk}
      tone="bad"
    />
  </div>
  );
}
```

12. App/src/components/capabilities/CapabilityBadge.tsx

```
export interface CapabilityBadgeProps {
  label: string;
```

```
  kind: "category" | "registry" | "implementation" | "risk" | "health";
}

function getBadgeClassName(kind: CapabilityBadgeProps["kind"]) {
  switch (kind) {
    case "category":
      return "bg-blue-900/30 text-blue-300";
    case "registry":
      return "bg-purple-900/30 text-purple-300";
    case "implementation":
      return "bg-emerald-900/30 text-emerald-300";
    case "risk":
      return "bg-red-900/30 text-red-300";
    case "health":
      return "bg-amber-900/30 text-amber-300";
    default:
      return "bg-gray-800 text-gray-300";
  }
}

export default function CapabilityBadge({
  label,
  kind,
}: CapabilityBadgeProps) {
  return (
    <span
      className={`inline-flex items-center rounded-full px-2 py-0.5 text-xs font-medium
${getBadgeClassName(
        kind
      )}`}
    >
      {label}
    </span>
  );
}
```

13. App/src/components/capabilities/MetricChip.tsx

```
export interface MetricChipProps {
  label: string;
  value: number;
  tone?: "neutral" | "good" | "warn" | "bad";
  onClick?: () => void;
}

function toneClassName(tone: MetricChipProps["tone"]) {
  switch (tone) {
    case "good":
      return "border-green-500/30 text-green-300";
    case "warn":
      return "border-yellow-500/30 text-yellow-300";
    case "bad":
      return "border-red-500/30 text-red-300";
    default:
      return "border-[hsl(var(--border))] text-[hsl(var(--muted-foreground))]";
  }
}

export default function MetricChip({
  label,
  value,
  tone = "neutral",
  onClick,
}: MetricChipProps) {
  return (
    <button
      type="button"
      onClick={onClick}
      className={`rounded-full border px-2.5 py-1 text-xs ${toneClassName(tone)} ${
        onClick ? "hover:border-[hsl(var(--primary))]" : "cursor-default"
      }`}
    >
      {value} {label}
    </button>
  );
}
```

14. App/src/components/capabilities/WarningPill.tsx

```
export interface WarningPillProps {
  label: string;
  severity: "info" | "warn" | "error";
  onClick?: () => void;
}

function severityClassName(severity: WarningPillProps["severity"]) {
  switch (severity) {
    case "warn":
      return "bg-yellow-500/10 text-yellow-300 border-yellow-500/30";
    case "error":
      return "bg-red-500/10 text-red-300 border-red-500/30";
    default:
      return "bg-blue-500/10 text-blue-300 border-blue-500/30";
  }
}

export default function WarningPill({
  label,
  severity,
  onClick,
}: WarningPillProps) {
  return (
    <button
      type="button"
      onClick={onClick}
      className={`rounded-full border px-2 py-0.5 text-xs ${severityClassName(
        severity
      )} ${onClick ? "hover:opacity-90" : "cursor-default"}`}
    >
      {label}
    </button>
  );
}
```

15. App/src/components/capabilities/CapabilityCard.tsx

```
import type { CapabilityRecord } from "@/types/capabilities";
import CapabilityBadge from "./CapabilityBadge";
import MetricChip from "./MetricChip";
import WarningPill from "./WarningPill";

export interface CapabilityCardProps {
  capability: CapabilityRecord;
  onOpen: (featureId: string) => void;
  onOpenDeps?: (featureId: string) => void;
  onOpenGates?: (featureId: string) => void;
  onOpenModules?: (featureId: string) => void;
}

function toTitleCase(value: string) {
  return value.replace(/_/g, " ");
}

export default function CapabilityCard({
  capability,
  onOpen,
}: CapabilityCardProps) {
  return (
    <button
      type="button"
      onClick={() => onOpen(capability.feature_id)}
      className="w-full rounded-xl border border-[hsl(var(--border))] bg-[hsl(var(--card))] p-4
text-left transition-colors hover:border-[hsl(var(--primary))]"
    >
      <div className="flex items-start justify-between gap-4">
        <div className="min-w-0">
          <div className="flex flex-wrap items-center gap-2">
            <span className="text-xs font-mono text-[hsl(var(--muted-foreground))]">
              {capability.feature_id}
            </span>
            <h3 className="truncate text-base font-semibold text-[hsl(var(--card-foreground))]">
              {capability.title}
            </h3>
```

```
          </div>

          <div className="mt-2 flex flex-wrap gap-2">
            <CapabilityBadge
              label={capability.category}
              kind="category"
            />
            <CapabilityBadge
              label={capability.registry_status}
              kind="registry"
            />
            <CapabilityBadge
              label={capability.implementation_status}
              kind="implementation"
            />
            <CapabilityBadge
              label={capability.risk_level}
              kind="risk"
            />
          </div>
        </div>

        <div className="shrink-0 text-right">
          <div className="text-sm font-semibold text-[hsl(var(--foreground))]">
            {capability.readiness_score}
          </div>
          <div className="text-xs text-[hsl(var(--muted-foreground))]">
            {toTitleCase(capability.readiness_label)}
          </div>
        </div>
      </div>

      <p className="mt-3 line-clamp-2 text-sm text-[hsl(var(--muted-foreground))]">
        {capability.description}
      </p>

      <div className="mt-4 flex flex-wrap gap-2">
        <MetricChip label="modules" value={capability.src_modules.length} />
        <MetricChip label="gates" value={capability.gates.length} />
        <MetricChip label="deps" value={capability.dependencies.length} />
        <MetricChip label="blockers" value={capability.blocker_count}
tone={capability.blocker_count > 0 ? "bad" : "neutral"} />
        <MetricChip label="warnings" value={capability.warning_count}
tone={capability.warning_count > 0 ? "warn" : "neutral"} />
```

```
      </div>

      {capability.attention_flags.length > 0 ? (
        <div className="mt-4 flex flex-wrap gap-2">
          {capability.attention_flags.slice(0, 4).map((flag) => (
            <WarningPill
              key={flag}
              label={toTitleCase(flag)}
              severity={
                flag === "error_state" || flag === "unresolved_dependencies"
                  ? "error"
                  : "warn"
              }
            />
          ))}
        </div>
      ) : null}
    </button>
  );
}
```

16. App/src/components/capabilities/CapabilitiesEmptyState.tsx

```
export interface CapabilitiesEmptyStateProps {
  title: string;
  message: string;
  onReset?: () => void;
}

export default function CapabilitiesEmptyState({
  title,
  message,
  onReset,
}: CapabilitiesEmptyStateProps) {
  return (
    <div className="rounded-xl border border-dashed border-[hsl(var(--border))]
bg-[hsl(var(--card))] p-8 text-center">
```

```
      <h3 className="text-lg font-semibold text-[hsl(var(--foreground))]">{title}</h3>
      <p className="mt-2 text-sm text-[hsl(var(--muted-foreground))]">{message}</p>
      {onReset ? (
        <button
          type="button"
          onClick={onReset}
          className="mt-4 rounded-md border border-[hsl(var(--border))] px-3 py-2 text-sm
hover:border-[hsl(var(--primary))]"
        >
          Reset filters
        </button>
      ) : null}
    </div>
  );
}
```

17. App/src/components/capabilities/CapabilityBoard.tsx

```
import type { CapabilityRecord } from "@/types/capabilities";
import CapabilitiesEmptyState from "./CapabilitiesEmptyState";
import CapabilityCard from "./CapabilityCard";

export interface CapabilityBoardProps {
  capabilities: CapabilityRecord[];
  onOpenCapability: (featureId: string) => void;
  onResetFilters?: () => void;
}

export default function CapabilityBoard({
  capabilities,
  onOpenCapability,
  onResetFilters,
}: CapabilityBoardProps) {
  if (capabilities.length === 0) {
    return (
      <CapabilitiesEmptyState
        title="No capabilities found"
```

```
        message="No capability records match the current filters."
        onReset={onResetFilters}
      />
    );
  }

  return (
    <div className="space-y-3">
      {capabilities.map((capability) => (
        <CapabilityCard
          key={capability.feature_id}
          capability={capability}
          onOpen={onOpenCapability}
        />
      ))}
    </div>
  );
}
```

18. App/src/components/capabilities/AttentionPanel.tsx

```
import type { CapabilityWarnings } from "@/types/capabilities";

export interface AttentionPanelProps {
  warnings: CapabilityWarnings;
  onSelectSlice?: (slice: keyof CapabilityWarnings) => void;
}

function SliceCard({
  label,
  count,
  onClick,
}: {
  label: string;
  count: number;
  onClick?: () => void;
}) {
```

```
  return (
    <button
      type="button"
      onClick={onClick}
      className="rounded-lg border border-[hsl(var(--border))] bg-[hsl(var(--card))] px-4 py-3
text-left hover:border-[hsl(var(--primary))]"
    >
      <div className="text-lg font-semibold text-[hsl(var(--foreground))]">{count}</div>
      <div className="text-xs text-[hsl(var(--muted-foreground))]">{label}</div>
    </button>
  );
}

export default function AttentionPanel({
  warnings,
  onSelectSlice,
}: AttentionPanelProps) {
  return (
    <div className="grid grid-cols-2 gap-3 xl:grid-cols-4">
      <SliceCard
        label="Blocked"
        count={warnings.blocked.length}
        onClick={onSelectSlice ? () => onSelectSlice("blocked") : undefined}
      />
      <SliceCard
        label="Missing Modules"
        count={warnings.missingModules.length}
        onClick={onSelectSlice ? () => onSelectSlice("missingModules") : undefined}
      />
      <SliceCard
        label="Missing Gates"
        count={warnings.missingGates.length}
        onClick={onSelectSlice ? () => onSelectSlice("missingGates") : undefined}
      />
      <SliceCard
        label="Active but Incomplete"
        count={warnings.activeButIncomplete.length}
        onClick={onSelectSlice ? () => onSelectSlice("activeButIncomplete") : undefined}
      />
    </div>
  );
}
```

19. App/src/components/capabilities/CoverageCalloutRow.tsx

```
import type { CapabilityStats } from "@/types/capabilities";

export interface CoverageCalloutRowProps {
  stats: CapabilityStats;
  onSelectCallout?: (
    kind: "missingModules" | "missingGates" | "activeButIncomplete" | "highRisk"
  ) => void;
}

export default function CoverageCalloutRow({
  stats,
  onSelectCallout,
}: CoverageCalloutRowProps) {
  const items = [
    { key: "missingModules", label: "Missing Modules", value: stats.missingModules },
    { key: "missingGates", label: "Missing Gates", value: stats.missingGates },
    { key: "activeButIncomplete", label: "Active but Incomplete", value: stats.activeButIncomplete
},
    { key: "highRisk", label: "High Risk", value: stats.highRisk },
  ] as const;

  return (
    <div className="grid grid-cols-2 gap-3 xl:grid-cols-4">
      {items.map((item) => (
        <button
          key={item.key}
          type="button"
          onClick={onSelectCallout ? () => onSelectCallout(item.key) : undefined}
          className="rounded-lg border border-[hsl(var(--border))] bg-[hsl(var(--card))] px-4 py-3
text-left hover:border-[hsl(var(--primary))]"
        >
          <div className="text-sm font-semibold text-[hsl(var(--foreground))]">
            {item.value}
          </div>
          <div className="text-xs text-[hsl(var(--muted-foreground))]">
            {item.label}
          </div>
```

```
        </button>
      ))}
    </div>
  );
}
```

20. App/src/components/capabilities/CapabilityControlBar.tsx

```
import type {
  CapabilityFilterState,
  CapabilitySortMode,
} from "@/types/capabilities";

export interface CapabilityControlBarProps {
  searchQuery: string;
  filters: CapabilityFilterState;
  sortMode: CapabilitySortMode;
  onSearchChange: (value: string) => void;
  onFiltersChange: (next: CapabilityFilterState) => void;
  onPatchFilters: (patch: Partial<CapabilityFilterState>) => void;
  onSortChange: (mode: CapabilitySortMode) => void;
  onResetFilters: () => void;
}

export default function CapabilityControlBar({
  searchQuery,
  filters,
  sortMode,
  onSearchChange,
  onPatchFilters,
  onSortChange,
  onResetFilters,
}: CapabilityControlBarProps) {
  return (
    <div className="rounded-xl border border-[hsl(var(--border))] bg-[hsl(var(--card))] p-4">
      <div className="grid gap-3 lg:grid-cols-4">
        <input
```

```
          value={searchQuery}
          onChange={(e) => onSearchChange(e.target.value)}
          placeholder="Search capability, module, dependency, or gate"
          className="rounded-md border border-[hsl(var(--border))] bg-transparent px-3 py-2
text-sm outline-none"
        />

        <select
          value={filters.category}
          onChange={(e) => onPatchFilters({ category: e.target.value as
CapabilityFilterState["category"] })}
          className="rounded-md border border-[hsl(var(--border))] bg-transparent px-3 py-2
text-sm"
        >
          <option value="all">All Categories</option>
          <option value="infrastructure">Infrastructure</option>
          <option value="core-logic">Core Logic</option>
          <option value="interface">Interface</option>
          <option value="security">Security</option>
          <option value="other">Other</option>
        </select>

        <select
          value={filters.implementationStatus}
          onChange={(e) =>
            onPatchFilters({
              implementationStatus:
                e.target.value as CapabilityFilterState["implementationStatus"],
            })
          }
          className="rounded-md border border-[hsl(var(--border))] bg-transparent px-3 py-2
text-sm"
        >
          <option value="all">All Implementation States</option>
          <option value="implemented">Implemented</option>
          <option value="partial">Partial</option>
          <option value="stubbed">Stubbed</option>
          <option value="spec_only">Spec Only</option>
          <option value="blocked">Blocked</option>
          <option value="unverified">Unverified</option>
        </select>

        <select
          value={sortMode}
```

```
          onChange={(e) => onSortChange(e.target.value as CapabilitySortMode)}
          className="rounded-md border border-[hsl(var(--border))] bg-transparent px-3 py-2
text-sm"
        >
          <option value="readiness_desc">Sort: Readiness</option>
          <option value="risk_desc">Sort: Risk</option>
          <option value="module_count_desc">Sort: Modules</option>
          <option value="gate_count_desc">Sort: Gates</option>
          <option value="dependency_count_desc">Sort: Dependencies</option>
          <option value="title_asc">Sort: Title</option>
          <option value="feature_id_asc">Sort: Feature ID</option>
        </select>
      </div>

      <div className="mt-3 flex flex-wrap gap-2">
        <label className="text-xs text-[hsl(var(--muted-foreground))]">
          <input
            className="mr-2"
            type="checkbox"
            checked={filters.blockedOnly}
            onChange={(e) => onPatchFilters({ blockedOnly: e.target.checked })}
          />
          Blocked only
        </label>

        <label className="text-xs text-[hsl(var(--muted-foreground))]">
          <input
            className="mr-2"
            type="checkbox"
            checked={filters.incompleteOnly}
            onChange={(e) => onPatchFilters({ incompleteOnly: e.target.checked })}
          />
          Incomplete only
        </label>

        <label className="text-xs text-[hsl(var(--muted-foreground))]">
          <input
            className="mr-2"
            type="checkbox"
            checked={filters.missingModulesOnly}
            onChange={(e) => onPatchFilters({ missingModulesOnly: e.target.checked })}
          />
          Missing modules
        </label>
```

```
        <label className="text-xs text-[hsl(var(--muted-foreground))]">
          <input
            className="mr-2"
            type="checkbox"
            checked={filters.missingGatesOnly}
            onChange={(e) => onPatchFilters({ missingGatesOnly: e.target.checked })}
          />
          Missing gates
        </label>

        <button
          type="button"
          onClick={onResetFilters}
          className="ml-auto rounded-md border border-[hsl(var(--border))] px-3 py-1.5 text-xs
hover:border-[hsl(var(--primary))]"
        >
          Reset
        </button>
      </div>
    </div>
  );
}
```

21. App/src/components/capabilities/CapabilityTabs.tsx

```
import type { CapabilityTabId } from "@/types/capabilities";

export interface CapabilityTabsProps {
  activeTab: CapabilityTabId;
  onTabChange: (tab: CapabilityTabId) => void;
  disabledTabs?: CapabilityTabId[];
}

const ALL_TABS: CapabilityTabId[] = [
  "overview",
  "readiness",
```

```
  "dependencies",
  "gates",
  "modules",
  "risk",
];

export default function CapabilityTabs({
  activeTab,
  onTabChange,
  disabledTabs = [],
}: CapabilityTabsProps) {
  return (
    <div className="flex flex-wrap gap-2">
      {ALL_TABS.map((tab) => {
        const disabled = disabledTabs.includes(tab);
        const active = activeTab === tab;

        return (
          <button
            key={tab}
            type="button"
            disabled={disabled}
            onClick={() => onTabChange(tab)}
            className={`rounded-full px-3 py-1.5 text-sm transition-colors ${
              active
                ? "bg-[hsl(var(--primary))] text-[hsl(var(--primary-foreground))]"
                : "border border-[hsl(var(--border))] bg-[hsl(var(--card))]
text-[hsl(var(--muted-foreground))]"
            } ${disabled ? "cursor-not-allowed opacity-40" : "hover:border-[hsl(var(--primary))]"}`}
          >
            {tab.replace("_", " ")}
          </button>
        );
      })}
    </div>
  );
}
```

22. App/src/components/capabilities/panels/OverviewPanel.tsx

```
import type {
  CapabilityRecord,
  CapabilityStats,
  CapabilityWarnings,
} from "@/types/capabilities";
import AttentionPanel from "../AttentionPanel";
import CapabilityBoard from "../CapabilityBoard";
import CoverageCalloutRow from "../CoverageCalloutRow";

export interface OverviewPanelProps {
  capabilities: CapabilityRecord[];
  stats: CapabilityStats;
  warnings: CapabilityWarnings;
  onOpenCapability: (featureId: string) => void;
  onResetFilters?: () => void;
}

export default function OverviewPanel({
  capabilities,
  stats,
  warnings,
  onOpenCapability,
  onResetFilters,
}: OverviewPanelProps) {
  return (
    <div className="space-y-4">
      <AttentionPanel warnings={warnings} />
      <CoverageCalloutRow stats={stats} />
      <CapabilityBoard
        capabilities={capabilities}
        onOpenCapability={onOpenCapability}
        onResetFilters={onResetFilters}
      />
    </div>
  );
}
```

23. App/src/pages/capabilities.tsx

```
import { Loader2 } from "lucide-react";
import { useLocation } from "wouter";
import CapabilitiesPageHeader from "@/components/capabilities/CapabilitiesPageHeader";
import CapabilityControlBar from "@/components/capabilities/CapabilityControlBar";
import CapabilityStatStrip from "@/components/capabilities/CapabilityStatStrip";
import CapabilityTabs from "@/components/capabilities/CapabilityTabs";
import OverviewPanel from "@/components/capabilities/panels/OverviewPanel";
import { useCapabilitiesPage } from "@/hooks/capabilities/useCapabilitiesPage";

export default function CapabilitiesPage() {
  const [, setLocation] = useLocation();

  const {
    isLoading,
    error,
    activeTab,
    setActiveTab,
    searchQuery,
    setSearchQuery,
    filters,
    setFilters,
    patchFilters,
    resetFilters,
    sortMode,
    setSortMode,
    model,
  } = useCapabilitiesPage();

  function handleOpenCapability(featureId: string) {
    setLocation(`/features/${featureId}`);
  }

  return (
    <div className="space-y-6">
      <CapabilitiesPageHeader
        title="Capability Control Center"
        subtitle="Capability readiness, dependency health, and implementation coverage"
      />

      <CapabilityStatStrip stats={model.stats} />
```

```
      <CapabilityControlBar
        searchQuery={searchQuery}
        filters={filters}
        sortMode={sortMode}
        onSearchChange={setSearchQuery}
        onFiltersChange={setFilters}
        onPatchFilters={patchFilters}
        onSortChange={setSortMode}
        onResetFilters={resetFilters}
      />

      <CapabilityTabs
        activeTab={activeTab}
        onTabChange={setActiveTab}
        disabledTabs={["readiness", "dependencies", "gates", "modules", "risk"]}
      />

      {isLoading ? (
        <div className="flex justify-center py-12">
          <Loader2 className="h-8 w-8 animate-spin text-[hsl(var(--muted-foreground))]" />
        </div>
      ) : error ? (
        <div className="rounded-xl border border-red-500/30 bg-red-500/10 p-4 text-sm
text-red-300">
          Failed to load capabilities.
        </div>
      ) : (
        <OverviewPanel
          capabilities={model.filtered}
          stats={model.stats}
          warnings={model.warnings}
          onOpenCapability={handleOpenCapability}
          onResetFilters={resetFilters}
        />
      )}
    </div>
  );
}
```

24. App/src/pages/features.tsx

```
export { default } from "./capabilities";
```

Important cleanup note

One issue in the skeleton above: in `deriveImplementationStatus.ts`, I referenced `DeriveImplementationStatusArgs` from the shared types import while also declaring it locally. Keep only the local interface, or move that interface into `types/capabilities.ts` and import it from there. Do not do both.

What this gives you immediately

After wiring these files, the page stops being:

- a hardcoded **Feature Registry**
- a fixed **Total / Active** board
- a category-only stacked list driven by raw registry status

And becomes:

- a **Capability Control Center**
- derived implementation/readiness/risk model
- search/filter/sortable overview
- honesty-first card system

n.

cleanup pass pack

Cleanup Pass Pack

1. Fix duplicate type declarations

In the starter skeleton, a few helper arg/result interfaces were declared inside the helper files. That is fine for drafting, but for the real build you should centralize them.

Move into

`App/src/types/capabilities.ts`

Move these

- `DeriveImplementationStatusArgs`
- `DeriveReadinessScoreArgs`
- `DeriveReadinessScoreResult`
- `CollectAttentionFlagsArgs`
- `CollectAttentionFlagsResult`
- `DeriveCapabilityRecordArgs`
- `BuildCapabilityPageModelArgs`
- `UseCapabilitiesPageModelArgs`
- `UseCapabilitiesPageResult`

Why

This prevents:

- duplicated contracts

- import drift
- helper files redefining the same shape differently later

---

2. Normalize import style

Pick one import convention and keep it everywhere.

Recommended

Use alias imports consistently:

import type { CapabilityRecord } from "@/types/capabilities";

Avoid mixing

- `@/types/capabilities`
- `../../types/capabilities`
- local redeclarations

Why

The page rewrite is already splitting one monolith into many files. Mixed imports will make maintenance worse fast.

---

3. Remove unused props until needed

A few props were included for future expansion. That is fine, but in the real first pass you should trim anything you are not using yet.

Keep for now

- `onOpen` on `CapabilityCard`

- `onResetFilters` where needed
- `disabledTabs` on `CapabilityTabs`

Remove until later

- `onOpenDeps`
- `onOpenGates`
- `onOpenModules`
- `onFiltersChange` if you only use patch-based updates in first pass

Why

The first pass should stay lean.
Future-proofing is good, but not at the cost of noisy interfaces.

---

4. Tighten the page entry strategy

Keep `features.tsx` as a thin alias for now.

Use

export { default } from "./capabilities";

Why this is better right now

- existing routing likely still points at `/features`
- detail pages already use `/features/:feature_id`
- you avoid breaking nav while still separating the new page implementation

Later, if you rename the route to `/capabilities`, do that as a dedicated routing pass.

---

5. Add label maps instead of raw enum display

Right now some components display raw values directly.

Add to `types/capabilities.ts` or a small constants file

```
export const CAPABILITY_CATEGORY_LABELS = {
 infrastructure: "Infrastructure",
 "core-logic": "Core Logic",
 interface: "Interface",
 security: "Security",
 other: "Other",
} as const;

export const REGISTRY_STATUS_LABELS = {
 active: "Active",
 draft: "Draft",
 error: "Error",
 deprecated: "Deprecated",
} as const;

export const IMPLEMENTATION_STATUS_LABELS = {
 implemented: "Implemented",
 partial: "Partial",
 stubbed: "Stubbed",
 spec_only: "Spec Only",
 blocked: "Blocked",
 unverified: "Unverified",
} as const;

export const READINESS_LABELS = {
 ready: "Ready",
 near_ready: "Near Ready",
 partial: "Partial",
 fragile: "Fragile",
 blocked: "Blocked",
} as const;
```

Why

This prevents UI text from drifting into raw underscore strings.

---

6. Add small formatting helpers

Put these in:
`App/src/lib/capabilities/formatters.ts`

Include

- `formatCapabilityLabel`
- `formatAttentionFlag`
- `formatRiskLevel`

Example

```
export function formatTokenLabel(value: string): string {
 return value
   .replace(/_/g, " ")
   .replace(/\b\w/g, (char) => char.toUpperCase());
}
```

Why

Right now `CapabilityCard` does small formatting inline. Pull that out so cards stay focused on rendering.

---

7. Stabilize sort behavior

The current sort helper is fine for a first draft, but it should add a tie-breaker.

Add this fallback

After primary sort comparison:

1. `title asc`

2. `feature_id asc`

Why

Without a stable tie-breaker, cards can shuffle around unnecessarily as filters change.

---

8. Make search defensive

The search helper is already broad, which is good. Tighten it slightly.

Ensure it safely handles

- empty strings
- undefined arrays
- repeated whitespace

Recommended

Normalize search text:

const q = searchQuery.trim().toLowerCase();

Why

This avoids false misses and keeps filtering predictable.

---

9. Tighten risk escalation logic

The draft risk logic is usable, but the reverse-dependency escalation is a bit too blunt.

Change

Instead of:

- low -> moderate

- anything else -> critical

Use explicit escalation:

- `0–1 reverse deps` → no change
- `2–4 reverse deps` → raise one band if already weak
- `5+ reverse deps` → at least high
- `8+ reverse deps` → critical

Why

You want risk to feel credible, not jumpy.

---

10. Tighten readiness scoring assumptions

The initial readiness score is okay for phase 1, but add one correction:

Add penalty

- if unresolved dependencies > 0, subtract meaningful weight or force low ceiling

Reason

A capability should not score “near ready” if it is dependency-blocked.

Simple fix

After computing score:

```
if (unresolvedDependencyCount > 0) {
 score = Math.min(score, 25);
}
```

---

11. Clean up warning generation

Right now warnings and attention flags overlap heavily, which is okay early, but you should separate them more cleanly.

Rule

- `attention_flags` = internal diagnostic signals
- `warnings` = user-facing display warnings

Example

Keep:

- `active_without_modules` as an attention flag
  Display:
- `no_modules` as a warning

Why

That keeps internal rule vocabulary from leaking directly into UI.

---

12. Guard against weak raw records

In `normalizeFeature`, add stronger defaults.

Recommended defaults

title: feature.title?.trim() || feature.feature_id || "Untitled Capability"
description: feature.description?.trim() || ""
dependencies: Array.isArray(feature.dependencies) ? feature.dependencies : []
src_modules: Array.isArray(feature.src_modules) ? feature.src_modules : []
gates: Array.isArray(feature.gates) ? feature.gates : []

Why

Registry data is often uneven. The model layer should absorb that mess, not the UI.

---

13. Make the board layout responsive early

The first skeleton used a vertical stack. That is safe, but I would move to a simple responsive grid immediately.

Recommended

- single column on small screens
- two columns on xl if card density still reads well

Why

This page is a control surface now, not a long registry log.

---

14. Disable fake interactivity

If a stat card, callout, or attention slice does not actually filter yet, do not make it feel clickable.

Apply to

- stat cards
- attention cards
- coverage callouts

Why

Dead-click UI hurts trust fast.

Only add hover/click styling when the interaction is wired.

---

15. Keep tabs honest

For first pass, Overview is the only real tab.

Recommended behavior

- show other tabs
- disable them visually
- optionally add a small “next phase” tooltip or muted state

Do not

Render empty panels for unfinished tabs.

Why

That makes the page feel unfinished instead of intentionally phased.

---

16. Add lightweight error and loading components

Do not leave those inline in the page forever.

Create

- `CapabilitiesLoadingState.tsx`
- `CapabilitiesErrorState.tsx`

Why

Even if tiny, they clean up `capabilities.tsx` and keep the page shell readable.

---

17. Keep the first-pass file boundary strict

Do not let first-pass files start pulling in:

- dependency graph views

- gate detail drawers
- module drilldowns
- risk explainer panels

Reason

That is how first-pass scope balloons and the page drifts again.

For this pass, the page's only job is:

- truth-based overview
- credible capability cards
- usable filtering

---

18. Recommended cleaned file set

After cleanup, first pass should look like this:

```
App/src/
 pages/
   capabilities.tsx
   features.tsx

 components/
   capabilities/
     CapabilitiesPageHeader.tsx
     CapabilitiesLoadingState.tsx
     CapabilitiesErrorState.tsx
     CapabilityStatStrip.tsx
     CapabilityStatCard.tsx
     CapabilityControlBar.tsx
     CapabilityTabs.tsx
     CapabilityBoard.tsx
     CapabilityCard.tsx
     CapabilityBadge.tsx
     MetricChip.tsx
```

```
    WarningPill.tsx
    AttentionPanel.tsx
    CoverageCalloutRow.tsx
    CapabilitiesEmptyState.tsx
    panels/
      OverviewPanel.tsx

hooks/
  capabilities/
    useCapabilitiesPage.ts
    useCapabilitiesPageModel.ts

lib/
  capabilities/
    buildCapabilityPageModel.ts
    deriveCapabilityRecord.ts
    deriveImplementationStatus.ts
    deriveReadinessScore.ts
    collectAttentionFlags.ts
    formatters.ts

types/
  capabilities.ts
```

---

19. First-pass cleanup completion check

This cleanup pass is done when:

- helper arg/result interfaces are centralized
- imports are consistent
- raw enum strings are not displayed directly
- dead-click UI is removed
- loading/error states are extracted
- the Overview page feels intentional, not drafty

---

20. Strong recommendation

Before building the second wave of tabs, do one verification pass on the Overview against real `/api/features` output.

You want to check:

- how many records get marked implemented
- how many get marked spec_only
- whether active-but-empty records are being exposed correctly
- whether the warnings feel believable

That is the moment where you will learn whether the page is finally telling the truth.

Capability Control Center — First-Pass

Capability Control Center — First-Pass Verification Checklist

1. Page identity verification

Confirm the page no longer presents itself as the old registry page.

Check

- page title is `Capability Control Center`
- subtitle is dynamic or at least no longer the old fixed registry sentence
- the page no longer uses “Feature Registry” language
- stat strip is no longer only `Total / Active`

Pass condition

The page visually reads as a control surface, not a registry browser.

---

2. Routing verification

Confirm the page is wired safely into the existing app.

Check

- `/features` still opens the new page
- clicking a capability still routes to `/features/:feature_id`
- no broken nav links were introduced
- alias page strategy works if `features.tsx` is forwarding to `capabilities.tsx`

Pass condition

The page rewrite does not break existing routing behavior.

---

3. Query/data verification

Confirm the page still loads from the same source cleanly.

Check

- `/api/features` is still the query source
- loading state appears correctly
- error state appears correctly if the query fails
- empty data does not crash the page
- missing fields in feature records do not break rendering

Pass condition

The page is stable across:

- loading
- error
- empty result
- partial/messy records

---

4. Model derivation verification

Confirm raw registry records are being transformed correctly.

Check each derived field on several real records

- `registry_status`
- `implementation_status`
- `readiness_score`
- `readiness_label`
- `risk_level`
- `dependency_health`
- `gate_health`
- `module_health`
- `attention_flags`
- `warnings`

Pass condition

No card is relying on raw `status` alone to imply implementation completeness.

---

5. Truth separation verification

This is the most important verification pass.

Check

Find at least one capability in each of these categories:

- registry active but weakly implemented
- draft with some code linkage
- spec-only

- blocked due to unresolved dependency
- active with no modules
- active with no gates

Pass condition

The page clearly distinguishes:

- registered
- implemented
- partial
- blocked
- unverified
- spec only

If “active” still visually reads as “done,” this pass failed.

---

6. Readiness scoring sanity check

Confirm readiness scores are believable.

Check

For a sample of records:

- active + modules + gates + no unresolved deps → should score high
- active + no modules → should score weak
- unresolved dependencies → should not score near-ready
- no modules + no gates → should land as spec-only / fragile

- error state → should not look healthy

Pass condition

Scores feel directionally correct and do not give false confidence.

---

7. Warning integrity verification

Confirm warnings are surfacing the real problems.

Check

Look for records that should trigger:

- `no_modules`
- `no_gates`
- `dependency_blocked`
- `weak_verification`
- `marked_active_but_incomplete`

Pass condition

Warnings are:

- present when they should be
- absent when they should be
- not duplicated excessively
- understandable in UI

---

8. Attention panel verification

Confirm the top issue zone is credible.

Check

`AttentionPanel` counts should match actual records for:

- blocked
- missing modules
- missing gates
- active but incomplete

Pass condition

The attention counts line up with the actual capability board data.

If the top counts and the card list disagree, the model pipeline is off.

---

9. Stat strip verification

Confirm the summary cards are derived from the full dataset, not just the filtered list.

Check

- changing filters should not unexpectedly rewrite the global summary unless you intentionally designed it that way
- totals should stay internally consistent
- implemented/partial/blocked/unverified/highRisk numbers should match raw derived counts

Pass condition

The stat strip is stable and mathematically consistent.

---

10. Search verification

Confirm the search behaves like an operator tool, not title-only search.

Check search against:

- feature ID
- title
- description
- category
- module names
- dependency IDs
- gate IDs

Pass condition

Search finds relevant cards from any of those fields.

---

11. Filter verification

Confirm each filter does exactly what it says.

Check

- category filter
- implementation status filter
- blocked only
- incomplete only

- missing modules only
- missing gates only

Pass condition

Each filter narrows the board correctly and predictably.

Extra check

Stack multiple filters together and confirm the result still makes sense.

---

12. Sort verification

Confirm sorting is stable and understandable.

Check

- readiness sort
- risk sort
- module count sort
- gate count sort
- dependency count sort
- title sort
- feature ID sort

Pass condition

Cards reorder correctly and do not jump unpredictably on ties.

---

13. Empty state verification

Confirm zero-result conditions are graceful.

Check

Apply restrictive filters until no capability matches.

Pass condition

The page shows:

- a clean empty state
- a reset path
- no broken layout
- no ghost cards or stale counts in the board area

---

14. Card verification

Confirm the new card actually communicates more truth than the old one.

Check each card shows:

- feature ID
- title
- category
- registry status
- implementation status
- risk
- readiness score
- description

- modules/gates/deps metrics
- warnings if applicable

Pass condition

A user can understand the capability's real state from the card without needing the detail page.

---

15. No fake interactivity verification

Confirm the first pass is honest about what is interactive.

Check

- disabled tabs do not feel clickable
- stat cards only feel clickable if wired
- attention slices only feel clickable if wired
- callouts only feel clickable if wired

Pass condition

Nothing invites clicks that do nothing.

---

16. Disabled tabs verification

Confirm phased tabs are visually honest.

Check

- Overview is enabled and working
- Readiness / Dependencies / Gates / Modules / Risk are visibly disabled or clearly not live yet

- disabled tabs do not render blank panels

Pass condition

The page feels intentionally phased, not half-broken.

---

17. Visual hierarchy verification

Confirm the facelift actually changed the page feel.

Check

- header reads as a control center
- stat strip feels operational
- control bar is visible and useful
- attention panel stands out
- cards are denser and more informative
- long registry-scroll feeling is reduced

Pass condition

The page feels like an Axion command surface, not a static docs page.

---

18. Real-data credibility check

This is the key final test.

Review a sample of real `/api/features` records and ask:

- does the page expose how incomplete the system really is?

- does it reveal spec-only items clearly?
- does it stop overstating “active” capabilities?
- does it make weak implementation visible?

Pass condition

The page now tells the truth better than the old one.

If it still flatters the system too much, refine:

- implementation status rules
- readiness scoring
- warning thresholds

---

19. Regression check against the old page

Compare old vs new.

Old page behavior

- grouped by category
- total/active counts
- raw registry status
- shallow metadata display

New page should clearly improve

- implementation truth
- risk/readiness visibility

- warning visibility
- operator filtering
- attention focus

Pass condition

The new page is not just prettier. It is more honest and more useful.

---

20. Ship gate for first pass

First pass is ready to move on only when all of these are true:

- page identity is corrected
- route is stable
- query and fallback states are stable
- derived model is functioning
- overview is useful on real data
- active is no longer confused with implemented
- filters/search/sort work
- warnings feel credible
- disabled tabs are honest
- the page feels like a capability board, not a registry

Readiness Tab

Readiness Tab

1. Goal

Expose the real implementation state of every capability.

The Readiness tab must answer:

- what is actually implemented
- what is partially built
- what is only spec-level
- what is blocked
- what is unverified
- what is most ready to move forward

This is the tab that directly fixes the “most of these features are not implemented” problem.

---

2. Readiness tab identity

Tab label

Readiness

Tab purpose

A truth-based implementation board grouped by derived implementation state instead of registry category.

Core shift

The current page organizes by category like:

- infrastructure
- core-logic
- interface
- security

The Readiness tab should instead organize by:

- Implemented
- Partial
- Stubbed
- Spec Only
- Blocked
- Unverified

That is the important reframe.

---

3. Readiness panel structure

Top-level layout

- `ReadinessPanel`
  - `ReadinessSummaryRow`
  - `ReadinessLegend`
  - `ReadinessColumns`
    - `ReadinessColumn implemented`
    - `ReadinessColumn partial`
    - `ReadinessColumn stubbed`
    - `ReadinessColumn spec_only`
    - `ReadinessColumn blocked`
    - `ReadinessColumn unverified`

First-pass display style

Use a multi-column board on large screens and a stacked lane layout on smaller screens.

---

4. ReadinessSummaryRow

Purpose

Give immediate counts by implementation truth.

Cards

- Implemented

- Partial
- Stubbed
- Spec Only
- Blocked
- Unverified

Optional extra metrics

- average readiness score
- active but not implemented
- implemented but registry still draft

Why this matters

The current page only exposes Total and Active at the top level.

features

This row becomes the first place where the system's real implementation shape is visible.

---

5. ReadinessLegend

Purpose

Explain how readiness states should be interpreted.

Contents

- **Implemented** — modules and gates exist, no unresolved dependency blockers
- **Partial** — some implementation surface exists, but not enough to count as ready
- **Stubbed** — lightweight implementation shell exists, but capability is not materially built
- **Spec Only** — registry/spec exists, no real implementation signal
- **Blocked** — unresolved dependency or structural blocker prevents readiness
- **Unverified** — implementation exists but verification/gate coverage is weak or missing

This should stay compact, but it is important because the new status layer is central to the redesign.

---

6. ReadinessColumns

Purpose

Group capabilities by implementation state, not by architectural category.

Required order

Use this exact order:

1. Implemented
2. Partial
3. Stubbed
4. Spec Only
5. Blocked
6. Unverified

Why this order

It reads from strongest implementation truth to weakest/problematic truth.

Column content

Each column contains:

- column header
- count
- optional short definition
- repeated `CapabilityCardCompact`

---

7. ReadinessColumn spec

`ReadinessColumn`

Responsibilities

- render one implementation-state lane
- show count
- render compact cards
- render empty state if no items in that lane

Props

```
import type {
 CapabilityRecord,
 ImplementationStatus,
} from "@/types/capabilities";

export interface ReadinessColumnProps {
 status: ImplementationStatus;
 label: string;
 description?: string;
 capabilities: CapabilityRecord[];
 onOpenCapability: (featureId: string) => void;
}
```

Header content

- label
- count badge
- optional description

Empty state text

Examples:

- “No implemented capabilities.”
- “No spec-only capabilities.”
- “No blocked capabilities.”

---

8. Compact card spec

`CapabilityCardCompact`

Purpose

Provide a denser card for lane-based scanning.

The main `CapabilityCard` is still useful for Overview.
The Readiness tab should use a more compact scan-friendly card.

Required content

- feature ID

- title
- registry status
- readiness score
- risk level
- small metrics row
- top warning line if any

Props

```
import type { CapabilityRecord } from "@/types/capabilities";

export interface CapabilityCardCompactProps {
 capability: CapabilityRecord;
 onOpen: (featureId: string) => void;
}
```

Compact layout

- top row: title + feature id
- second row: registry badge + risk badge + readiness score
- third row: modules / gates / deps
- fourth row: top warning if present

Why compact here

The Readiness tab is about scanning many records fast.

---

9. Readiness data contract

The tab should consume already-derived records from the model layer, not invent any new derivation in the panel.

Required input

`groupedByReadiness: Record<ImplementationStatus, CapabilityRecord[]>`

Add to page model

Update `CapabilityPageModel` to include:

```
groupedByReadiness: Record<ImplementationStatus, CapabilityRecord[]>;
```

Update builder

`buildCapabilityPageModel.ts` should now also build readiness grouping.

---

10. Grouping rules

Implemented

`implementation_status === "implemented"`

Partial

`implementation_status === "partial"`

Stubbed

`implementation_status === "stubbed"`

Spec Only

`implementation_status === "spec_only"`

Blocked

`implementation_status === "blocked"`

Unverified

`implementation_status === "unverified"`

Rule

No panel-level regrouping.
This grouping must come from the model layer.

---

11. Readiness tab filtering behavior

The global control bar should still apply.

Filters should affect the Readiness tab

- search
- category
- registry status
- implementation status
- risk
- blocked only
- incomplete only
- missing modules only
- missing gates only

Result

The tab shows filtered records grouped by readiness state.

Important note

Even when filtered, empty lanes should remain visible if the layout still benefits from scan consistency.
You can mute empty lanes instead of removing them entirely.

---

12. Recommended UX behavior

Click behavior

Clicking a compact card should still open:
`/features/:feature_id`

features

Hover behavior

Subtle emphasis only.
This is a scan board, not a marketing dashboard.

Warning emphasis

Highlight:

- active but low readiness
- blocked
- unverified
- no modules
- no gates

These are the exact cases this tab is supposed to expose.

---

13. Readiness-specific summary rules

Add these counts to the tab-level summary row:

- implemented count
- partial count
- stubbed count
- spec-only count
- blocked count
- unverified count

Optional later:

- “active but not implemented”
- “draft but implementation exists”

Those two are especially useful because they expose misalignment between registry and implementation.

---

14. New helper contracts

`buildGroupedByReadiness`

File:

App/src/lib/capabilities/buildGroupedByReadiness.ts

Contract

```
import type {
 CapabilityRecord,
 ImplementationStatus,
```

```
} from "@/types/capabilities";

export function buildGroupedByReadiness(
 records: CapabilityRecord[]
): Record<ImplementationStatus, CapabilityRecord[]> {
 return {
   implemented: records.filter((r) => r.implementation_status === "implemented"),
   partial: records.filter((r) => r.implementation_status === "partial"),
   stubbed: records.filter((r) => r.implementation_status === "stubbed"),
   spec_only: records.filter((r) => r.implementation_status === "spec_only"),
   blocked: records.filter((r) => r.implementation_status === "blocked"),
   unverified: records.filter((r) => r.implementation_status === "unverified"),
 };
}
```

---

15. New component file pack for Pass 8

Create:

App/src/components/capabilities/panels/ReadinessPanel.tsx
App/src/components/capabilities/ReadinessSummaryRow.tsx
App/src/components/capabilities/ReadinessColumn.tsx
App/src/components/capabilities/CapabilityCardCompact.tsx
App/src/lib/capabilities/buildGroupedByReadiness.ts

Update:

App/src/types/capabilities.ts
App/src/lib/capabilities/buildCapabilityPageModel.ts
App/src/hooks/capabilities/useCapabilitiesPageModel.ts
App/src/pages/capabilities.tsx
App/src/components/capabilities/CapabilityTabs.tsx

---

16. `ReadinessPanel` prop contract

```
import type {
 CapabilityRecord,
 CapabilityStats,
 CapabilityPageModel,
 ImplementationStatus,
} from "@/types/capabilities";
```

```
export interface ReadinessPanelProps {
 groupedByReadiness: Record<ImplementationStatus, CapabilityRecord[]>;
 stats: CapabilityStats;
 onOpenCapability: (featureId: string) => void;
}
```

Responsibilities

- render summary row
- render legend
- render readiness columns

---

17. `ReadinessSummaryRow` prop contract

```
export interface ReadinessSummaryRowProps {
 counts: {
   implemented: number;
   partial: number;
   stubbed: number;
   spec_only: number;
   blocked: number;
   unverified: number;
 };
}
```

Display

Use compact stat tiles similar to the top summary strip, but scoped to readiness-state counts.

---

18. `CapabilityCardCompact` display rules

Badge stack

Show:

- registry status
- risk level

Score display

Show:

- readiness score
- readiness label

Metric strip

Show:

- modules
- gates
- deps

Warning line

Show only the most important one:

- blocked dependency
- no modules
- no gates
- active but incomplete

Do not show the full warning pill stack here unless density still reads cleanly.

---

19. Page wiring for Pass 8

Update `capabilities.tsx` so the tab switch is real.

Replace

Overview-only render

With

```
{isLoading ? (
 <CapabilitiesLoadingState />
) : error ? (
 <CapabilitiesErrorState />
) : activeTab === "overview" ? (
 <OverviewPanel
   capabilities={model.filtered}
   stats={model.stats}
   warnings={model.warnings}
```

```
  onOpenCapability={handleOpenCapability}
  onResetFilters={resetFilters}
 />
) : activeTab === "readiness" ? (
 <ReadinessPanel
  groupedByReadiness={model.groupedByReadiness}
  stats={model.stats}
  onOpenCapability={handleOpenCapability}
 />
) : null}
```

Tab update

Enable `readiness` in `CapabilityTabs`.
Keep the others disabled for now.

---

20. Build order for Pass 8

Build in this order:

1. update App/src/types/capabilities.ts
2. add App/src/lib/capabilities/buildGroupedByReadiness.ts
3. update App/src/lib/capabilities/buildCapabilityPageModel.ts
4. update App/src/hooks/capabilities/useCapabilitiesPageModel.ts
5. add App/src/components/capabilities/CapabilityCardCompact.tsx
6. add App/src/components/capabilities/ReadinessColumn.tsx
7. add App/src/components/capabilities/ReadinessSummaryRow.tsx
8. add App/src/components/capabilities/panels/ReadinessPanel.tsx
9. update App/src/components/capabilities/CapabilityTabs.tsx
10. update App/src/pages/capabilities.tsx

---

21. Pass 8 completion criteria

Pass 8 is done when:

- Readiness tab is enabled
- Readiness tab groups by implementation truth, not category
- Implemented / Partial / Stubbed / Spec Only / Blocked / Unverified are all visible
- Compact cards scan cleanly
- Clicking a readiness card still opens the feature detail page

- Global filters/search/sort still apply
- The page clearly exposes how many capabilities are only planned or weakly built

That last one is the real success condition.

---

22. Why Pass 8 matters

Overview gives you attention and filtering.
Readiness gives you **truth structure**.

The current page can tell you a capability is “active” and show counts for modules, gates, and dependencies, but it still leaves too much room for registry optimism.

features

The Readiness tab fixes that by making the main question:
“How real is this capability?”

Pass 8 starter prop pack

Pass 8 — Readiness Tab Starter Prop Pack

1. Type additions

Update:

App/src/types/capabilities.ts

Add readiness grouping support:

```
export interface ReadinessCounts {
 implemented: number;
 partial: number;
 stubbed: number;
 spec_only: number;
 blocked: number;
 unverified: number;
}
```

Extend the page model:

```
export interface CapabilityPageModel {
 raw: Feature[];
 derived: CapabilityRecord[];
 filtered: CapabilityRecord[];
 stats: CapabilityStats;
 warnings: CapabilityWarnings;
 groupedByReadiness: Record<ImplementationStatus, CapabilityRecord[]>;
}
```

---

2. `ReadinessPanel` props

File:

App/src/components/capabilities/panels/ReadinessPanel.tsx

```
import type {
 CapabilityRecord,
 CapabilityStats,
```

```
 ImplementationStatus,
} from "@/types/capabilities";

export interface ReadinessPanelProps {
 groupedByReadiness: Record<ImplementationStatus, CapabilityRecord[]>;
 stats: CapabilityStats;
 onOpenCapability: (featureId: string) => void;
}
```

Responsibility

- render the tab shell
- render readiness summary row
- render readiness legend
- render readiness columns

Rule

This panel does not derive or regroup anything.

---

3. `ReadinessSummaryRow` props

File:

App/src/components/capabilities/ReadinessSummaryRow.tsx

```
import type { ReadinessCounts } from "@/types/capabilities";

export interface ReadinessSummaryRowProps {
 counts: ReadinessCounts;
 onSelectState?: (state: keyof ReadinessCounts) => void;
}
```

Responsibility

Render compact readiness counts for:

- implemented

- partial
- stubbed
- spec only
- blocked
- unverified

Rule

`onSelectState` stays optional for planning flexibility.

---

4. `ReadinessLegend` props

You can either inline this inside `ReadinessPanel` or make it a tiny dedicated component.
I recommend a dedicated component so the panel stays cleaner.

File:

App/src/components/capabilities/ReadinessLegend.tsx

```
export interface ReadinessLegendProps {
 compact?: boolean;
}
```

Responsibility

Explain the meaning of each readiness state.

Rule

No data dependency needed in first pass.

---

5. `ReadinessColumn` props

File:

App/src/components/capabilities/ReadinessColumn.tsx

```
import type {
 CapabilityRecord,
 ImplementationStatus,
} from "@/types/capabilities";

export interface ReadinessColumnProps {
 status: ImplementationStatus;
 label: string;
 description?: string;
 capabilities: CapabilityRecord[];
 onOpenCapability: (featureId: string) => void;
}
```

Responsibility

- render one readiness lane
- show count
- show optional explanatory copy
- render compact cards
- render lane-empty state when needed

---

6. `CapabilityCardCompact` props

File:

App/src/components/capabilities/CapabilityCardCompact.tsx

```
import type { CapabilityRecord } from "@/types/capabilities";

export interface CapabilityCardCompactProps {
 capability: CapabilityRecord;
 onOpen: (featureId: string) => void;
}
```

Responsibility

Render a denser scan-friendly version of a capability card for lane layouts.

Rule

This card displays existing derived values only.

---

7. `ReadinessLaneEmptyState` props

Optional but useful for keeping columns explicit even when empty.

File:

```
App/src/components/capabilities/ReadinessLaneEmptyState.tsx
export interface ReadinessLaneEmptyStateProps {
 title: string;
 message?: string;
}
```

Responsibility

Show a lightweight empty state inside a readiness column.

---

8. `buildGroupedByReadiness` helper contract

File:

```
App/src/lib/capabilities/buildGroupedByReadiness.ts
import type {
 CapabilityRecord,
 ImplementationStatus,
} from "@/types/capabilities";

export interface BuildGroupedByReadinessResult {
 implemented: CapabilityRecord[];
 partial: CapabilityRecord[];
 stubbed: CapabilityRecord[];
 spec_only: CapabilityRecord[];
 blocked: CapabilityRecord[];
 unverified: CapabilityRecord[];
}
```

```
export type BuildGroupedByReadiness = (
 records: CapabilityRecord[]
) => Record<ImplementationStatus, CapabilityRecord[]>;
```

Responsibility

Produce readiness lanes from already-filtered, already-sorted records.

---

9. `buildReadinessCounts` helper contract

File:

```
App/src/lib/capabilities/buildReadinessCounts.ts
import type {
 CapabilityRecord,
 ReadinessCounts,
} from "@/types/capabilities";

export type BuildReadinessCounts = (
 records: CapabilityRecord[]
) => ReadinessCounts;
```

Responsibility

Produce summary counts for the Readiness tab header row.

Rule

Counts should be based on the same filtered readiness dataset used by the tab.

---

10. `CapabilityTabs` prop update

No new prop shape required unless you want badge counts later.

Current shape is enough:

```
import type { CapabilityTabId } from "@/types/capabilities";

export interface CapabilityTabsProps {
```

```
 activeTab: CapabilityTabId;
 onTabChange: (tab: CapabilityTabId) => void;
 disabledTabs?: CapabilityTabId[];
}
```

Planning note

For Pass 8:

- `overview` enabled
- `readiness` enabled
- the rest disabled

---

11. Page wiring contract update

The page container does not need a new prop type, but it does need new model expectations:

```
const {
 isLoading,
 error,
 activeTab,
 setActiveTab,
 searchQuery,
 setSearchQuery,
 filters,
 setFilters,
 patchFilters,
 resetFilters,
 sortMode,
 setSortMode,
 model,
} = useCapabilitiesPage();
```

And `model` must now include:

```
model.groupedByReadiness
```

---

12. `useCapabilitiesPageModel` output update

File:

App/src/hooks/capabilities/useCapabilitiesPageModel.ts

Existing return type can stay `CapabilityPageModel`, but that model must now include readiness grouping.

No new hook prop contract is required beyond the type extension.

---

13. Display token helpers for Readiness

Add label maps so the tab does not rely on raw enum strings.

```
export const READINESS_STATE_LABELS: Record<ImplementationStatus, string> = {
 implemented: "Implemented",
 partial: "Partial",
 stubbed: "Stubbed",
 spec_only: "Spec Only",
 blocked: "Blocked",
 unverified: "Unverified",
};
```

Optional descriptions:

```
export const READINESS_STATE_DESCRIPTIONS: Record<ImplementationStatus, string> = {
 implemented: "Operationally built with implementation and verification signals.",
 partial: "Some implementation exists, but readiness is incomplete.",
 stubbed: "A light shell exists, but the capability is not materially built.",
 spec_only: "Defined in registry/spec, but lacks implementation evidence.",
 blocked: "Blocked by unresolved dependency or structural issue.",
 unverified: "Implementation exists, but verification coverage is weak.",
};
```

---

14. Recommended file list for Pass 8 planning

App/src/components/capabilities/panels/ReadinessPanel.tsx
App/src/components/capabilities/ReadinessSummaryRow.tsx
App/src/components/capabilities/ReadinessLegend.tsx
App/src/components/capabilities/ReadinessColumn.tsx

App/src/components/capabilities/CapabilityCardCompact.tsx
App/src/components/capabilities/ReadinessLaneEmptyState.tsx
App/src/lib/capabilities/buildGroupedByReadiness.ts
App/src/lib/capabilities/buildReadinessCounts.ts
App/src/types/capabilities.ts

---

15. Pass 8 prop-pack completion check

This prop pack is complete when:

- the readiness panel contract is locked
- readiness summary counts are typed
- readiness columns are typed
- compact cards are typed
- helper outputs are typed
- page model contract includes readiness grouping

Pass 8 — Readiness Tab Starter Code Skeleton

Pass 8 — Readiness Tab Starter Code Skeleton Pack

1. `App/src/types/capabilities.ts`

Add the readiness-specific types and extend the page model.

```
export interface ReadinessCounts {
 implemented: number;
 partial: number;
 stubbed: number;
 spec_only: number;
 blocked: number;
 unverified: number;
}

export const READINESS_STATE_LABELS: Record<ImplementationStatus, string> = {
 implemented: "Implemented",
 partial: "Partial",
 stubbed: "Stubbed",
 spec_only: "Spec Only",
 blocked: "Blocked",
 unverified: "Unverified",
};

export const READINESS_STATE_DESCRIPTIONS: Record<ImplementationStatus, string> = {
 implemented: "Operationally built with implementation and verification signals.",
 partial: "Some implementation exists, but readiness is incomplete.",
 stubbed: "A light shell exists, but the capability is not materially built.",
 spec_only: "Defined in registry/spec, but lacks implementation evidence.",
 blocked: "Blocked by unresolved dependency or structural issue.",
 unverified: "Implementation exists, but verification coverage is weak.",
};

export interface CapabilityPageModel {
 raw: Feature[];
 derived: CapabilityRecord[];
 filtered: CapabilityRecord[];
 stats: CapabilityStats;
 warnings: CapabilityWarnings;
```

```
 groupedByReadiness: Record<ImplementationStatus, CapabilityRecord[]>;
}
```

---

2. App/src/lib/capabilities/buildGroupedByReadiness.ts

```
import type {
 CapabilityRecord,
 ImplementationStatus,
} from "@/types/capabilities";

export function buildGroupedByReadiness(
 records: CapabilityRecord[]
): Record<ImplementationStatus, CapabilityRecord[]> {
 return {
   implemented: records.filter((r) => r.implementation_status === "implemented"),
   partial: records.filter((r) => r.implementation_status === "partial"),
   stubbed: records.filter((r) => r.implementation_status === "stubbed"),
   spec_only: records.filter((r) => r.implementation_status === "spec_only"),
   blocked: records.filter((r) => r.implementation_status === "blocked"),
   unverified: records.filter((r) => r.implementation_status === "unverified"),
 };
}
```

---

3. App/src/lib/capabilities/buildReadinessCounts.ts

```
import type {
 CapabilityRecord,
 ReadinessCounts,
} from "@/types/capabilities";

export function buildReadinessCounts(
 records: CapabilityRecord[]
): ReadinessCounts {
 return {
   implemented: records.filter((r) => r.implementation_status === "implemented").length,
```

```
  partial: records.filter((r) => r.implementation_status === "partial").length,
  stubbed: records.filter((r) => r.implementation_status === "stubbed").length,
  spec_only: records.filter((r) => r.implementation_status === "spec_only").length,
  blocked: records.filter((r) => r.implementation_status === "blocked").length,
  unverified: records.filter((r) => r.implementation_status === "unverified").length,
 };
}
```

---

4. App/src/components/capabilities/CapabilityCardCompact.tsx

```
import type { CapabilityRecord } from "@/types/capabilities";
import CapabilityBadge from "./CapabilityBadge";

export interface CapabilityCardCompactProps {
 capability: CapabilityRecord;
 onOpen: (featureId: string) => void;
}

export default function CapabilityCardCompact({
 capability,
 onOpen,
}: CapabilityCardCompactProps) {
 return (
  <button
   type="button"
   onClick={() => onOpen(capability.feature_id)}
   className="w-full rounded-lg border border-[hsl(var(--border))] bg-[hsl(var(--card))] p-3
text-left hover:border-[hsl(var(--primary))]"
  >
   <div className="flex items-start justify-between gap-3">
    <div className="min-w-0">
     <div className="text-xs font-mono text-[hsl(var(--muted-foreground))]">
      {capability.feature_id}
     </div>
     <div className="truncate text-sm font-semibold text-[hsl(var(--foreground))]">
      {capability.title}
     </div>
    </div>

    <div className="text-right">
```

```
        <div className="text-sm font-semibold text-[hsl(var(--foreground))]">
          {capability.readiness_score}
        </div>
        <div className="text-[11px] text-[hsl(var(--muted-foreground))]">
          {capability.readiness_label}
        </div>
      </div>
    </div>

    <div className="mt-2 flex flex-wrap gap-2">
      <CapabilityBadge label={capability.registry_status} kind="registry" />
      <CapabilityBadge label={capability.risk_level} kind="risk" />
    </div>

    <div className="mt-3 flex flex-wrap gap-2 text-[11px] text-[hsl(var(--muted-foreground))]">
      <span>{capability.src_modules.length} modules</span>
      <span>{capability.gates.length} gates</span>
      <span>{capability.dependencies.length} deps</span>
    </div>

    {capability.warnings.length > 0 ? (
      <div className="mt-2 text-[11px] text-yellow-300">
        {capability.warnings[0]}
      </div>
    ) : null}
  </button>
 );
}
```

---

5. App/src/components/capabilities/ReadinessLaneEmptyState.tsx

```
export interface ReadinessLaneEmptyStateProps {
 title: string;
 message?: string;
}

export default function ReadinessLaneEmptyState({
 title,
 message,
}: ReadinessLaneEmptyStateProps) {
```

```
 return (
   <div className="rounded-lg border border-dashed border-[hsl(var(--border))]
bg-[hsl(var(--card))] p-4">
     <div className="text-sm font-medium text-[hsl(var(--foreground))]">{title}</div>
     {message ? (
       <div className="mt-1 text-xs text-[hsl(var(--muted-foreground))]">
         {message}
       </div>
     ) : null}
   </div>
 );
}
```

---

6. App/src/components/capabilities/ReadinessColumn.tsx

```
import type {
 CapabilityRecord,
 ImplementationStatus,
} from "@/types/capabilities";
import CapabilityCardCompact from "./CapabilityCardCompact";
import ReadinessLaneEmptyState from "./ReadinessLaneEmptyState";

export interface ReadinessColumnProps {
 status: ImplementationStatus;
 label: string;
 description?: string;
 capabilities: CapabilityRecord[];
 onOpenCapability: (featureId: string) => void;
}

export default function ReadinessColumn({
 label,
 description,
 capabilities,
 onOpenCapability,
}: ReadinessColumnProps) {
 return (
   <section className="space-y-3 rounded-xl border border-[hsl(var(--border))]
bg-[hsl(var(--card))] p-4">
     <div>
```

```
      <div className="flex items-center justify-between gap-3">
        <h3 className="text-sm font-semibold text-[hsl(var(--foreground))]">{label}</h3>
        <span className="rounded-full border border-[hsl(var(--border))] px-2 py-0.5 text-xs
text-[hsl(var(--muted-foreground))]">
          {capabilities.length}
        </span>
      </div>
      {description ? (
        <p className="mt-1 text-xs text-[hsl(var(--muted-foreground))]">
          {description}
        </p>
      ) : null}
    </div>

    {capabilities.length === 0 ? (
      <ReadinessLaneEmptyState
        title={`No ${label.toLowerCase()} capabilities`}
      />
    ) : (
      <div className="space-y-2">
        {capabilities.map((capability) => (
          <CapabilityCardCompact
            key={capability.feature_id}
            capability={capability}
            onOpen={onOpenCapability}
          />
        ))}
      </div>
    )}
  </section>
 );
}
```

---

7. App/src/components/capabilities/ReadinessSummaryRow.tsx

```
import type { ReadinessCounts } from "@/types/capabilities";
import CapabilityStatCard from "./CapabilityStatCard";

export interface ReadinessSummaryRowProps {
 counts: ReadinessCounts;
```

```
 onSelectState?: (state: keyof ReadinessCounts) => void;
}

export default function ReadinessSummaryRow({
 counts,
 onSelectState,
}: ReadinessSummaryRowProps) {
 return (
   <div className="grid grid-cols-2 gap-3 md:grid-cols-3 xl:grid-cols-6">
     <CapabilityStatCard
       label="Implemented"
       value={counts.implemented}
       tone="good"
       onClick={onSelectState ? () => onSelectState("implemented") : undefined}
     />
     <CapabilityStatCard
       label="Partial"
       value={counts.partial}
       tone="warn"
       onClick={onSelectState ? () => onSelectState("partial") : undefined}
     />
     <CapabilityStatCard
       label="Stubbed"
       value={counts.stubbed}
       tone="warn"
       onClick={onSelectState ? () => onSelectState("stubbed") : undefined}
     />
     <CapabilityStatCard
       label="Spec Only"
       value={counts.spec_only}
       tone="default"
       onClick={onSelectState ? () => onSelectState("spec_only") : undefined}
     />
     <CapabilityStatCard
       label="Blocked"
       value={counts.blocked}
       tone="bad"
       onClick={onSelectState ? () => onSelectState("blocked") : undefined}
     />
     <CapabilityStatCard
       label="Unverified"
       value={counts.unverified}
       tone="warn"
       onClick={onSelectState ? () => onSelectState("unverified") : undefined}
```

```
    />
  </div>
 );
}
```

---

8. App/src/components/capabilities/ReadinessLegend.tsx

```
export interface ReadinessLegendProps {
 compact?: boolean;
}

const ITEMS = [
 ["Implemented", "Operationally built with implementation and verification signals."],
 ["Partial", "Some implementation exists, but readiness is incomplete."],
 ["Stubbed", "A light shell exists, but the capability is not materially built."],
 ["Spec Only", "Defined in registry/spec, but lacks implementation evidence."],
 ["Blocked", "Blocked by unresolved dependency or structural issue."],
 ["Unverified", "Implementation exists, but verification coverage is weak."],
] as const;

export default function ReadinessLegend({
 compact = false,
}: ReadinessLegendProps) {
 return (
   <div className="rounded-xl border border-[hsl(var(--border))] bg-[hsl(var(--card))] p-4">
     <h3 className="text-sm font-semibold text-[hsl(var(--foreground))]">
       Readiness Legend
     </h3>
     <div className={`mt-3 grid gap-3 ${compact ? "md:grid-cols-2" : "lg:grid-cols-3"}`}>
       {ITEMS.map(([label, description]) => (
         <div key={label}>
           <div className="text-xs font-medium text-[hsl(var(--foreground))]">{label}</div>
           <div className="mt-1 text-xs text-[hsl(var(--muted-foreground))]">
             {description}
           </div>
         </div>
       ))}
     </div>
   </div>
 );
```

```
}
```

---

9. App/src/components/capabilities/panels/ReadinessPanel.tsx

```
import type {
 CapabilityRecord,
 CapabilityStats,
 ImplementationStatus,
} from "@/types/capabilities";
import {
 READINESS_STATE_DESCRIPTIONS,
 READINESS_STATE_LABELS,
} from "@/types/capabilities";
import { buildReadinessCounts } from "@/lib/capabilities/buildReadinessCounts";
import ReadinessColumn from "../ReadinessColumn";
import ReadinessLegend from "../ReadinessLegend";
import ReadinessSummaryRow from "../ReadinessSummaryRow";

export interface ReadinessPanelProps {
 groupedByReadiness: Record<ImplementationStatus, CapabilityRecord[]>;
 stats: CapabilityStats;
 onOpenCapability: (featureId: string) => void;
}

const READINESS_ORDER: ImplementationStatus[] = [
 "implemented",
 "partial",
 "stubbed",
 "spec_only",
 "blocked",
 "unverified",
];

export default function ReadinessPanel({
 groupedByReadiness,
 onOpenCapability,
}: ReadinessPanelProps) {
 const counts = buildReadinessCounts(
   READINESS_ORDER.flatMap((status) => groupedByReadiness[status] ?? [])
 );
```

```
 return (
   <div className="space-y-4">
     <ReadinessSummaryRow counts={counts} />
     <ReadinessLegend />

     <div className="grid gap-4 xl:grid-cols-3">
       {READINESS_ORDER.map((status) => (
         <ReadinessColumn
           key={status}
           status={status}
           label={READINESS_STATE_LABELS[status]}
           description={READINESS_STATE_DESCRIPTIONS[status]}
           capabilities={groupedByReadiness[status] ?? []}
           onOpenCapability={onOpenCapability}
         />
       ))}
     </div>
   </div>
 );
}
```

---

10. App/src/lib/capabilities/buildCapabilityPageModel.ts

Update the existing model builder.

```
import { buildGroupedByReadiness } from "./buildGroupedByReadiness";

// inside buildCapabilityPageModel(...)
const filtered = sortRecords(applyFilters(derived, filters, searchQuery), sortMode);
const warnings = buildWarnings(derived);
const groupedByReadiness = buildGroupedByReadiness(filtered);

return {
 raw,
 derived,
 filtered,
 stats,
 warnings,
```

```
 groupedByReadiness,
};
```

Planning rule

Use `filtered` for readiness grouping, not `derived`, so the Readiness tab respects the active global filters.

---

11. `App/src/pages/capabilities.tsx`

Update the page render switch.

```
import ReadinessPanel from "@/components/capabilities/panels/ReadinessPanel";

// inside render:
<CapabilityTabs
 activeTab={activeTab}
 onTabChange={setActiveTab}
 disabledTabs={["dependencies", "gates", "modules", "risk"]}
/>

{isLoading ? (
 <CapabilitiesLoadingState />
) : error ? (
 <CapabilitiesErrorState />
) : activeTab === "overview" ? (
 <OverviewPanel
   capabilities={model.filtered}
   stats={model.stats}
   warnings={model.warnings}
   onOpenCapability={handleOpenCapability}
   onResetFilters={resetFilters}
 />
) : activeTab === "readiness" ? (
 <ReadinessPanel
   groupedByReadiness={model.groupedByReadiness}
   stats={model.stats}
   onOpenCapability={handleOpenCapability}
 />
) : null}
```

---

12. Pass 8 file pack

By the end of this planning slice, Pass 8 should have these files defined:

App/src/components/capabilities/panels/ReadinessPanel.tsx
App/src/components/capabilities/ReadinessSummaryRow.tsx
App/src/components/capabilities/ReadinessLegend.tsx
App/src/components/capabilities/ReadinessColumn.tsx
App/src/components/capabilities/CapabilityCardCompact.tsx
App/src/components/capabilities/ReadinessLaneEmptyState.tsx
App/src/lib/capabilities/buildGroupedByReadiness.ts
App/src/lib/capabilities/buildReadinessCounts.ts

And these updated:

App/src/types/capabilities.ts
App/src/lib/capabilities/buildCapabilityPageModel.ts
App/src/pages/capabilities.tsx

---

13. Pass 8 planning completion check

Pass 8 planning is complete when you have:

- readiness prop pack
- readiness starter code skeleton pack
- readiness cleanup pass
- readiness verification checklist

Pass 8 — Readiness Tab Cleanup Pass Pack

Pass 8 — Readiness Tab Cleanup Pass Pack

1. Keep readiness truth centralized

The biggest cleanup rule for this tab:

Readiness UI must never derive readiness truth on its own.

Keep all truth in the model/helper layer

- `implementation_status`
- `readiness_score`
- `readiness_label`
- `risk_level`
- `warnings`
- `groupedByReadiness`

Do not compute inside

- `ReadinessPanel`
- `ReadinessColumn`
- `CapabilityCardCompact`

Why

If lane components start re-deriving “implemented” or “blocked,” the tab will drift from the Overview and create inconsistent truth across the page.

---

2. Keep grouping based on filtered records

For this tab, `groupedByReadiness` should be built from the **filtered + sorted** dataset, not from all derived records.

Rule

- top page stat strip = global/full derived dataset
- readiness lane grouping = active filtered dataset

Why

The tab should reflect the current control-bar state.
If the user filters to `category = interface`, the Readiness tab should only show readiness lanes for interface capabilities.

---

3. Separate global stats from tab stats

The top stat strip and the Readiness summary row should not be confused.

Keep distinct

Global stat strip

- total
- implemented
- partial
- blocked
- unverified
- high risk

Readiness summary row

- implemented
- partial
- stubbed
- spec only
- blocked
- unverified

Why

The readiness row is tab-scoped and truth-structured.
The page stat strip is page-scoped and cross-tab.

---

4. Add a dedicated readiness formatter

Do not let compact cards print raw enum strings directly.

Add

App/src/lib/capabilities/formatReadinessLabel.ts

Role

Map:

- `spec_only` → `Spec Only`
- `near_ready` → `Near Ready`
- `active_but_low_readiness` → `Active but Incomplete`

Why

The compact card is small. Raw enum text will make it feel rough fast.

---

5. Clean up compact card warning output

The starter skeleton shows the first warning directly. Tighten that.

Replace raw display

Do not print:

- `dependency_blocked`
- `no_modules`

as raw code strings.

Use a warning label map

Example:

```
export const WARNING_LABELS = {
 no_modules: "No linked modules",
 no_gates: "No gate coverage",
 dependency_blocked: "Blocked by dependency",
 weak_verification: "Weak verification",
 marked_active_but_incomplete: "Active but incomplete",
 implementation_unknown: "Implementation unknown",
 orphan_registry_entry: "Orphan registry entry",
 dependency_unknown: "Dependency unknown",
 high_risk_dependency_chain: "High-risk dependency chain",
} as const;
```

Why

This tab is meant for fast scanning. Raw warning codes weaken credibility.

---

6. Tighten readiness column order and keep it locked

Do not let lane order float or sort alphabetically.

Lock this order

1. Implemented
2. Partial
3. Stubbed
4. Spec Only
5. Blocked
6. Unverified

Why

This order carries meaning:

- strongest implementation truth first
- weakest/problem cases later

If the order changes by count or alphabetically, the board becomes harder to scan.

---

7. Keep lanes visible even when empty

Do not hide empty readiness states in first pass.

Recommended behavior

Show the lane with:

- title
- count = 0
- compact empty state

Why

The tab is a truth board.
Seeing that there are zero blocked or zero implemented capabilities is useful information.

---

8. Tighten lane density

The Readiness tab is scan-first, so each lane should stay compact.

Keep in `CapabilityCardCompact`

- feature ID
- title
- registry badge
- risk badge
- readiness score
- modules/gates/deps
- one warning line max

Do not include

- long descriptions
- full warning pill stacks
- multiple action rows
- dependency lists
- gate detail rows

Why

That belongs in Overview, Dependencies, Gates, and Modules later.
Readiness should stay lightweight and comparative.

---

9. Do not overload the legend

The legend should explain the states, not become documentation.

Keep

- one line per state
- short operational definition

Avoid

- long paragraphs
- scoring explanations
- rule logic explanations
- dependency algorithm notes

Why

The legend is support text, not the main content.

---

10. Normalize lane empty-state language

Keep the empty-state copy short and consistent.

Recommended pattern

- “No implemented capabilities.”
- “No partial capabilities.”
- “No blocked capabilities.”

Optional second line:

- “Nothing in the current filter set matches this readiness state.”

Why

This keeps the tab readable and consistent when filters narrow results.

---

11. Make compact card labels consistent with main card

The compact card and full card should not disagree in terminology.

Align labels across both

- registry status names
- implementation status names
- risk labels
- warning labels
- readiness labels

Why

A capability should not look “Spec Only” in one tab and “spec_only” or “Stub” elsewhere.

---

12. Keep tab-specific interactions honest

For first pass, the Readiness tab should support only one primary action:

- open capability detail

Optional later

- click readiness stat to filter to lane
- click warning line to apply filter
- click badge to pivot tab

For now

Do not style these as interactive unless they really are.

Why

No fake interactivity.

---

13. Keep tab-level summary count source explicit

Decide this now and lock it:

Recommended

`ReadinessSummaryRow` counts should come from the **same filtered dataset** used by the lanes.

Why

That makes the summary row and lanes agree exactly.

If you mix filtered lanes with global counts, the tab becomes confusing.

---

14. Update the page model contract cleanly

Do not keep adding tab-specific fields loosely.

Keep `CapabilityPageModel` structured

- global fields

- filtered board fields
- tab-grouped fields

Recommended organization

```
export interface CapabilityPageModel {
 raw: Feature[];
 derived: CapabilityRecord[];
 filtered: CapabilityRecord[];

 stats: CapabilityStats;
 warnings: CapabilityWarnings;

 groupedByReadiness: Record<ImplementationStatus, CapabilityRecord[]>;
}
```

Why

The page model should stay easy to scan as more tabs are added.

---

15. Add a readiness constants file if the type file gets too heavy

If `types/capabilities.ts` starts getting overloaded, move display constants out.

Move these to

App/src/lib/capabilities/constants.ts

Candidates

- `READINESS_STATE_LABELS`
- `READINESS_STATE_DESCRIPTIONS`
- `WARNING_LABELS`

Why

Keep `types` focused on contracts and shapes.
Keep display tokens in a constants layer.

---

16. Keep styling distinct from Overview

The Readiness tab should feel denser and more board-like than Overview.

Overview

- broad control surface
- richer cards
- attention-first

Readiness

- lane-based
- denser cards
- implementation-truth-first

Why

If both tabs feel visually identical, the tab switch loses value.

---

17. Add responsive layout guidance now

Lock the intended responsive behavior in planning.

Recommended

Small screens

- stacked lanes, one per row

Medium screens

- two-column lane grid

Large screens

- three-column lane grid

Why

Six full lanes in a single row will usually be too cramped.

---

18. Keep `stats` optional in `ReadinessPanel` unless used

The current prop contract includes `stats`, but the starter skeleton does not really need it if counts are built from grouped records.

Cleanup choice

Either:

- remove `stats` from `ReadinessPanelProps`

or

- explicitly use it for a small cross-check/banner

Recommendation

Remove it for first pass.

Why

Unused props create confusion and invite drift.

---

19. Tighten `buildReadinessCounts`

There are two clean ways to calculate readiness counts:

- from filtered flat records
- from grouped readiness lanes

Better choice

Build counts from `groupedByReadiness`

Why

That guarantees exact agreement between:

- lane counts
- summary row counts

Cleaner contract

```
export function buildReadinessCounts(
 grouped: Record<ImplementationStatus, CapabilityRecord[]>
): ReadinessCounts
```

That is better than rebuilding counts from the flat list again.

---

20. Add a planning rule for future tab interactions

Do not let Readiness become a substitute for later tabs.

Readiness should answer

- how real is this capability?

Not

- why exactly is it blocked? → Dependencies tab
- what verification is missing? → Gates tab

- which modules are absent? → Modules tab
- how dangerous is change impact? → Risk tab

Why

This protects later tab scope and keeps Readiness focused.

---

21. Recommended cleaned Pass 8 file set

```
App/src/components/capabilities/panels/ReadinessPanel.tsx
App/src/components/capabilities/ReadinessSummaryRow.tsx
App/src/components/capabilities/ReadinessLegend.tsx
App/src/components/capabilities/ReadinessColumn.tsx
App/src/components/capabilities/CapabilityCardCompact.tsx
App/src/components/capabilities/ReadinessLaneEmptyState.tsx
App/src/lib/capabilities/buildGroupedByReadiness.ts
App/src/lib/capabilities/buildReadinessCounts.ts
App/src/lib/capabilities/constants.ts        // optional but recommended
App/src/lib/capabilities/formatters.ts       // if not already created
App/src/types/capabilities.ts
```

---

22. Pass 8 cleanup completion check

Pass 8 cleanup is done when:

- readiness truth is fully model-driven
- grouped lanes use filtered records
- lane order is locked
- empty lanes remain visible
- compact card output uses friendly labels
- unused props are removed
- tab-level counts and lane counts agree

- the Readiness tab remains focused on implementation truth only

Pass 8 — Readiness Tab Verification Checklist

Pass 8 — Readiness Tab Verification Checklist

1. Tab activation verification

Confirm the Readiness tab is now a real live tab and not a placeholder.

Check

- `overview` is enabled
- `readiness` is enabled
- `dependencies`, `gates`, `modules`, `risk` remain disabled
- selecting `readiness` renders the readiness board
- selecting `overview` still returns to the overview board cleanly

Pass condition

The page now has two intentional working views:

- Overview
- Readiness

---

2. Truth-model verification

Confirm the Readiness tab is using the derived implementation truth model, not raw registry status.

Check

Each lane is based on:

- `implementation_status`

and not on:

- `status`
- `category`

Pass condition

A capability's lane placement is determined only by derived implementation truth.

If a capability is showing in "Implemented" only because it is registry `active`, this pass fails.

---

3. Lane grouping verification

Confirm the readiness lanes are correct and exhaustive.

Required lanes

- Implemented
- Partial
- Stubbed
- Spec Only
- Blocked
- Unverified

Check

- every filtered capability appears in exactly one lane
- no capability appears in multiple lanes

- no capability disappears from the readiness board
- lane order is fixed and matches the planned sequence

Pass condition

The Readiness board is a complete one-pass partition of the filtered dataset.

---

4. Filter alignment verification

Confirm the Readiness tab respects the global control bar.

Check

Apply:

- search query
- category filter
- registry status filter
- implementation status filter
- risk filter
- blocked only
- incomplete only
- missing modules only
- missing gates only

Then confirm:

- lane contents update correctly
- lane counts update correctly

- empty lanes remain visible
- only matching records stay in the readiness board

Pass condition

The Readiness tab reflects the active filtered dataset exactly.

---

5. Summary-row alignment verification

Confirm the Readiness summary row and the lane counts agree.

Check

For each readiness state:

- summary card count
- visible lane count

must match exactly for:

- implemented
- partial
- stubbed
- spec only
- blocked
- unverified

Pass condition

There is zero mismatch between:

- readiness summary row

- readiness columns

If counts disagree, the grouping/count pipeline is wrong.

---

6. Compact-card verification

Confirm the compact card is dense but still readable.

Each compact card should clearly show

- feature ID
- title
- registry status
- risk level
- readiness score
- modules count
- gates count
- dependency count
- one top warning if present

Check

- cards are scan-friendly
- cards do not feel overloaded
- no long descriptions are shown
- no duplicated warning stacks
- labels are human-readable, not raw enums

Pass condition

A user can scan lane contents quickly without opening detail pages.

---

7. Friendly-label verification

Confirm no raw enum/code strings leak into the Readiness tab.

Check for leaks like

- `spec_only`
- `near_ready`
- `dependency_blocked`
- `active_but_low_readiness`

Pass condition

UI labels are friendly and formatted, such as:

- Spec Only
- Near Ready
- Blocked by dependency
- Active but incomplete

If raw code strings appear in cards or lanes, this pass fails.

---

8. Empty-lane verification

Confirm empty readiness states remain visible and useful.

Check

Use filters that produce:

- zero implemented
- zero blocked
- zero spec-only
  or other empty lanes

Pass condition

Each empty lane still renders:

- lane header
- count = 0
- compact empty state copy

The tab should not collapse lanes out of existence.

---

9. Lane-order verification

Confirm readiness lane order never drifts.

Required locked order

1. Implemented
2. Partial
3. Stubbed
4. Spec Only
5. Blocked

6. Unverified

Check

- lane order does not change with counts
- lane order does not change with filtering
- lane order is not alphabetical

Pass condition

The board always reads in the intended implementation-truth sequence.

---

10. Readiness-meaning verification

Confirm each lane's meaning stays clean.

Check sample records and validate:

- **Implemented** = materially built and not dependency-blocked
- **Partial** = some implementation exists, but still incomplete
- **Stubbed** = thin shell only
- **Spec Only** = registry/spec exists, no implementation signals
- **Blocked** = unresolved dependency or structural blocker
- **Unverified** = implementation exists, verification is weak

Pass condition

The lanes feel semantically distinct.

If "Partial" and "Stubbed" are visually or logically indistinguishable, this needs tuning.

---

11. Misalignment exposure verification

This is one of the highest-value checks.

Find records that show:

- active registry status but low readiness
- active with no modules
- active with no gates
- draft with implementation signals
- blocked but still “looks active” in registry

Pass condition

The Readiness tab makes those misalignments obvious.

If it still flatters the registry too much, the tab is not doing its job.

---

12. Warning-signal verification

Confirm warning lines in compact cards reinforce the lane truth.

Check that top warnings surface the right problem

Examples:

- Blocked by dependency
- No linked modules
- No gate coverage
- Active but incomplete
- Weak verification

Pass condition

The warning line helps explain why the capability sits in that lane.

The warning line should support the lane, not confuse it.

---

13. Visual differentiation verification

Confirm Readiness feels distinct from Overview.

Overview should feel like

- attention-first
- broader operational dashboard
- richer cards

Readiness should feel like

- lane-based scan board
- denser cards
- implementation-truth grouping

Pass condition

Switching tabs changes the lens, not just the layout.

If Readiness feels like Overview with smaller cards, refine the presentation.

---

14. Responsiveness verification

Confirm the lane layout holds up across viewport sizes.

Check

- small screens: stacked lanes
- medium screens: two-column layout
- large screens: multi-column layout
- compact cards remain readable at every size
- no lane becomes too cramped to scan

Pass condition

The board remains readable and stable across breakpoints.

---

15. Single-source-of-truth verification

Confirm no readiness logic is being recomputed inside display components.

Check

These components should only render:

- `ReadinessPanel`
- `ReadinessColumn`
- `CapabilityCardCompact`
- `ReadinessSummaryRow`

Verify they do not derive

- implementation status
- readiness score
- risk level

- grouped lane membership

Pass condition

All readiness truth originates in the model/helper layer.

---

16. Interaction-honesty verification

Confirm the Readiness tab does not promise interactions it does not support yet.

Check

- compact cards are clickable if they open detail
- summary cards only feel clickable if filtering is wired
- lane headers do not imply interaction unless real
- legend items do not look interactive

Pass condition

No fake interactivity exists.

---

17. Detail-route verification

Confirm the Readiness tab still uses the existing feature detail path safely.

Check

Clicking a compact card opens:

- `/features/:feature_id`

Pass condition

Readiness integrates cleanly with the existing feature detail route.

---

18. Real-data credibility verification

This is the main outcome test.

Review actual readiness distribution and ask:

- does the board reveal how many capabilities are only on paper?
- does it reveal how many are partial?
- does it clearly expose blocked items?
- does it stop “active” from being mistaken for “implemented”?
- does it highlight weakly verified implementation?

Pass condition

The tab makes the implementation truth more obvious than the old category-based feature list ever did.

---

19. Boundary verification

Confirm Readiness does not absorb the jobs of future tabs.

It should answer

- how real is this capability?

It should not fully answer

- exactly what dependency chain is blocking it → Dependencies tab
- exactly which gates are missing → Gates tab

- exactly which modules are absent → Modules tab
- exact blast radius analysis → Risk tab

Pass condition

The Readiness tab stays focused and does not become bloated.

---

20. Pass 8 ship gate

Pass 8 planning is successfully realized when all of these are true:

- Readiness is a live tab
- lanes are grouped by implementation truth
- lane counts and summary counts match
- filters/search affect readiness correctly
- empty lanes remain visible
- compact cards scan cleanly
- raw enum strings do not leak into UI
- active vs implemented misalignment is exposed clearly
- Readiness feels distinct from Overview
- the tab remains focused on implementation truth only

At that point, **Pass 8 planning is complete**.